



ECOLE
POLYTECHNIQUE
DE BRUXELLES

04. From 3D to 2D

INFO-H502

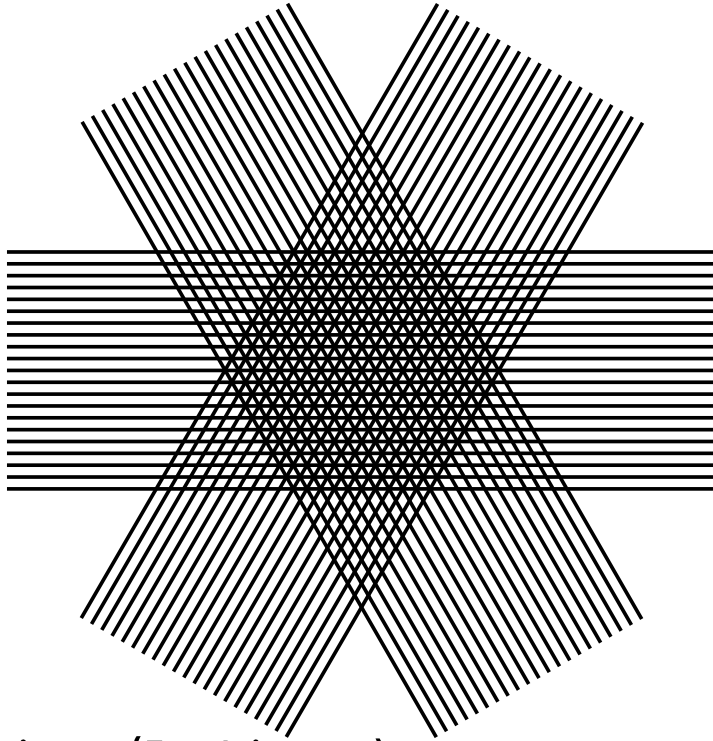
Daniele Bonatto

LISA-VR

2024

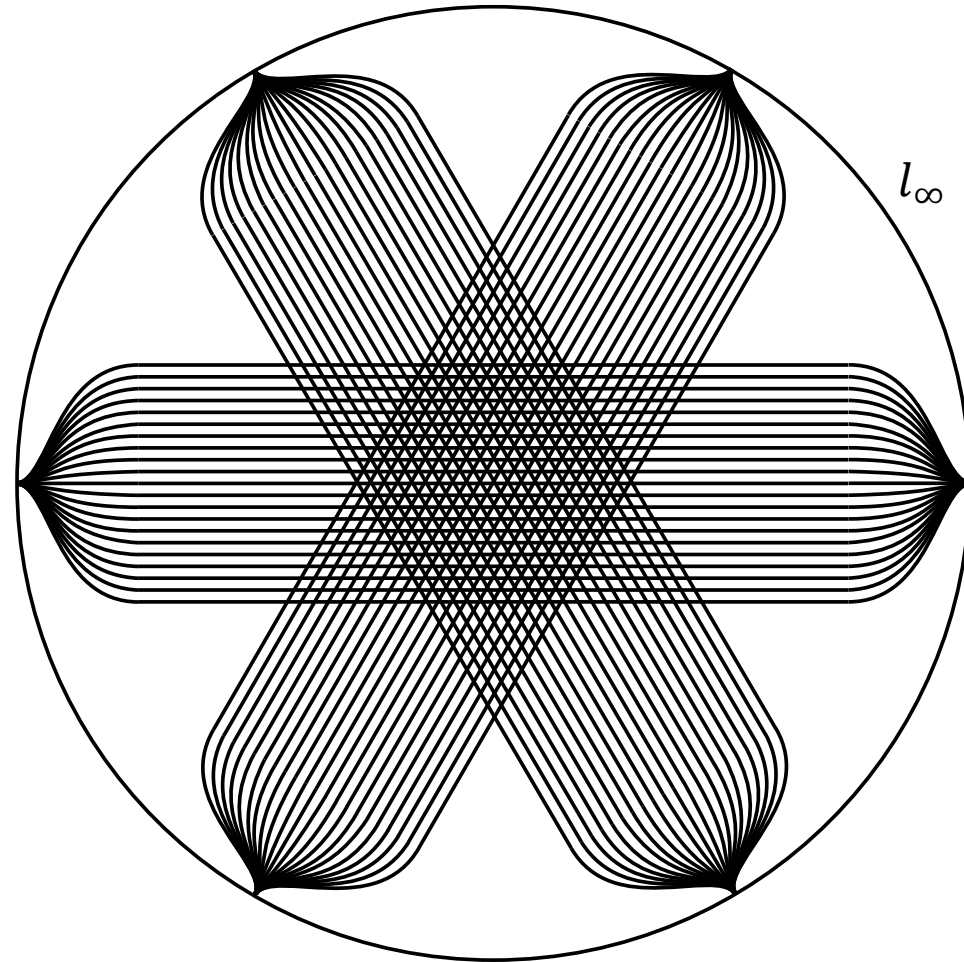
Previously

- Projective geometry !



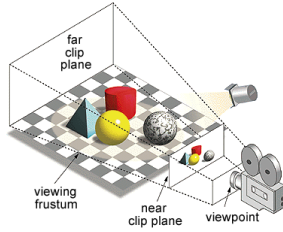
- And rotations (Rodrigues)

Euclidean Geometry

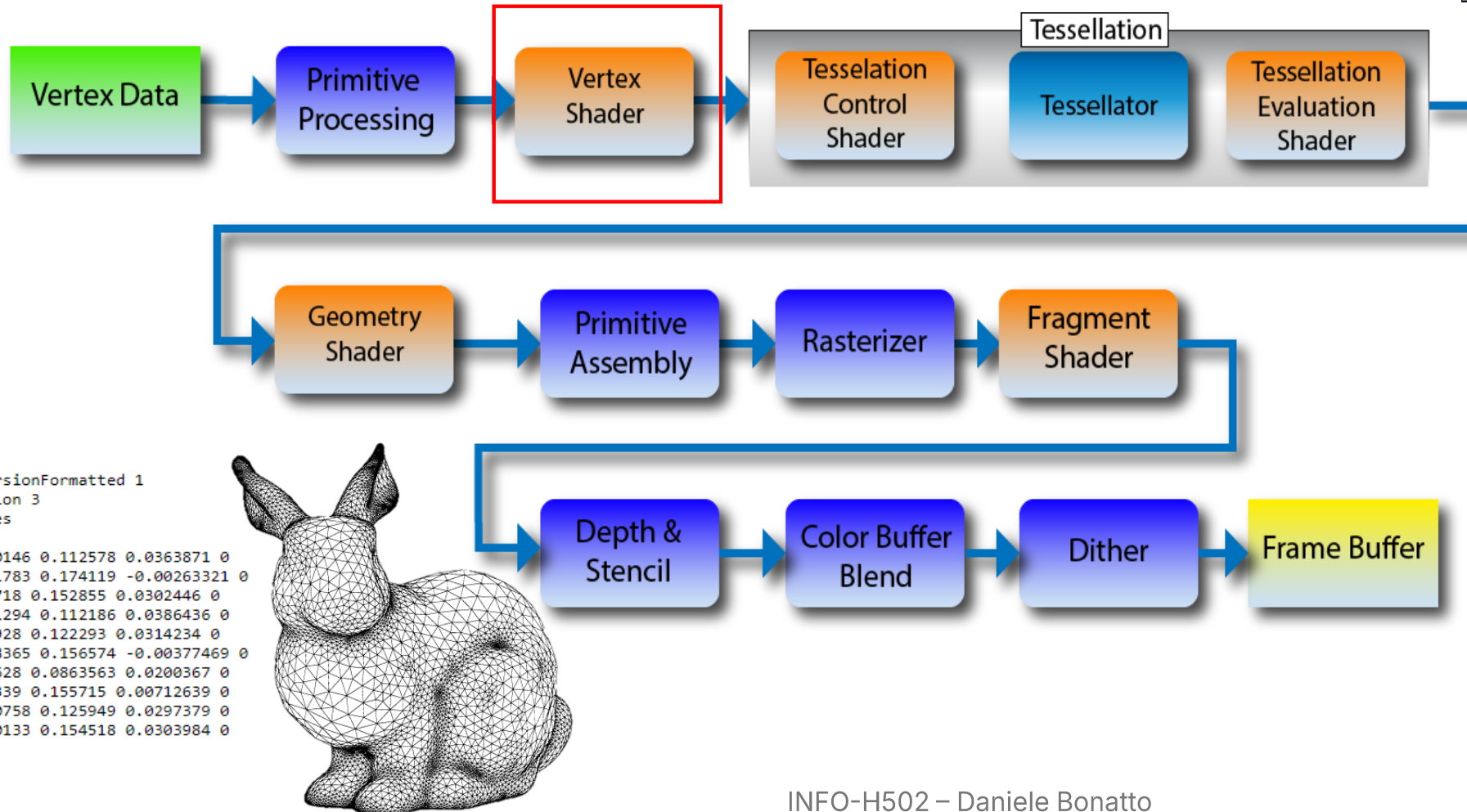
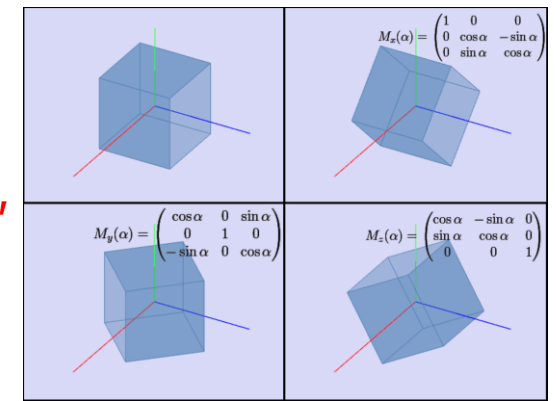


Projective Geometry

OpenGL Pipeline



Linear Algebra: Matrix transformations – Lecture 2,
Model, Camera, Projection spaces – This lecture !

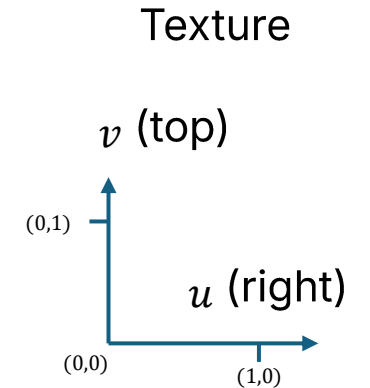
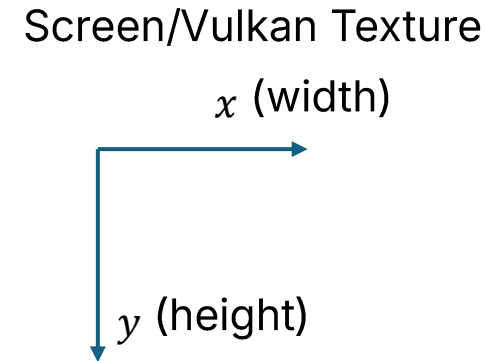
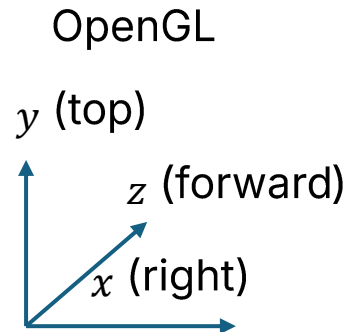
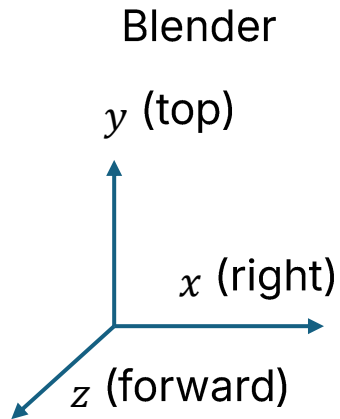


```

MeshVersionFormatted 1
Dimension 3
Vertices
5433
-0.0260146 0.112578 0.0363871 0
-0.0321783 0.174119 -0.00263321 0
-0.080718 0.152855 0.0302446 0
-0.0231294 0.112186 0.0386436 0
0.0164928 0.122293 0.0314234 0
-0.0248365 0.156574 -0.00377469 0
0.0452628 0.0863563 0.0200367 0
-0.071339 0.155715 0.00712639 0
-0.0100758 0.125949 0.0297379 0
-0.0720133 0.154518 0.0303984 0
    
```

Common Axis Systems

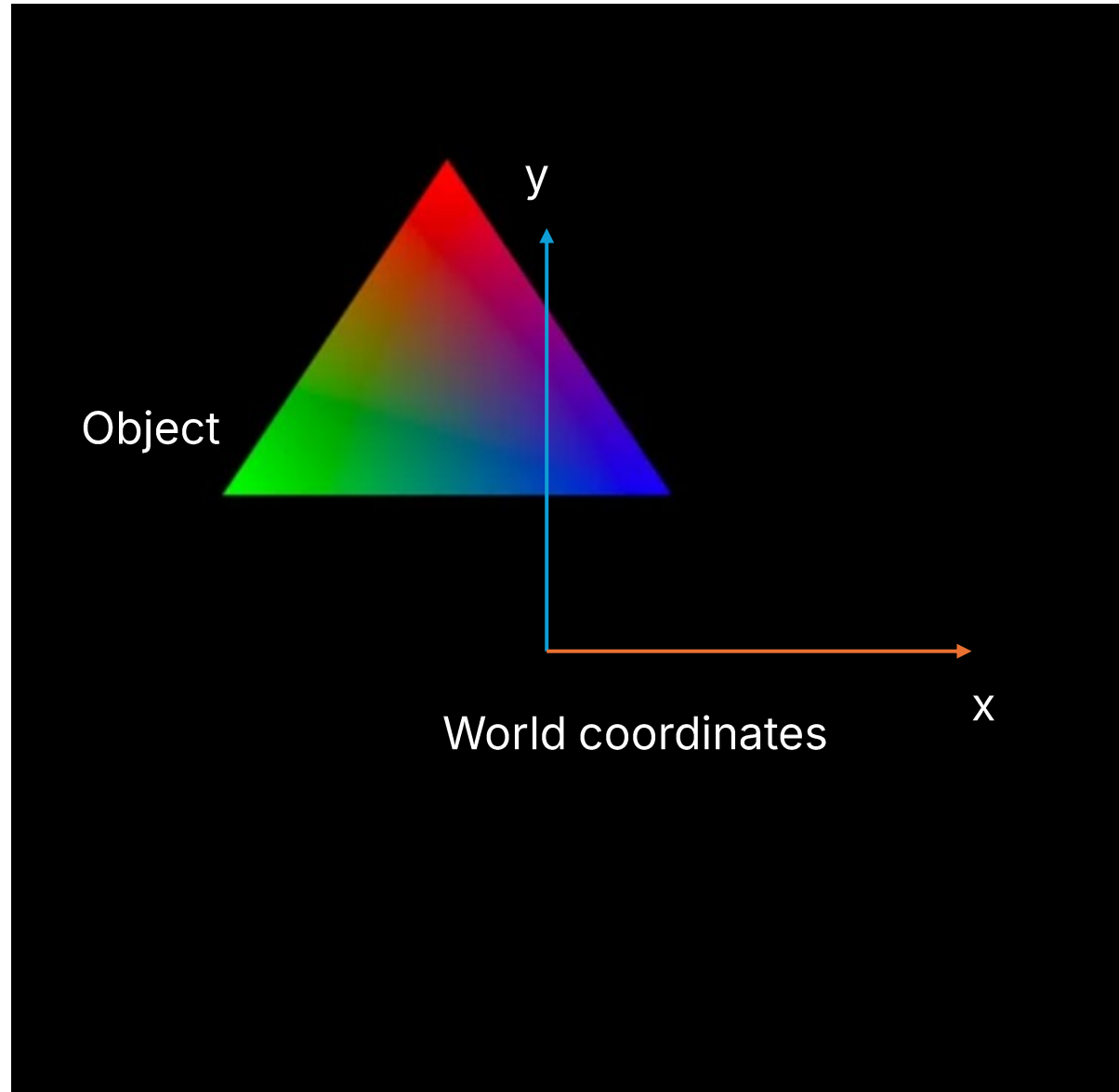
- First of all, a note on axis systems.
- Everything comes with an axis system..



- You need to convert between them when coding.
- Note that the handedness of OpenGL is not the usual one !
- Every time you see a minus popping in an equation, think of the coordinates!

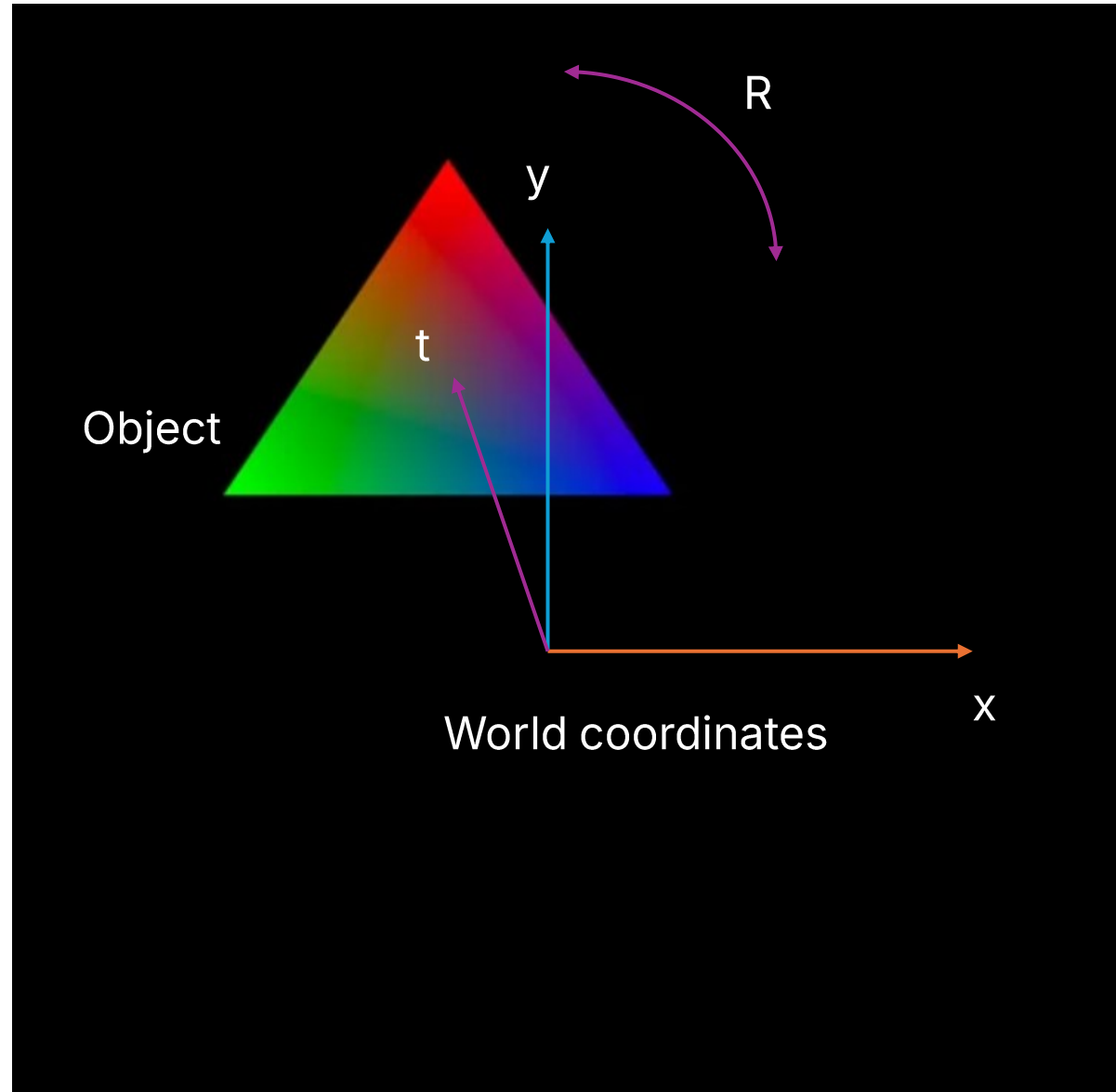
What is a 2D Scene

- Object(s)
- Light(s)
- World coordinates
- Both in **2D** coordinates
 - (x, y)



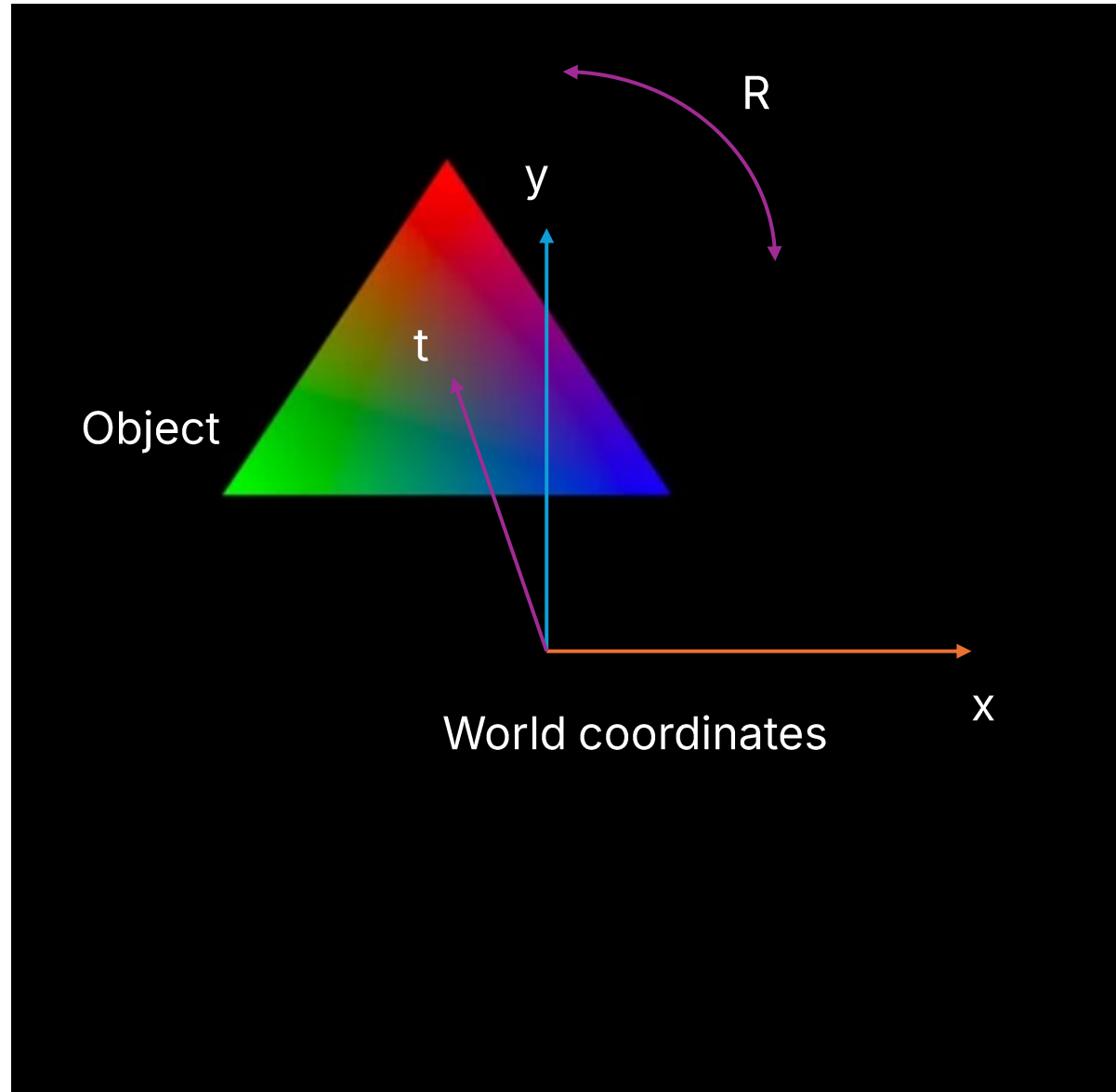
What is a 2D Scene

- Object(s)
- Light(s)
- World coordinates
- Both in 2D coordinates
 - (x, y)
- To move an object w.r.t the world coordinates system, we use a **2D** translation and **2D** rotation.
- $g = (R, t), R \in SO(2), t \in \mathbb{R}^2$



What is a 2D Scene

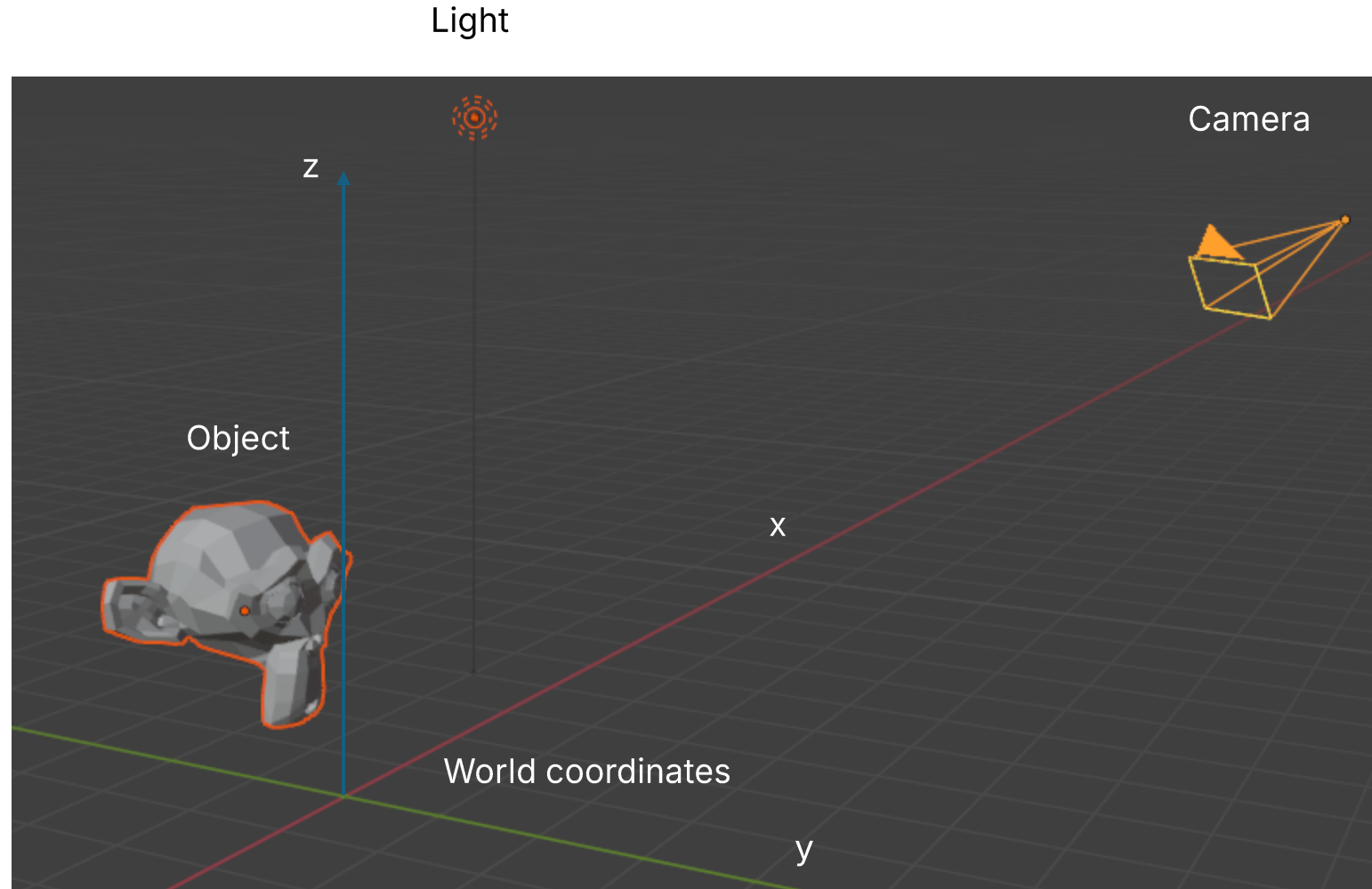
- Object(s)
- Light(s)
- World coordinates
- Both in 2D coordinates
 - (x, y)
- In homogeneous coordinates
This is the **Model** matrix
- $M = \begin{pmatrix} r_{11} & r_{12} & t_1 \\ r_{21} & r_{22} & t_2 \\ 0 & 0 & 1 \end{pmatrix}$



What is a 3D Scene

- Object(s)
- World coordinates
- Light(s)
- Camera (new)
- All in **3D** coordinates
 - (x, y, z)
- The **model** matrix is similar

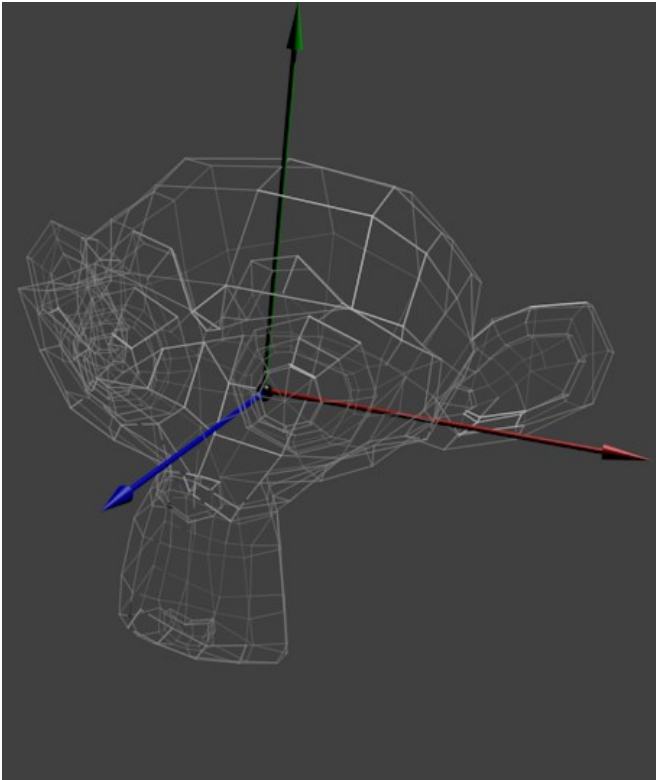
$$M = \begin{pmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



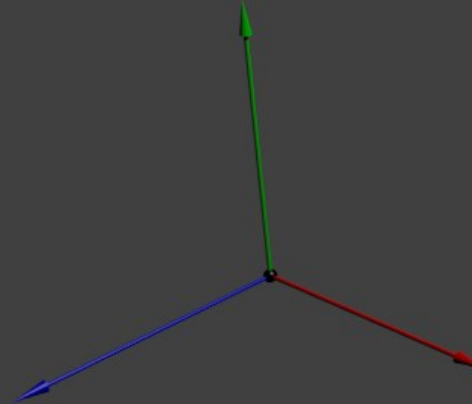
What is a 3D Scene

- $M = \begin{pmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

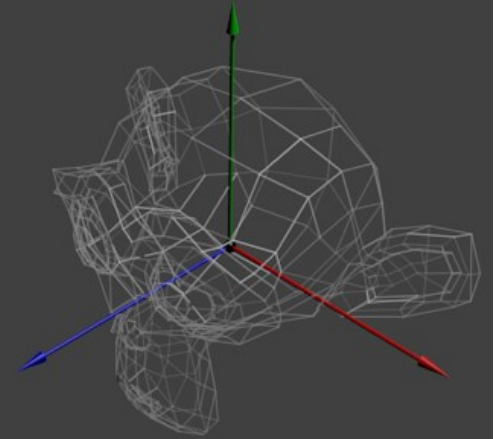
Object
coordinates system



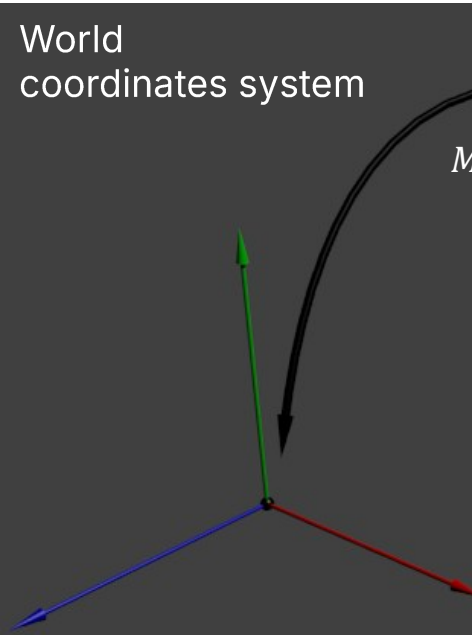
World
coordinates system



Object
coordinates system

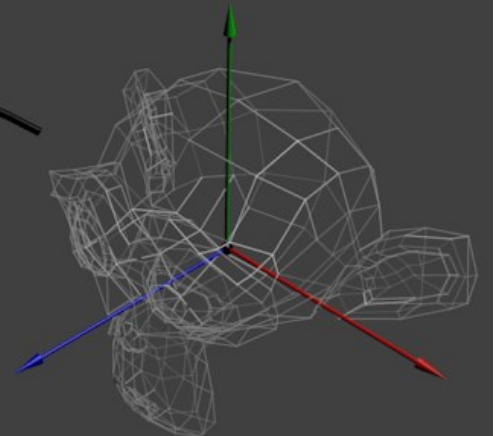


World
coordinates system



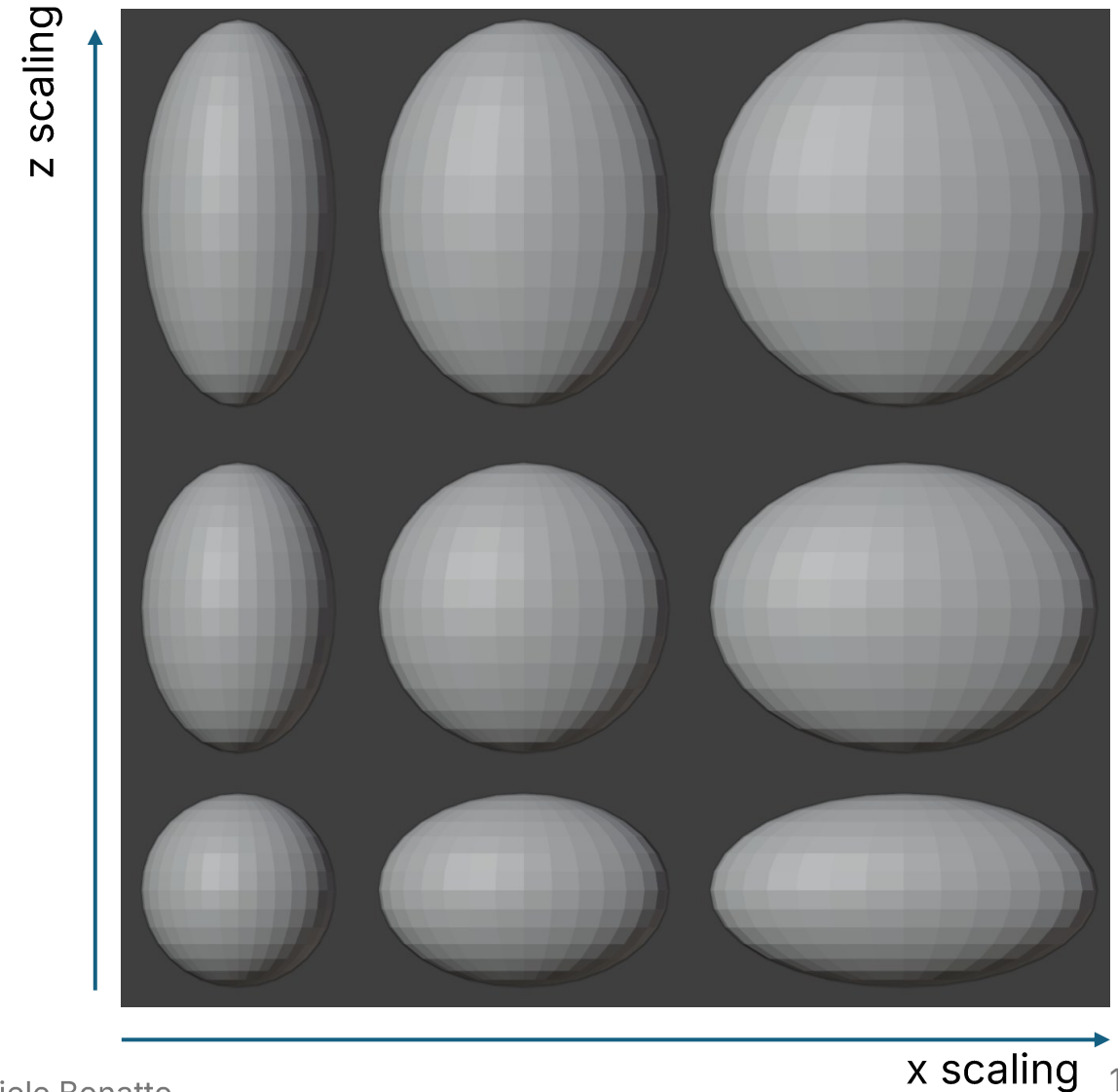
M

Object
coordinates system



Model matrix and scaling

- Often we do not want to only move an object in the scene.
- We want to scale it!
- We can scale all directions independently
- How does this change our model matrix?



Model matrix and scaling

- Mathematically, a scaling is easy:

$$S = \begin{pmatrix} s_1 & 0 & 0 & 0 \\ 0 & s_2 & 0 & 0 \\ 0 & 0 & s_3 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} S & 0 \\ 0 & 1 \end{pmatrix}$$

- For example:

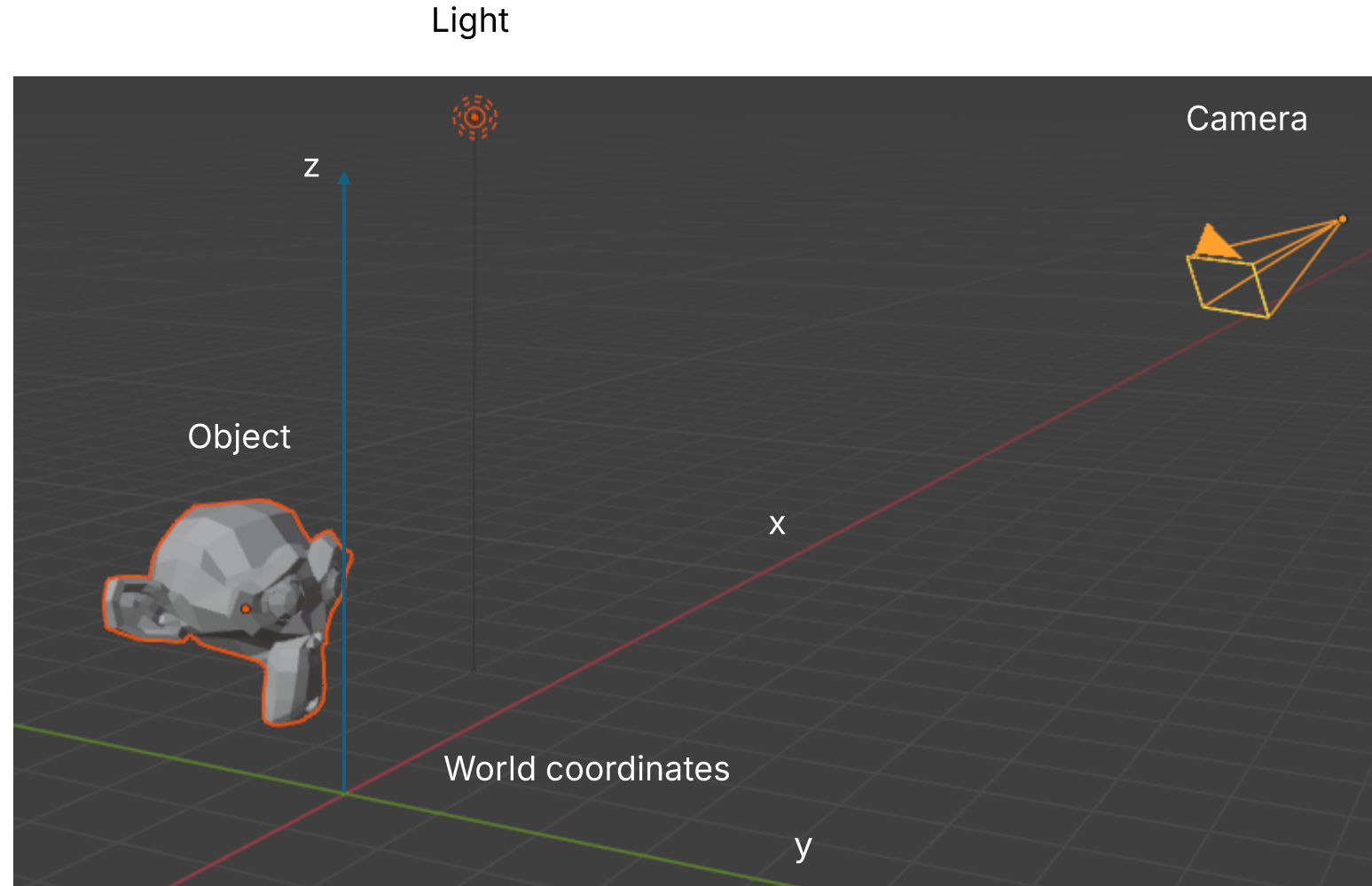
$$\begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix} = \begin{pmatrix} 2x \\ 3y \\ 4z \\ w \end{pmatrix}$$

- We can combine it directly with our model matrix !

$$M' = MS = \begin{pmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \textcolor{red}{s}_1 & 0 & 0 & 0 \\ 0 & \textcolor{red}{s}_2 & 0 & 0 \\ 0 & 0 & \textcolor{red}{s}_3 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} \textcolor{red}{s}_1 r_{11} & \textcolor{red}{s}_2 r_{12} & \textcolor{red}{s}_3 r_{13} & t_1 \\ \textcolor{red}{s}_1 r_{21} & \textcolor{red}{s}_2 r_{22} & \textcolor{red}{s}_3 r_{23} & t_2 \\ \textcolor{red}{s}_1 r_{31} & \textcolor{red}{s}_2 r_{32} & \textcolor{red}{s}_3 r_{33} & t_3 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

What is a 3D Scene

- Object(s)
 - World coordinates
 - Light(s)
 - Camera
- We need a way to move the **object(s)** w.r.t the **world coordinates system** AND w.r.t the **camera**.

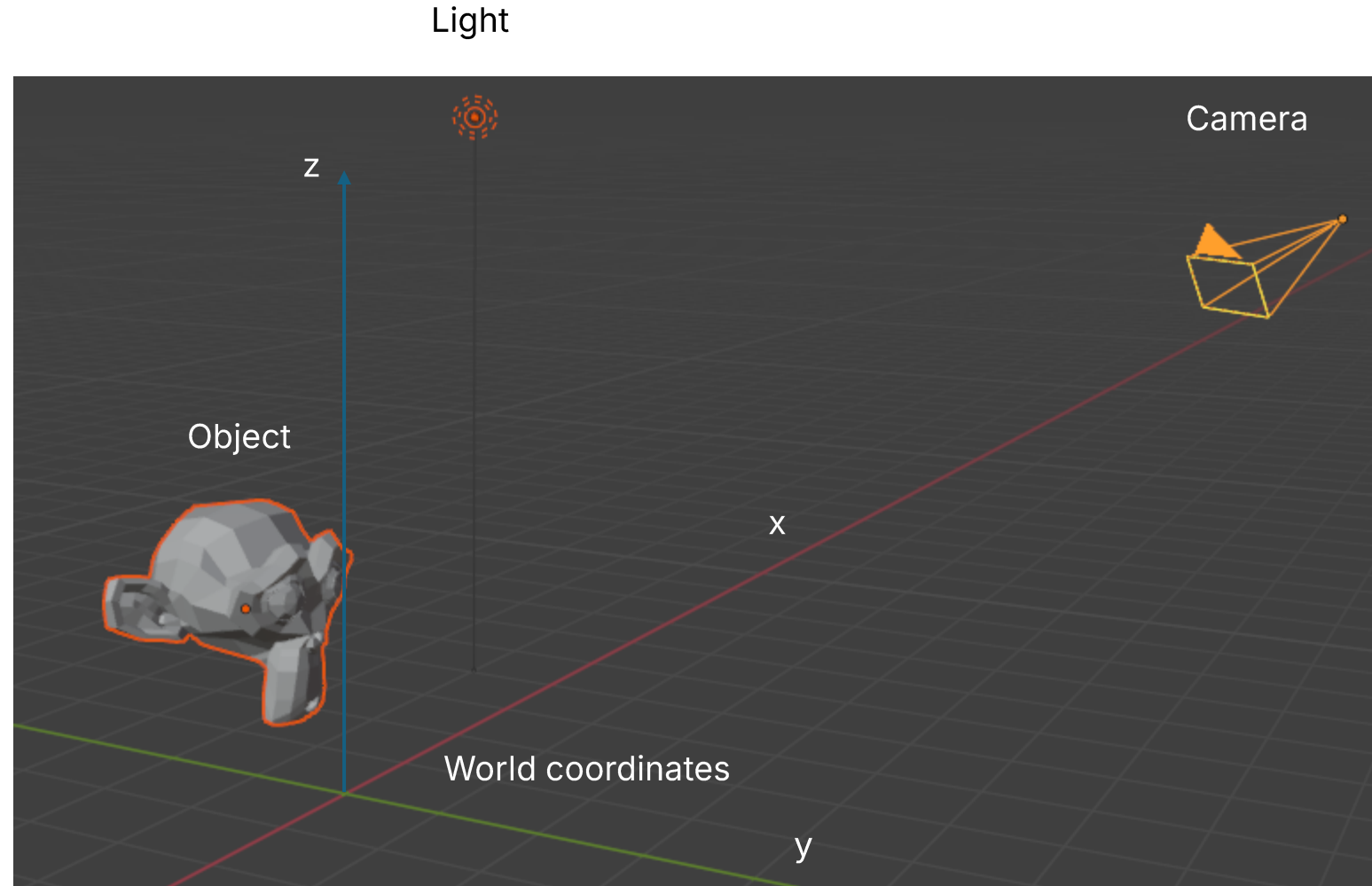


What is a 3D Scene

- Object(s)
- World coordinates
- Light(s)
- Camera

- This new matrix is the **View** matrix

- $$V = \begin{pmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

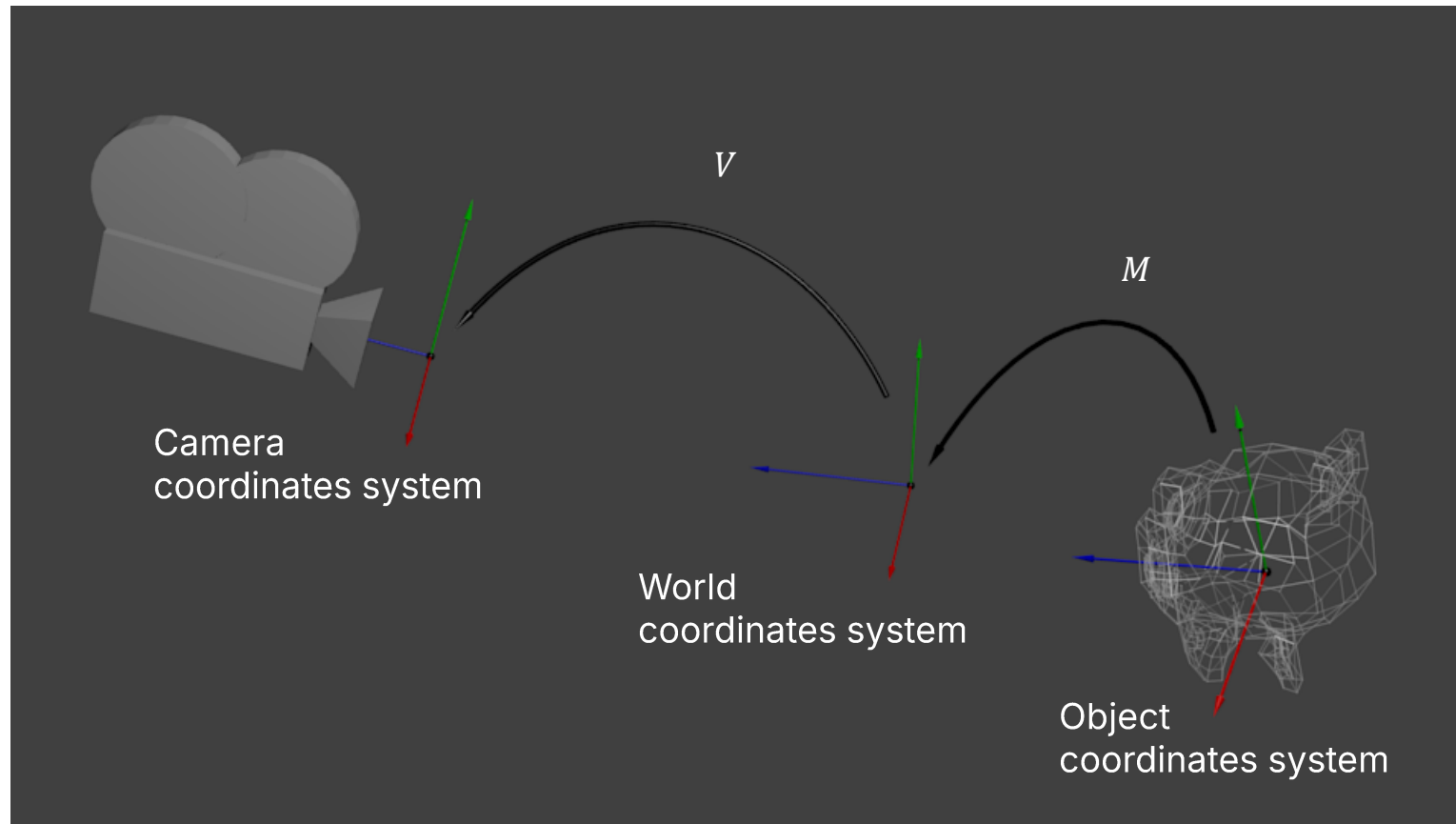


- Another rigid body motion in homogeneous coordinates.

What is a 3D Scene

$$M = \begin{pmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$V = \begin{pmatrix} r'_{11} & r'_{12} & r'_{13} & t'_1 \\ r'_{21} & r'_{22} & r'_{23} & t'_2 \\ r'_{31} & r'_{32} & r'_{33} & t'_3 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



ModelView matrix

- As both the **model** and the **view** matrices are rigid body motions

$$M = \begin{pmatrix} R_M & T_M \\ 0 & 1 \end{pmatrix}, \quad V = \begin{pmatrix} R_V & T_V \\ 0 & 1 \end{pmatrix}$$

- And rigid body motions are a group $SE(3)$
- We can combine them together as the **ModelView** matrix.

$$M_V = \begin{pmatrix} R_M R_V & R_M T_V + T_M \\ 0 & 1 \end{pmatrix}$$

- This matrix directly send the vertices of our mesh from the object coordinates system to the camera coordinates system.

ModelView matrix

- Do all properties in a mesh scale with this M_V matrix?
- We know it does for a point $p' = M_V p$
- What other informations do we associate to our mesh ?

ModelView matrix

- Do all properties in a mesh scale with this M_V matrix?
- We know it does for a point $p' = M_V p$
- What other informations do we associate to our mesh ?
 - Vertices (the points)
 - Colors
 - Texture coordinates
 - Normals
- Do they all transform or need to be transformed with M_V ?

ModelView matrix

- Do all properties in a mesh scale with this M_V matrix?
- We know it does for a point $p' = M_V p$
- What other informations do we associate to our mesh ?
 - Vertices (the points)
 - Yes
 - Colors
 - Not needed, a color stay the same after the movement
 - Texture coordinates
 - Same as with colors
 - Normals
 - Maybe ?
- Do they all transform or need to be transformed with M_V ?

Normal vectors and the ModelView matrix

- The vertices of our mesh are moved in space using the M_V matrix

$$x' = M_V x$$

- But, each vertex has a normal vector with properties:

- $\|n\| = 1$
- The should stay normal to the associated vertex

- We also need to transform them

$$n' = M_V n$$

- Is this the correct transformation?

Normal vectors and the ModelView matrix

$$n' = M_V n$$

- Is this the correct transformation?
- What happens if M_V is a pure rotation ?

$$M_V = \begin{pmatrix} r_{11} & r_{12} & r_{13} & 0 \\ r_{21} & r_{22} & r_{23} & 0 \\ r_{31} & r_{32} & r_{33} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- For example for some rotation R and $n = (0, 0, 1)$
- $n' = M_V n = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.866 & -0.5 & 0 \\ 0 & 0.5 & 0.866 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ -0.5 \\ 0.866 \\ 1 \end{pmatrix}$

Normal vectors and the ModelView matrix

$$n' = M_V n$$

- Is this the correct transformation?
- What happens if M_V is a pure rotation ?

$$M_V = \begin{pmatrix} r_{11} & r_{12} & r_{13} & 0 \\ r_{21} & r_{22} & r_{23} & 0 \\ r_{31} & r_{32} & r_{33} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- For example for some rotation R and $n = (0, 0, 1)$
- $n' = M_V n = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.866 & -0.5 & 0 \\ 0 & 0.5 & 0.866 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ -0.5 \\ 0.866 \\ 1 \end{pmatrix}$
- This works: $\|(0, -0.5, 0.866)^T\| = \sqrt{0.25 + 0.75} = 1$

Normal vectors and the ModelView matrix

$$n' = M_V n$$

- Is this the correct transformation?
- What happens if M_V is a pure translation?

$$M_V = \begin{pmatrix} 1 & 0 & 0 & t_1 \\ 0 & 1 & 0 & t_2 \\ 0 & 0 & 1 & t_3 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- For example $T = (2, 3, 4)^T$ and $n = (0, 0, 1)$

- $n' = M_V n = \begin{pmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 3 \\ 0 & 0 & 1 & 4 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 3 \\ 5 \\ 1 \end{pmatrix}$

Normal vectors and the ModelView matrix

$$n' = M_V n$$

- Is this the correct transformation?
- What happens if M_V is a pure translation?

$$M_V = \begin{pmatrix} 1 & 0 & 0 & t_1 \\ 0 & 1 & 0 & t_2 \\ 0 & 0 & 1 & t_3 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- For example $T = (2, 3, 4)^T$ and $n = (0, 0, 1)$

$$\bullet \quad n' = M_V n = \begin{pmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 3 \\ 0 & 0 & 1 & 4 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 3 \\ 5 \\ 1 \end{pmatrix}$$

- This is not correct $\|(2, 3, 5)^T\| = \sqrt{4 + 9 + 25} = \sqrt{38} \neq 1$

Normal vectors and the ModelView matrix

$$n' = M_V n$$

- Is this the correct transformation?
- What happens if M_V is a pure scaling?

$$M_V = \begin{pmatrix} s_1 & 0 & 0 & 0 \\ 0 & s_2 & 0 & 0 \\ 0 & 0 & s_3 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- For example $s = I(2, 3, 4)^T$ and $n = (0, 0, 1)$

- $n' = M_V n = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 4 \\ 1 \end{pmatrix}$

Normal vectors and the ModelView matrix

$$n' = M_V n$$

- Is this the correct transformation?
- What happens if M_V is a pure scaling?

$$M_V = \begin{pmatrix} s_1 & 0 & 0 & 0 \\ 0 & s_2 & 0 & 0 \\ 0 & 0 & s_3 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

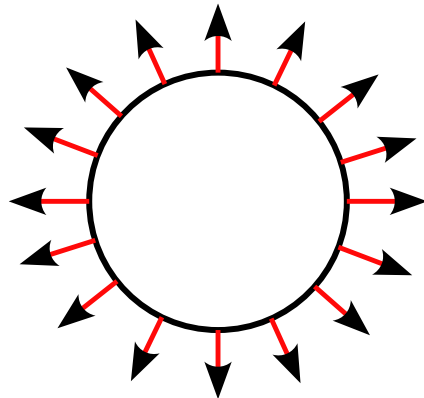
- For example $s = I(2, 3, 4)^T$ and $n = (0, 0, 1)$
- $n' = M_V n = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 4 \\ 1 \end{pmatrix}$
- This is not correct $\|(0, 0, 4)^T\| = \sqrt{16} = 4 \neq 1$

Normal vectors and the ModelView matrix

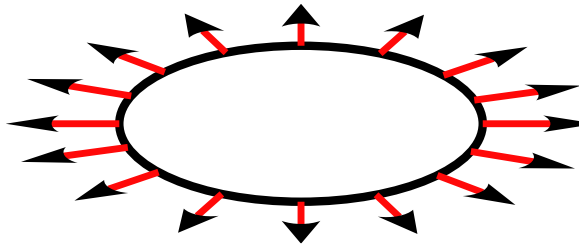
$$n' = M_V n$$

- Is this the correct transformation?
- This transformation works for pure rotations but **not** for pure translations or anisotropic scaling
- So it will not work for a general rigid body motion

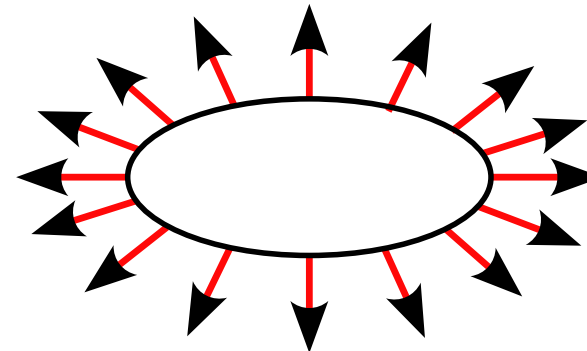
$$M_V = \begin{pmatrix} R_M R_V & R_M T_V + T_M \\ 0 & 1 \end{pmatrix}$$



Original



Bad



Good

- How to solve this ?

Normal vectors and the ModelView matrix

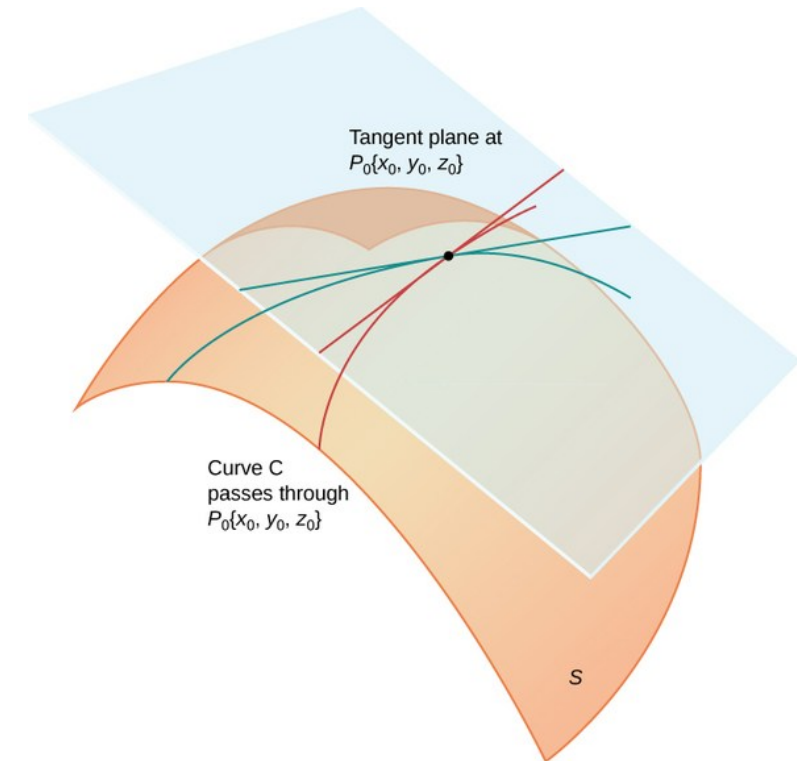
- Let $n \in \mathbb{R}^3$ be a normal at a point $p \in \mathbb{R}^3$ and $M_V \in \mathbb{R}^{4 \times 4}$ a ModelView matrix.
- We have $p' = M_V p$ to move the point.
- We look for $n' = Wn$, i.e., some matrix W that transform correctly n .

- Every point p defines a tangent plane π_t to the object.
- The normal and the a vector t in the plane are perpendicular

$$n^T t = 0$$

- After transformation, perpendicularity between p' and n' must be preserved

$$n'^T t' = 0$$



Normal vectors and the ModelView matrix

- Let $n \in \mathbb{R}^3$ be a normal at a point $p \in \mathbb{R}^3$ and $M_V \in \mathbb{R}^{4 \times 4}$ a ModelView matrix.
- After transformation, perpendicularity between p' and n' must be preserved

$$n'^T t' = 0$$

- We have $t' = M_V t$ and $n' = W n$

$$n'^T t' = 0 \Leftrightarrow (W n)^T (M_V t) = 0 \Leftrightarrow n^T W^T M_V t = 0$$

- This is true if $W^T M_V = I \Leftrightarrow W^T = M_V^{-1}$ (or the pathological case $W = 0$)
- So

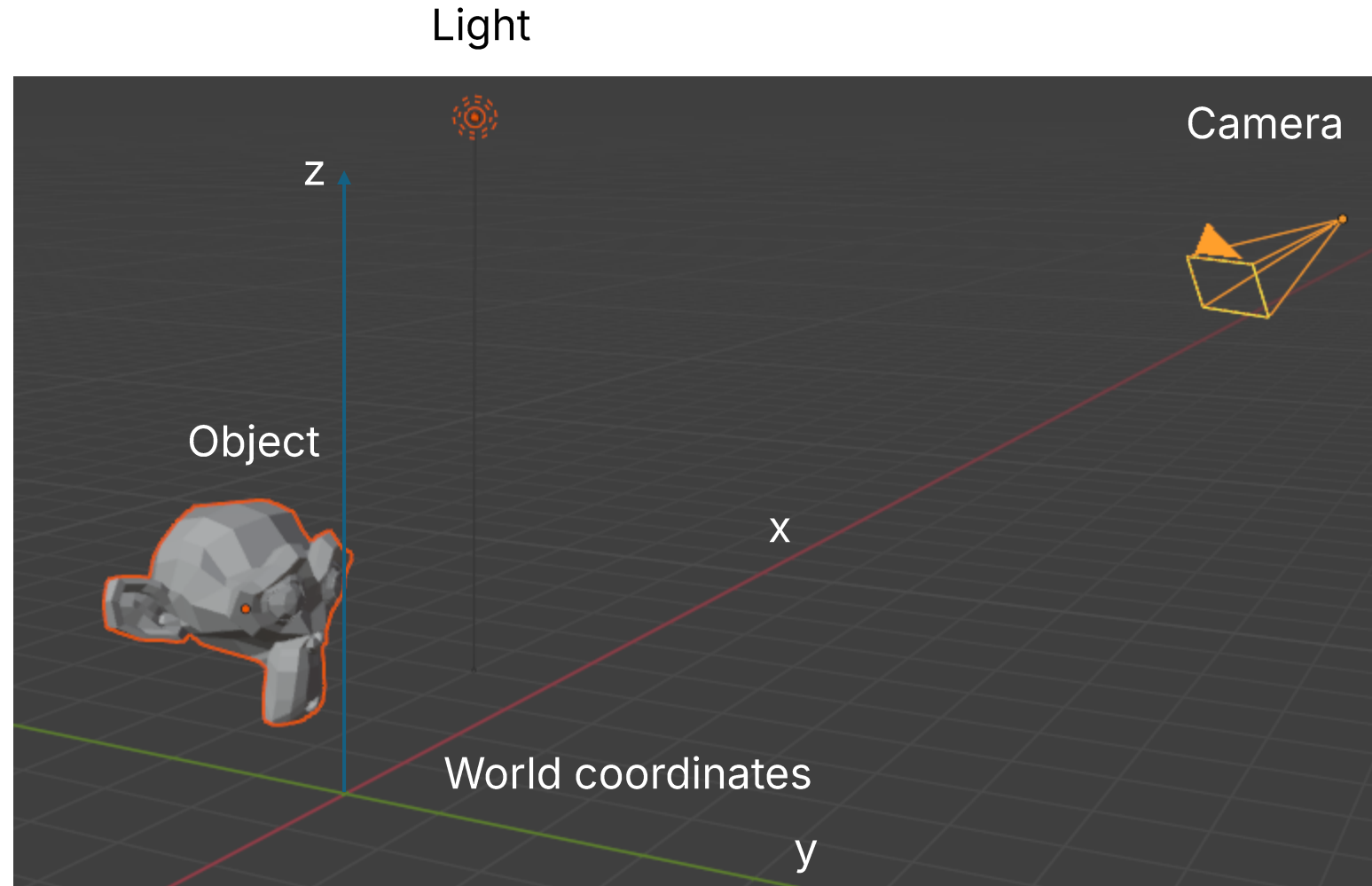
$$W = M_V^{-T}$$

- We found our transformation !

What is a 3D Scene

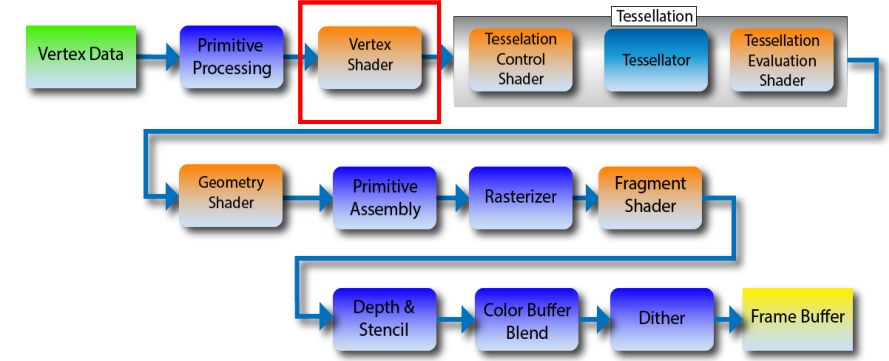
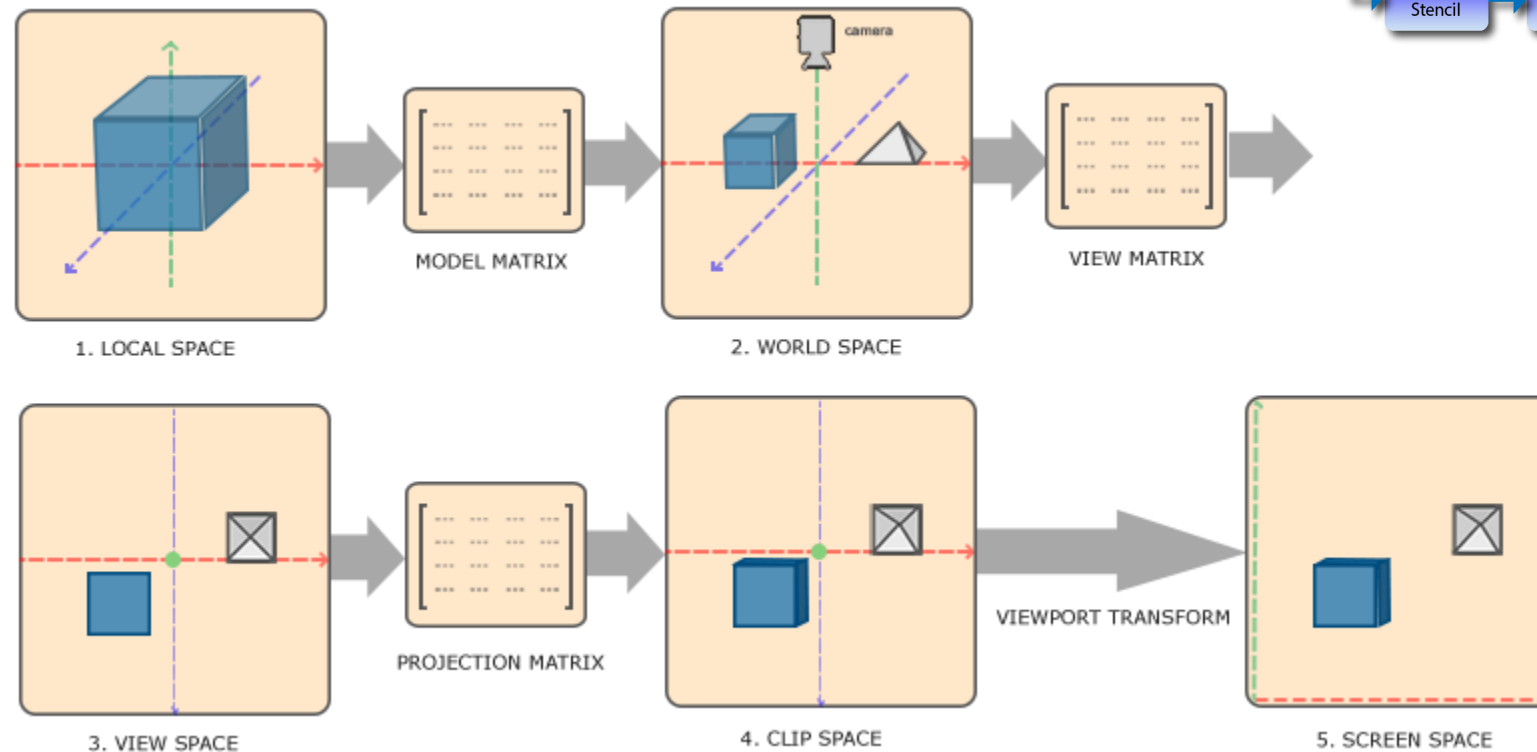
- Object(s)
- World coordinates
- Light(s)
- Camera

- But cameras do not only move points !
- They also warp space in such a way that the perspective is correct
- This is the **projection** matrix, we devote this lesson to find it !



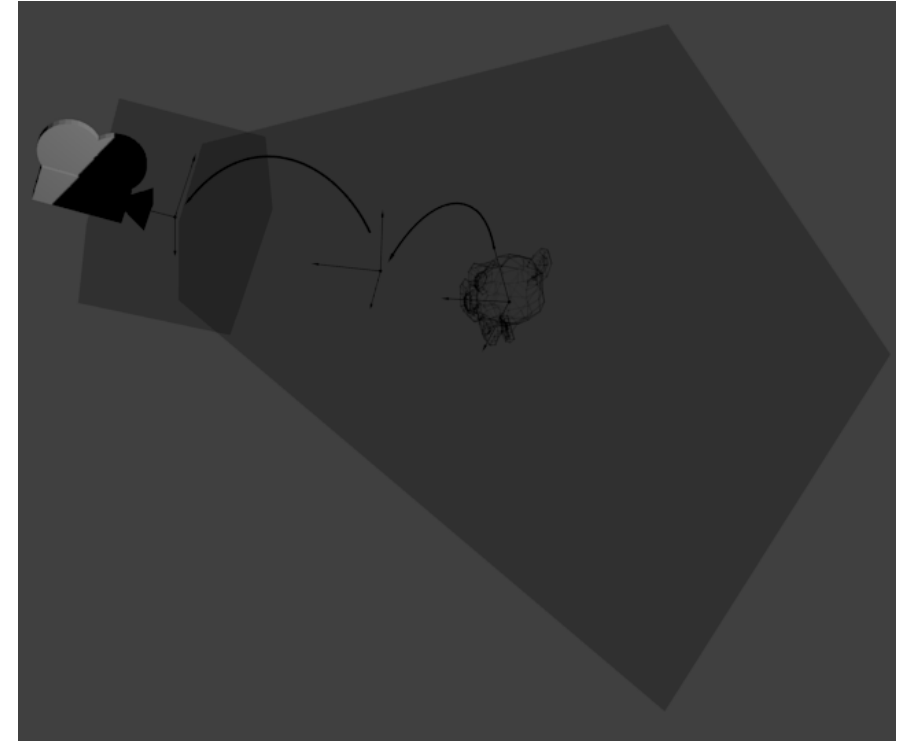
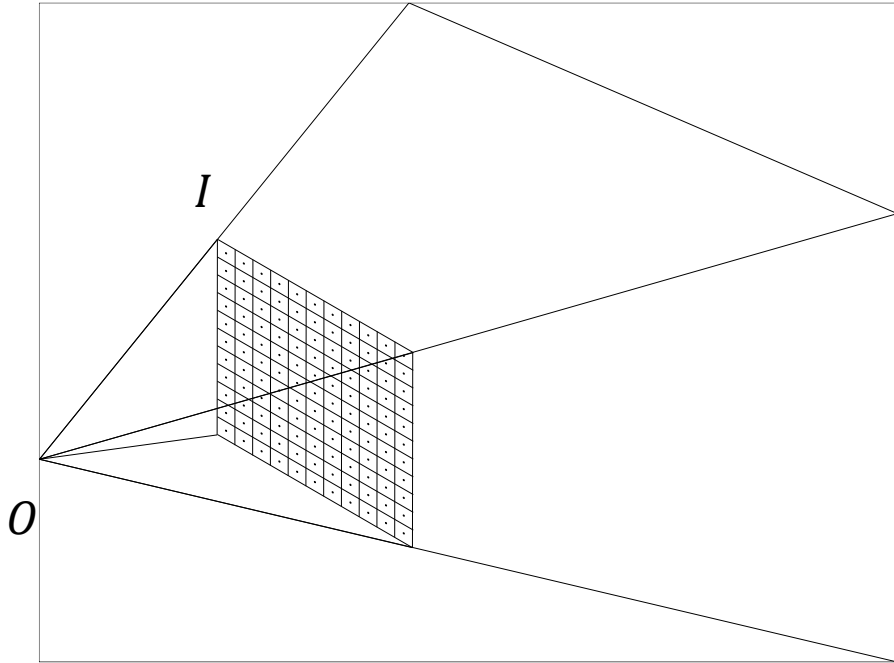
Model View Projection

- We introduced 3 matrices
 - **(Model, View)** to move the object w.r.t the world coordinates and the camera.
 - **Projection** to have perspective correct transformations



What is a projection matrix?

- But what is the **projection** matrix?
- We saw previously that a camera projects the space to the image plane.



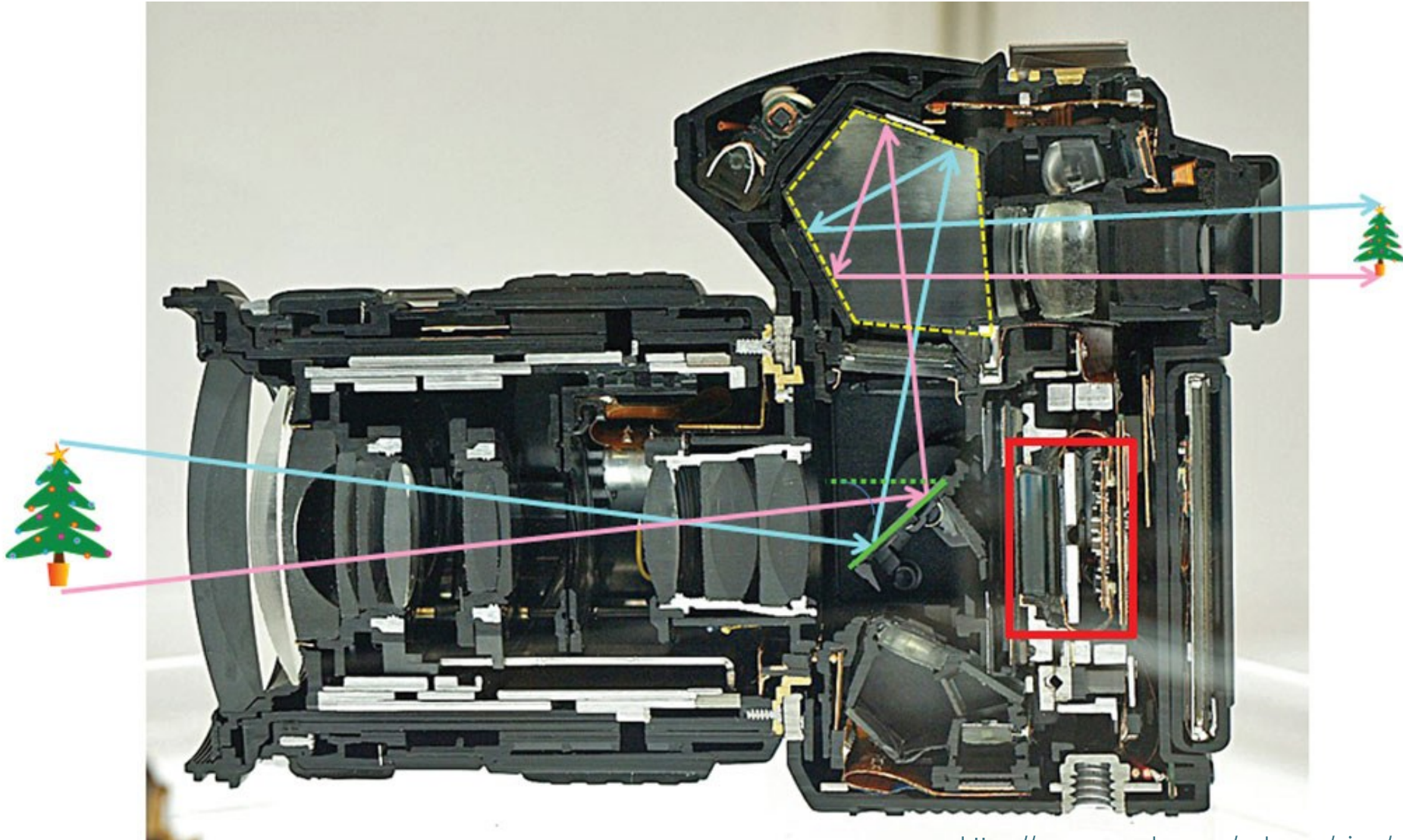
- We need a way to map objects inside the frustum to the image plane

What is a projection matrix?

- But what is the **projection** matrix?
- We saw previously that a camera projects the space to the image plane.
- But everything was using linear equations not in the form of a matrix
- Let's try to understand how a real camera behaves !

A classical camera

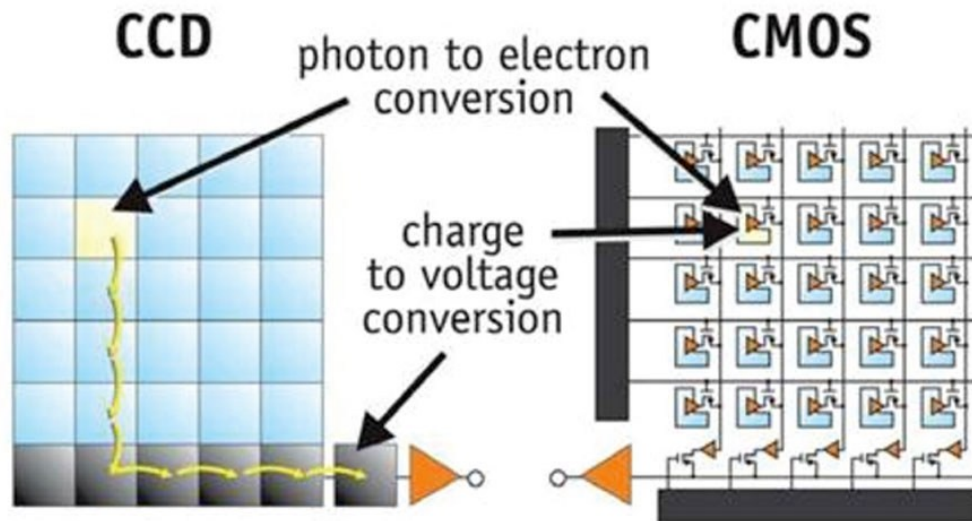
- A camera contains numerous components, its is very complicated



Link with a classic camera

- Let's model it with its two main elements:
- A sensor and a main lens

Sensor (to acquire the image)

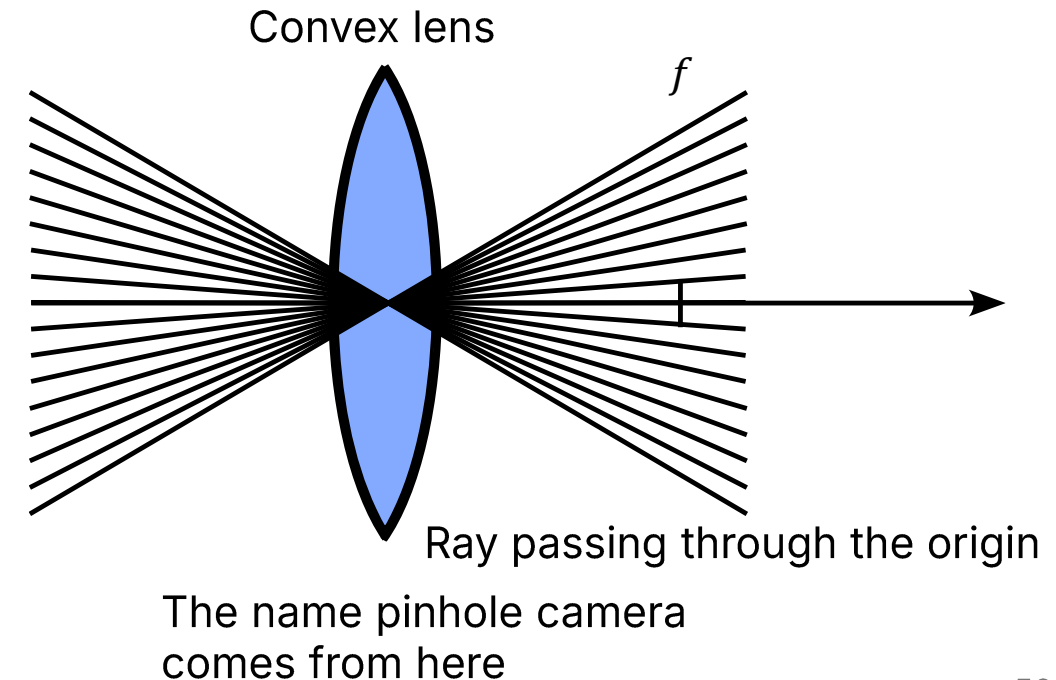
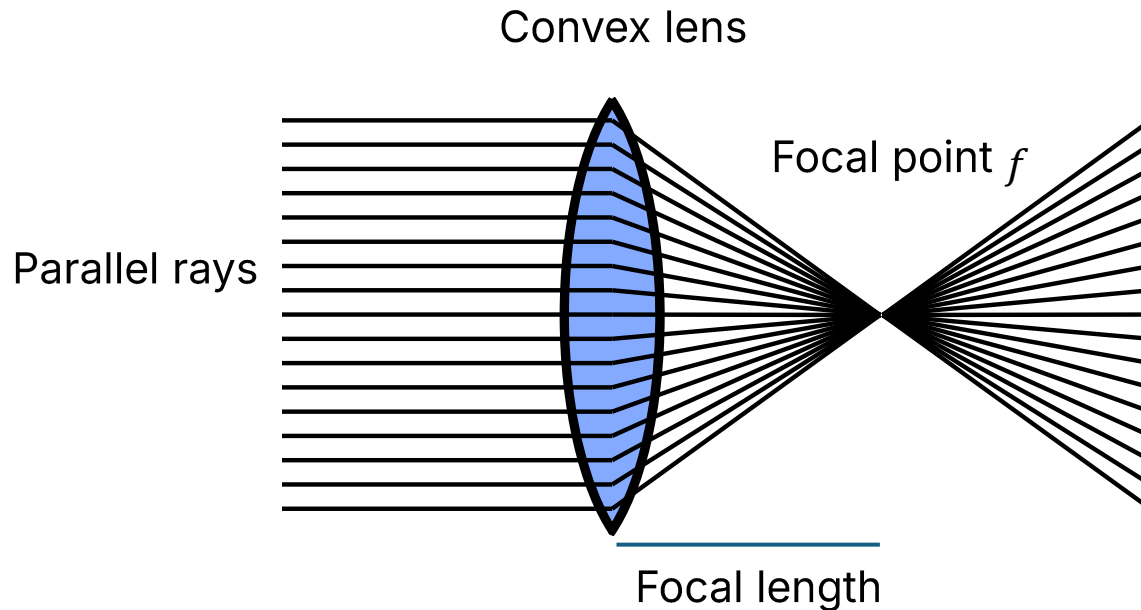


Main lens



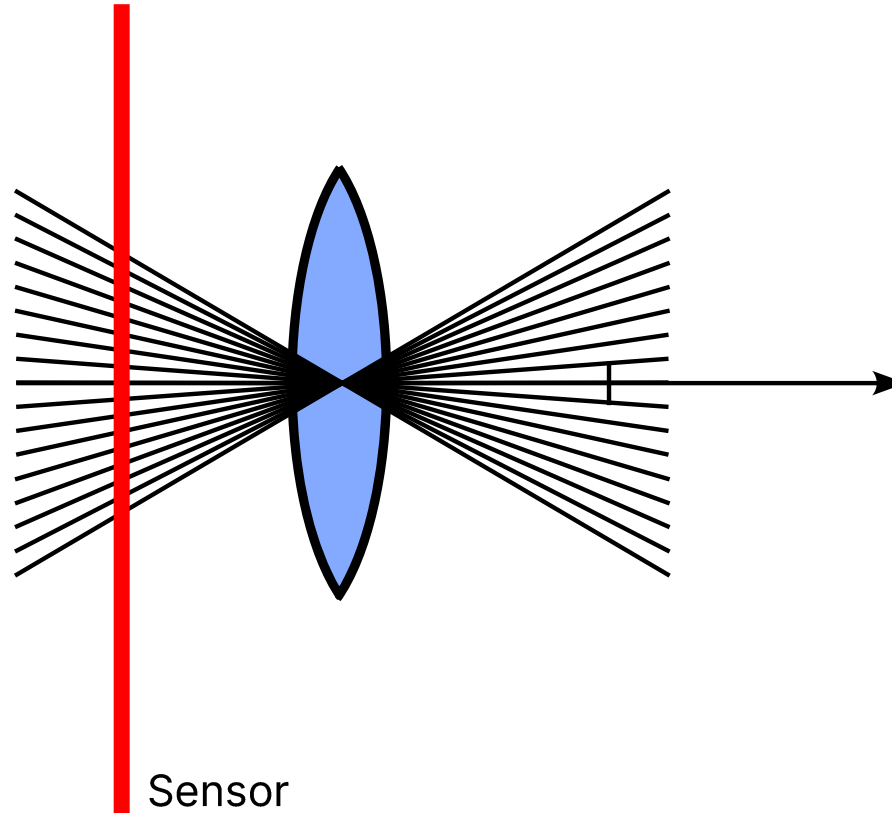
Element 1: Lens

- We need an equation that describe how the lens works
- A simple model is the **paraxial ray approximation** for thin lenses (ideal lens)
- Lens have a focal length f , it is the point at which parallel lines intersect after the lens.
- **Rays that pass through the origin do not bend**



Element 2: Sensor

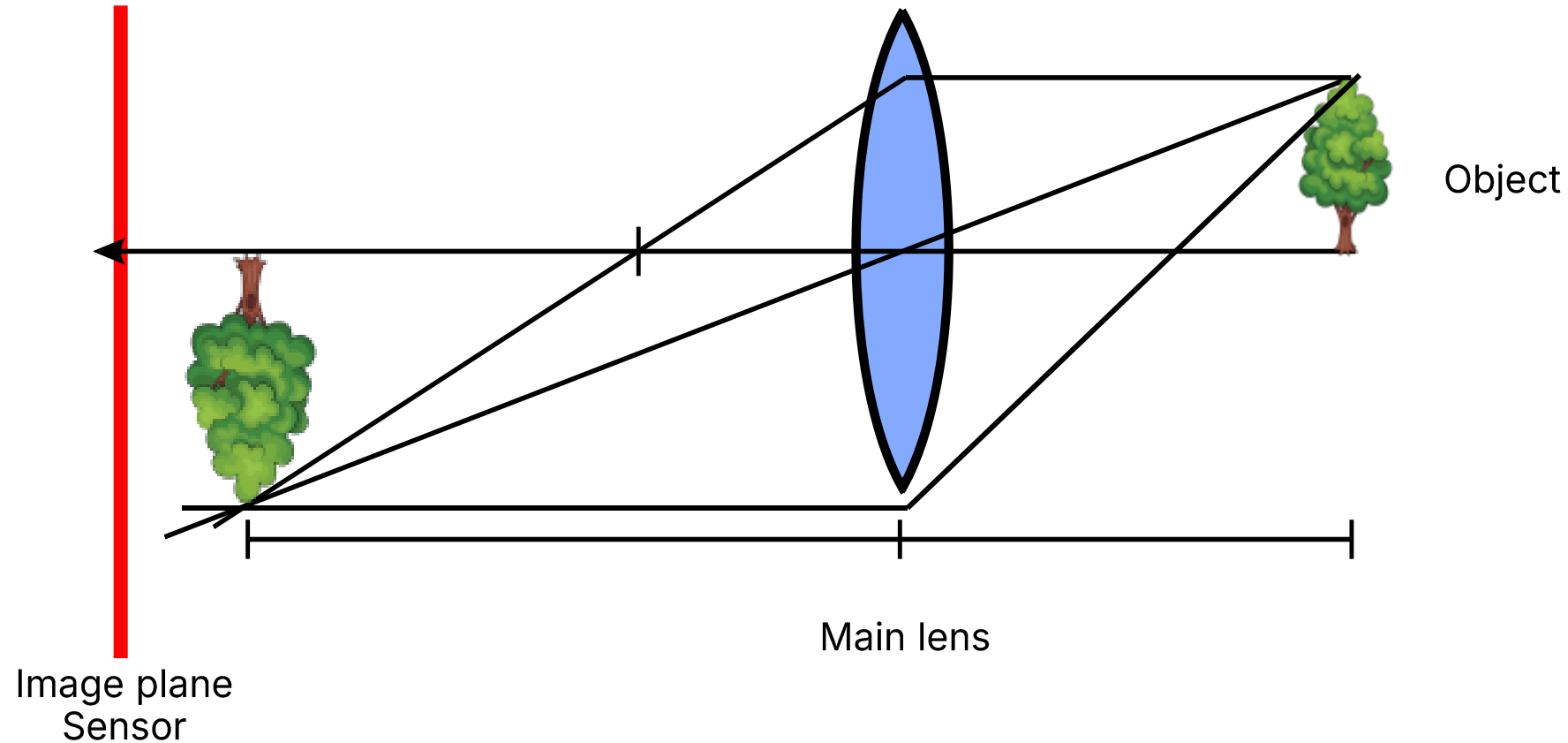
- The sensor is what receives the image



- It receives many rays from everywhere. How does it form an image?

Classical camera model

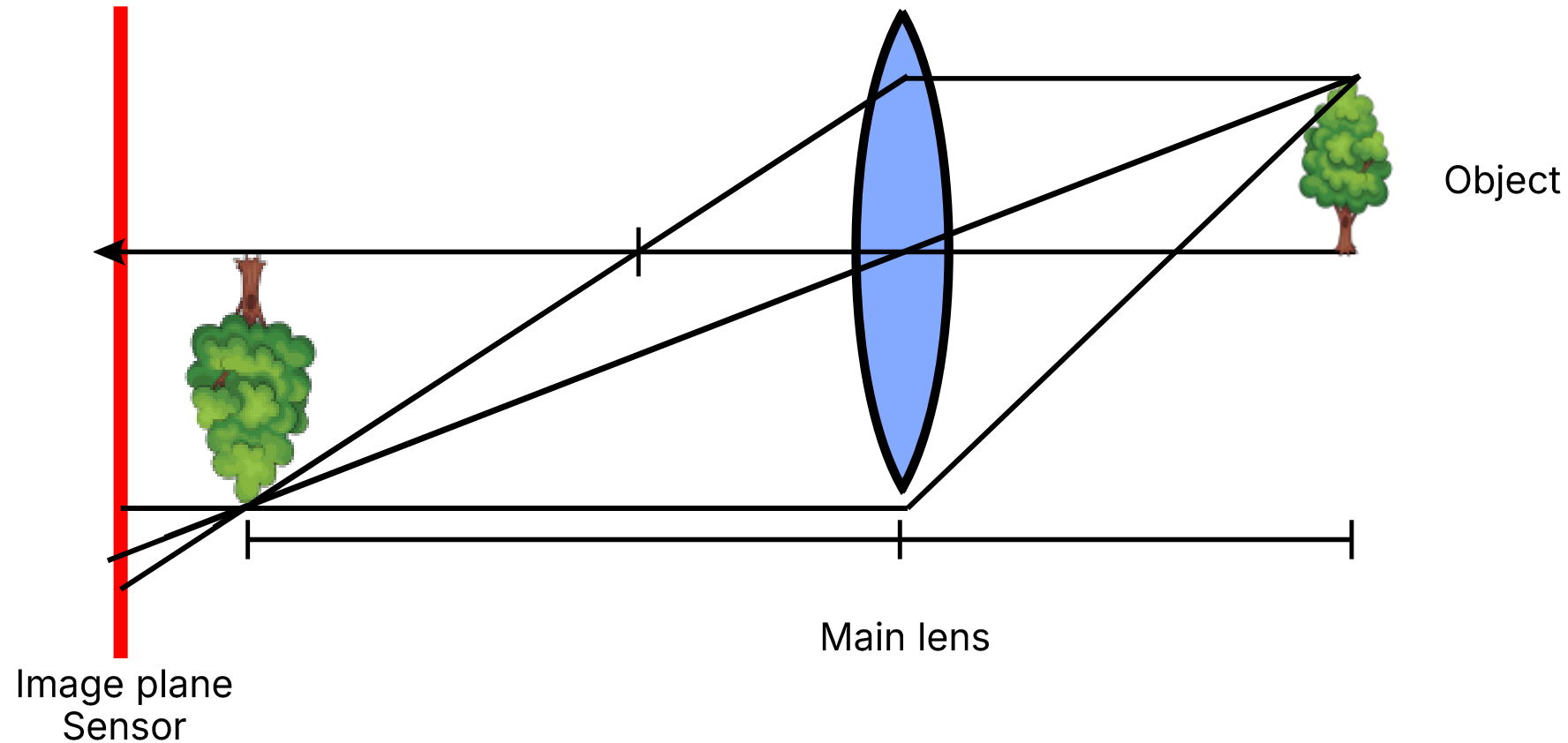
- Let's try to see what happens when we combine our elements (sensor+lens)
- We have a lens, a sensor, and a scene object



- We know what happens in the lens (geometric optics)
- What happens on the sensor?

Classical camera model

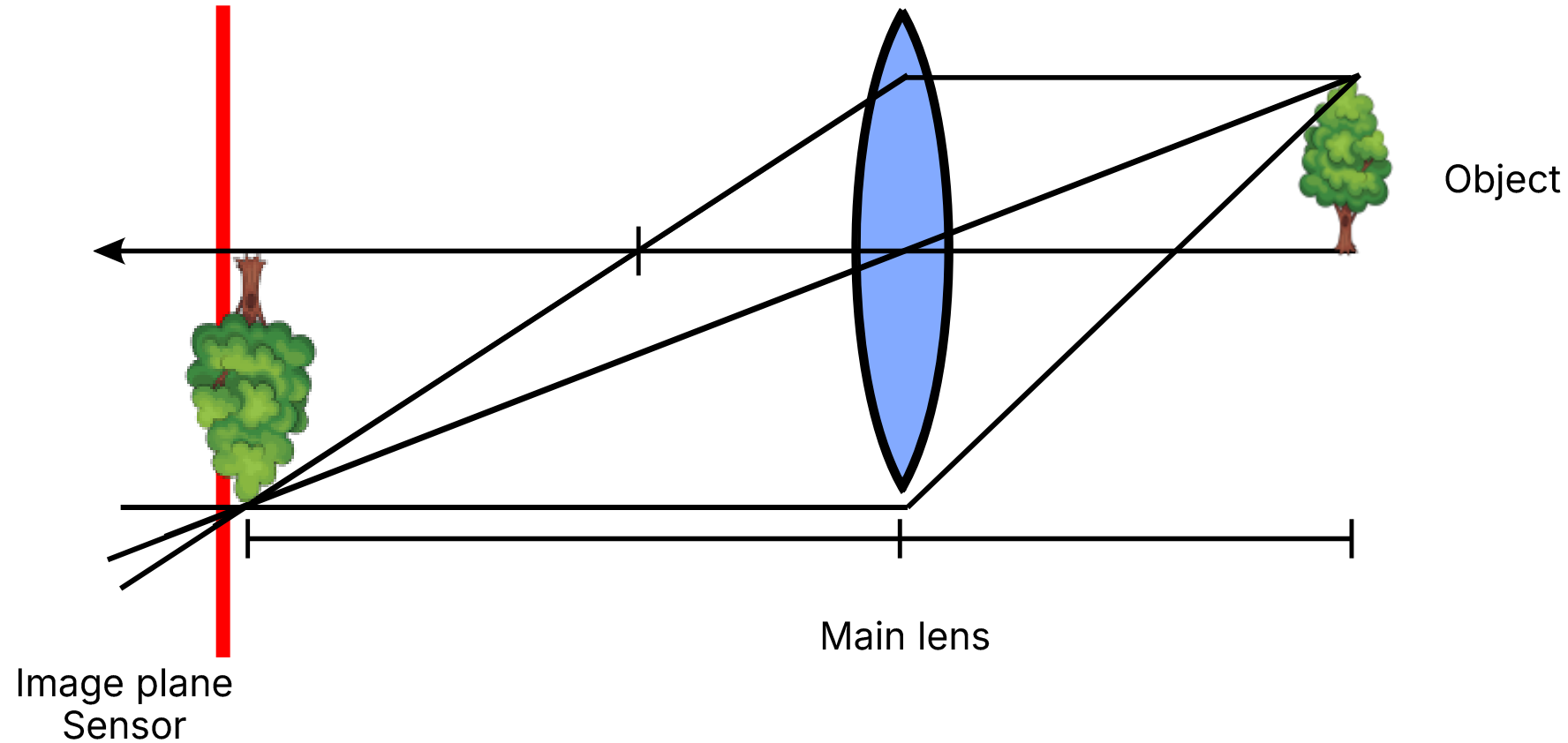
- Let's try to see what happen when we combine our elements
- We have a lens, a sensor, and a scene object



- Let's continue the rays !
- There are several sensor points corresponding to the same 3D point! The image is blurry!

Classical camera model

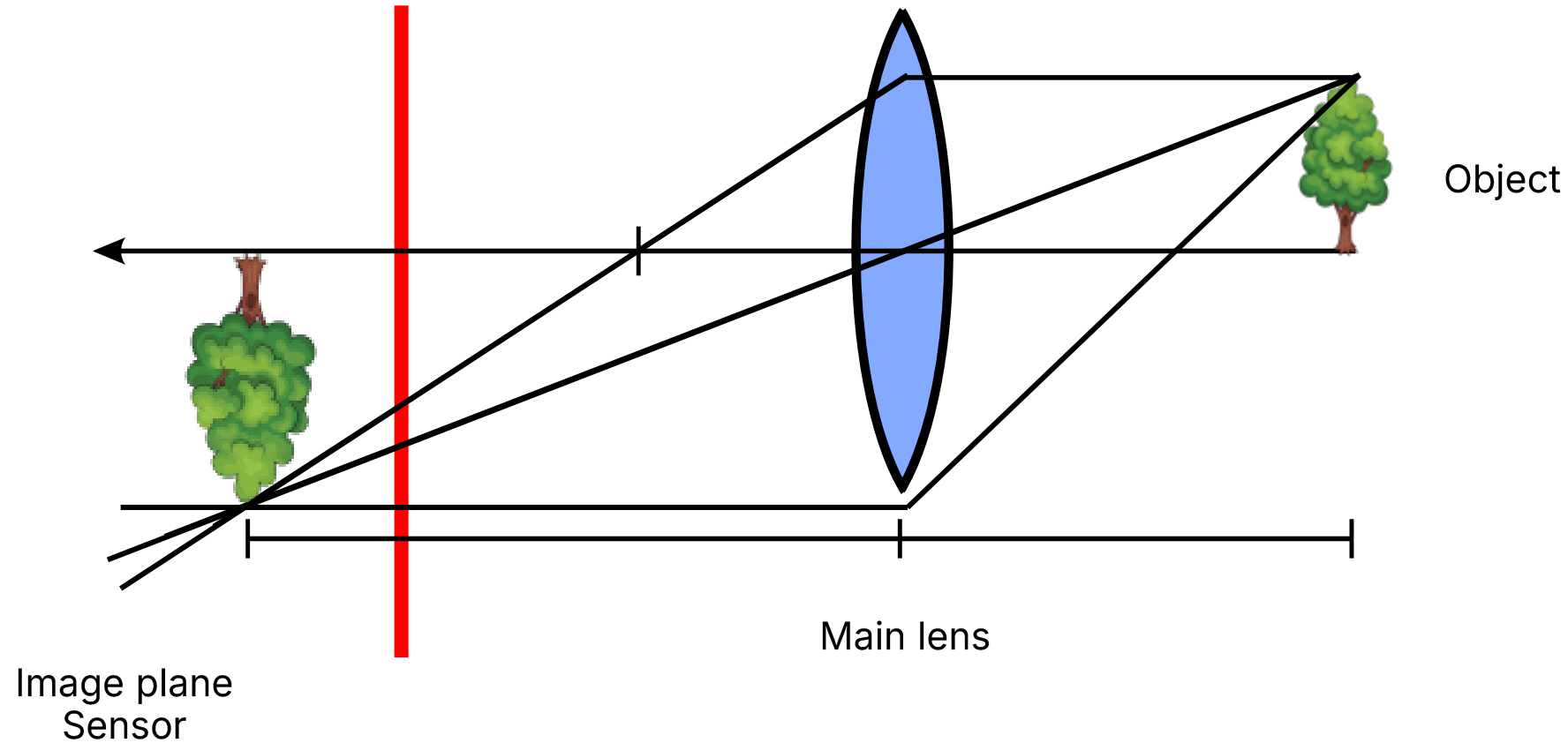
- Let's try to see what happen when we combine our elements
- We have a lens, a sensor, and a scene object



- The blur can be adjusted by changing the distance sensor-lens (focusing)

Classical camera model

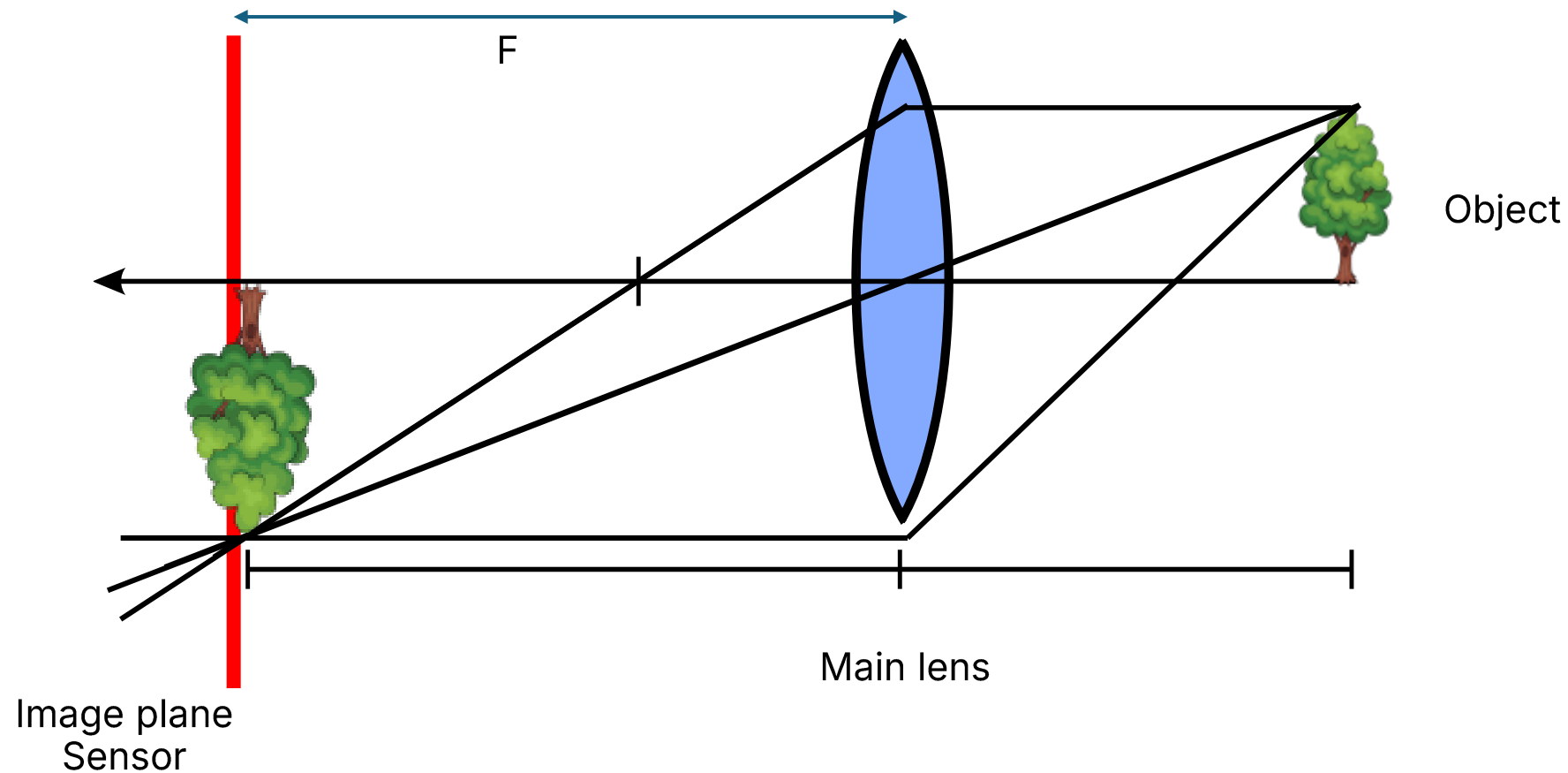
- Let's try to see what happen when we combine our elements
- We have a lens, a sensor, and a scene object



- The blur can be adjusted by changing the distance sensor-lens (focusing)

Classical camera model

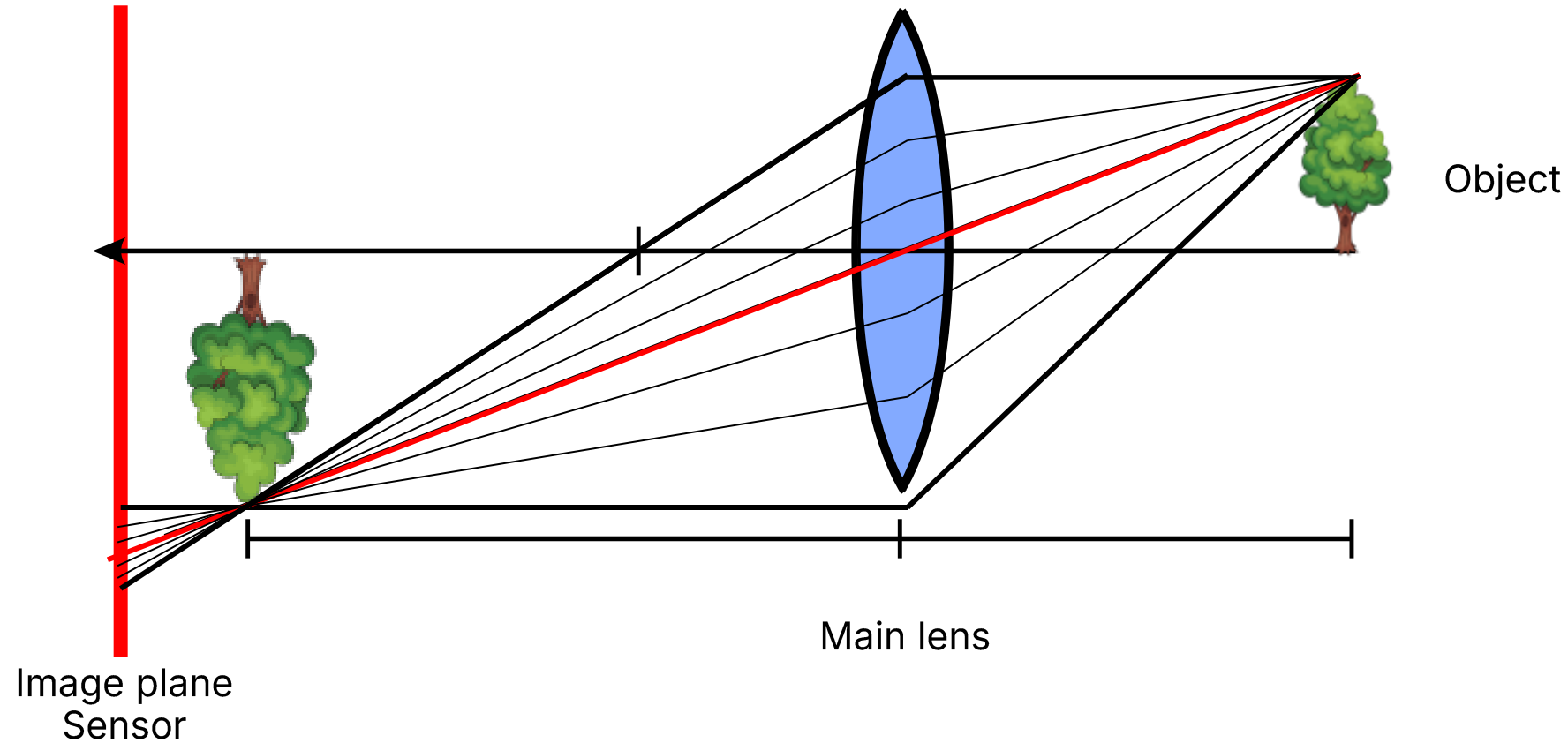
- Let's try to see what happen when we combine our elements
- We have a lens, a sensor, and a scene object



- The blur can be adjusted by changing the distance sensor-lens (focusing)
- The image of far objects will be sharp if the distance $F = f$ (lens' focal length)

Classical camera model

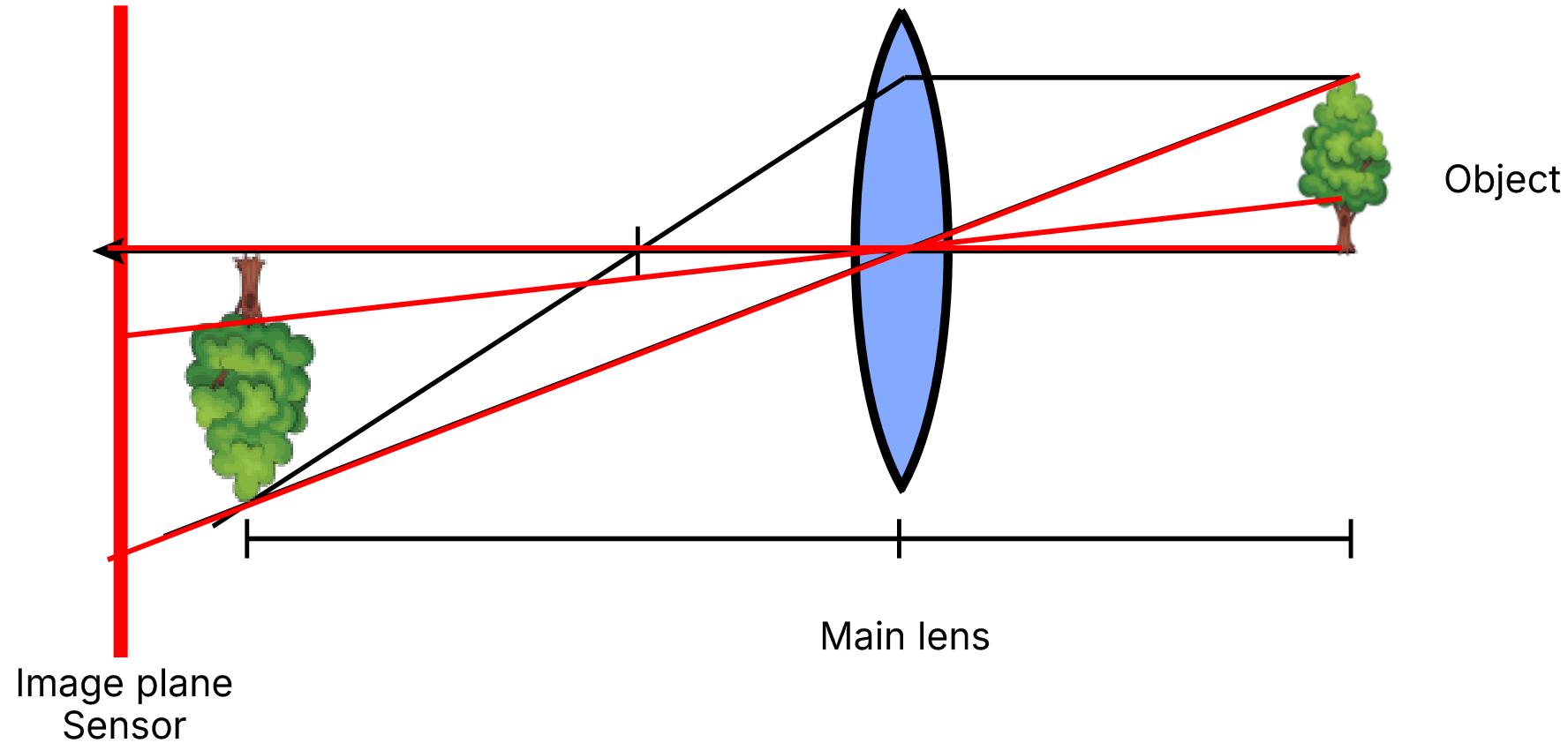
- Let's try to see what happen when we combine our elements
- We have a lens, a sensor, and a scene object



- Let's focus on the central ray
- It is the center of the blurred image

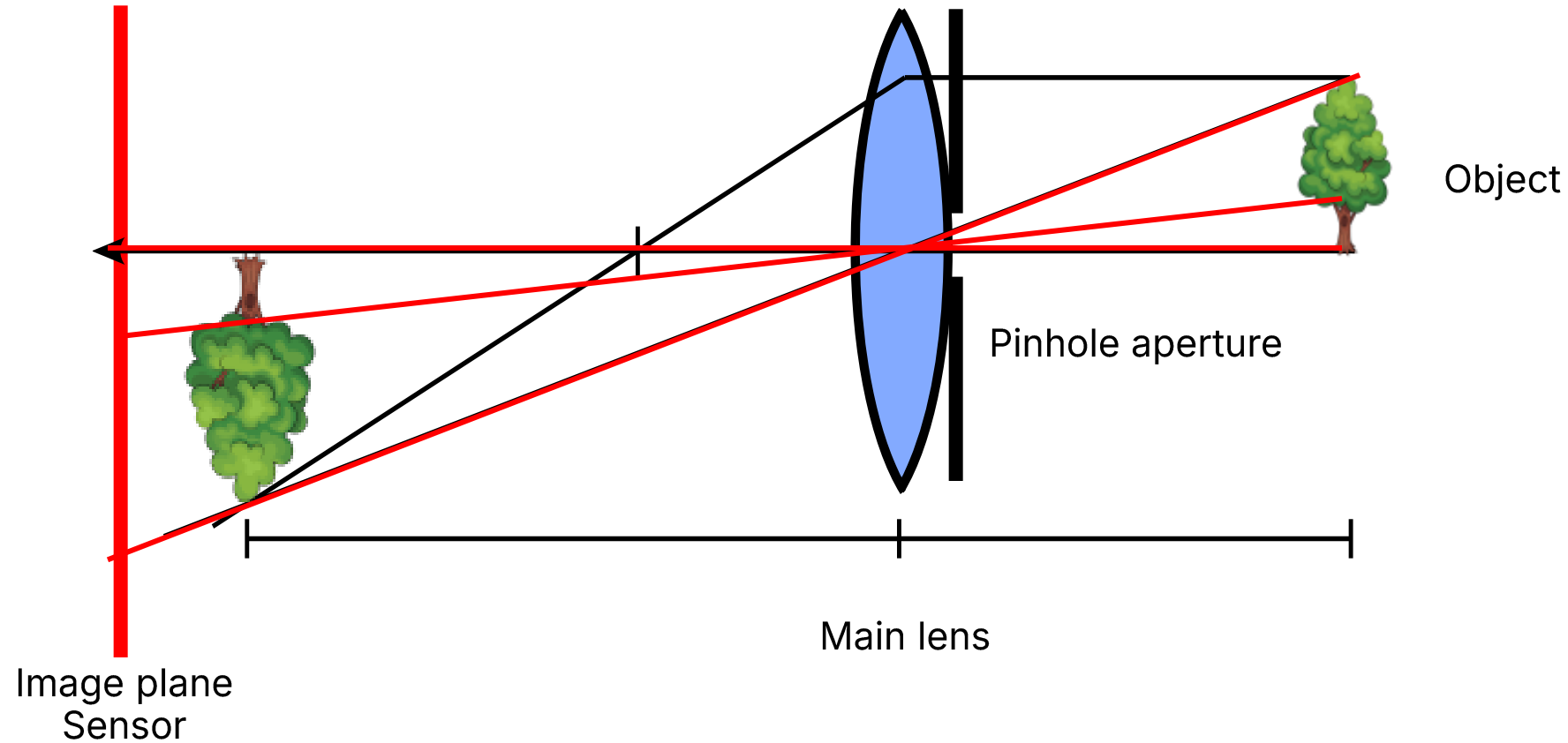
Classical camera model

- Each ray passing through the center will be the center of the blurred region



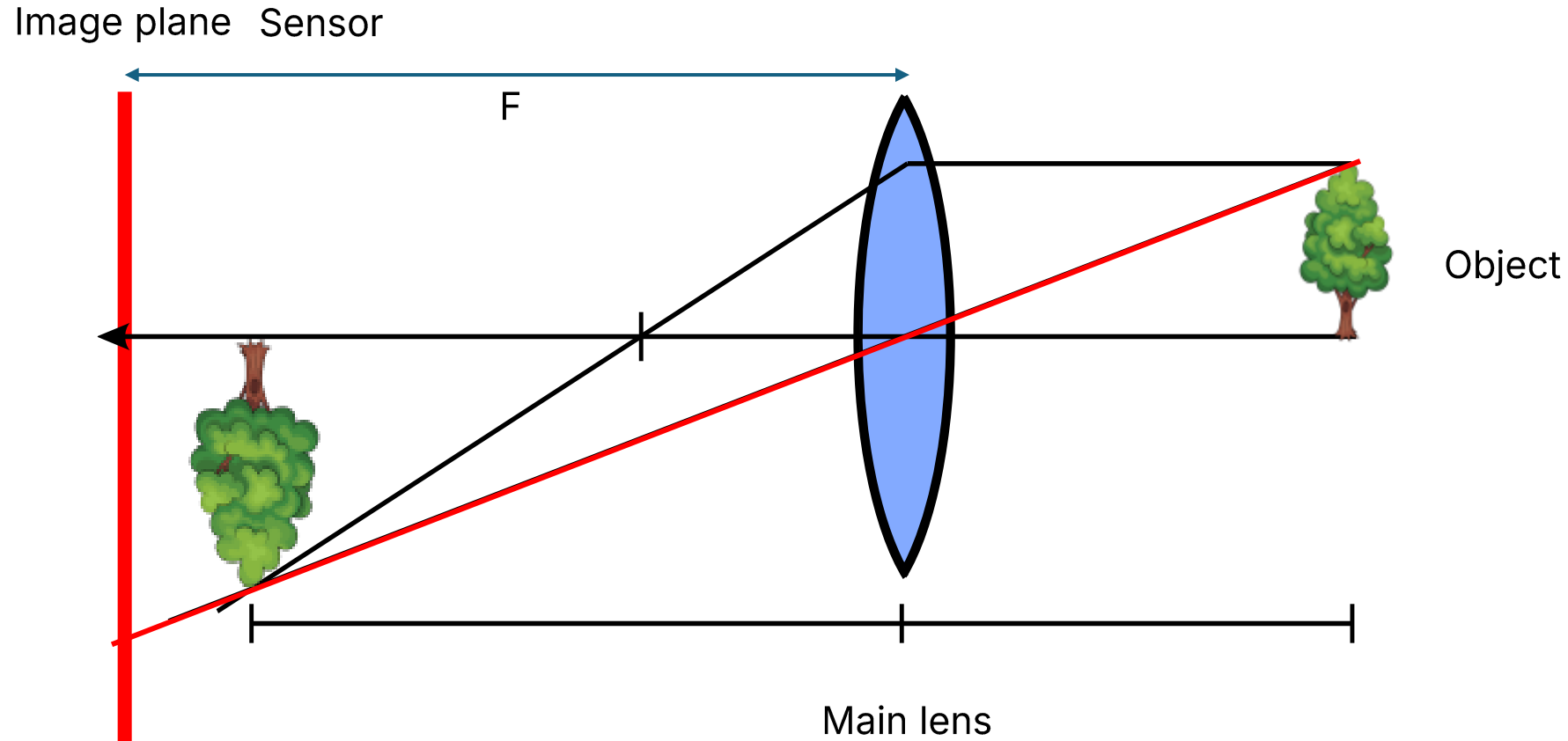
Classical camera model

- Each ray passing through the center will be the center of the blurred region
- By removing all other rays, it's a pinhole camera !



Pinhole camera model

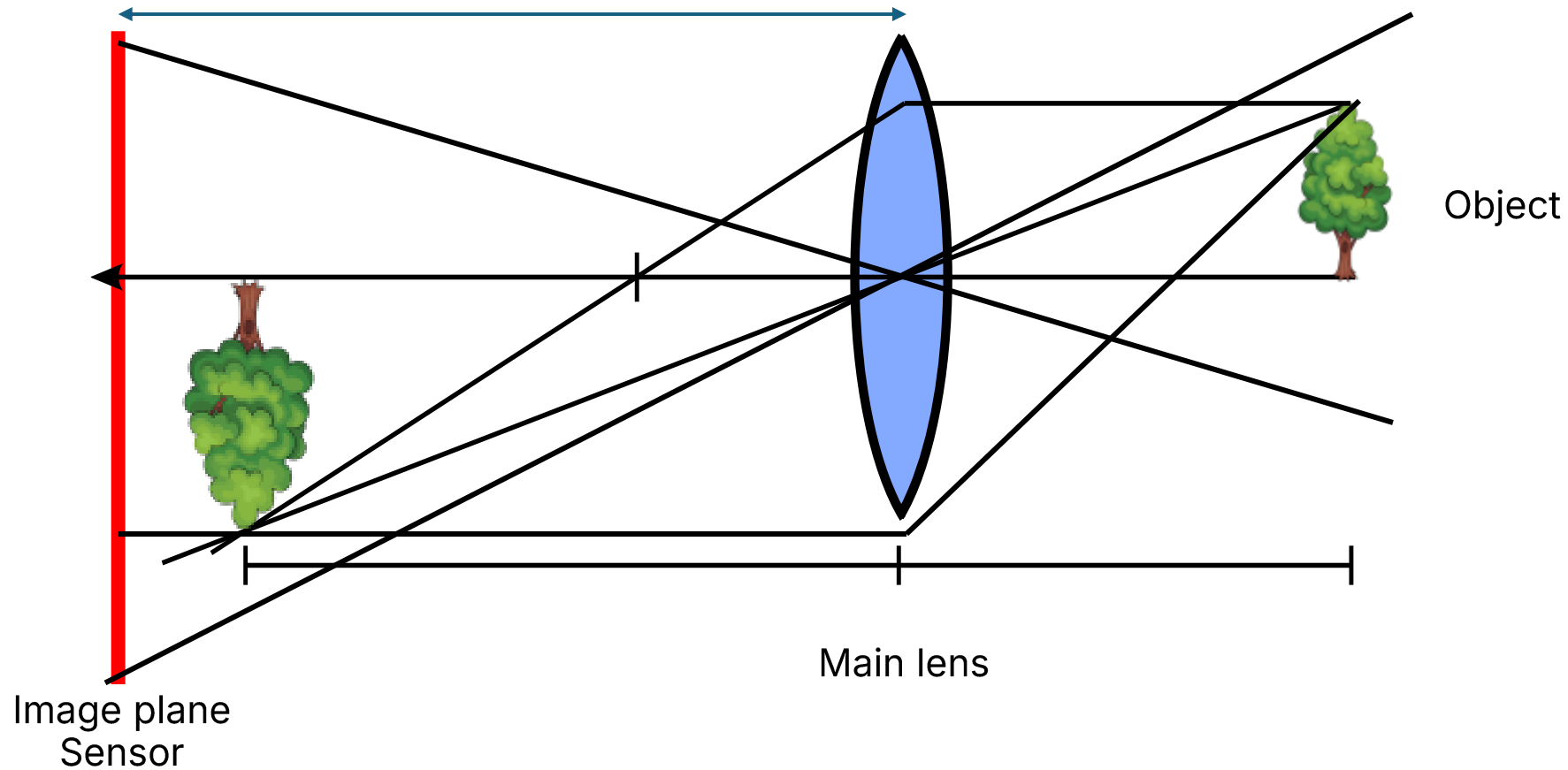
- Now, let's try to find the projection matrix



- We have 3 problems:
 - What is F ?
 - Finding the coordinates of the projected point to the sensor
 - Going from lens coordinates to sensor coordinates

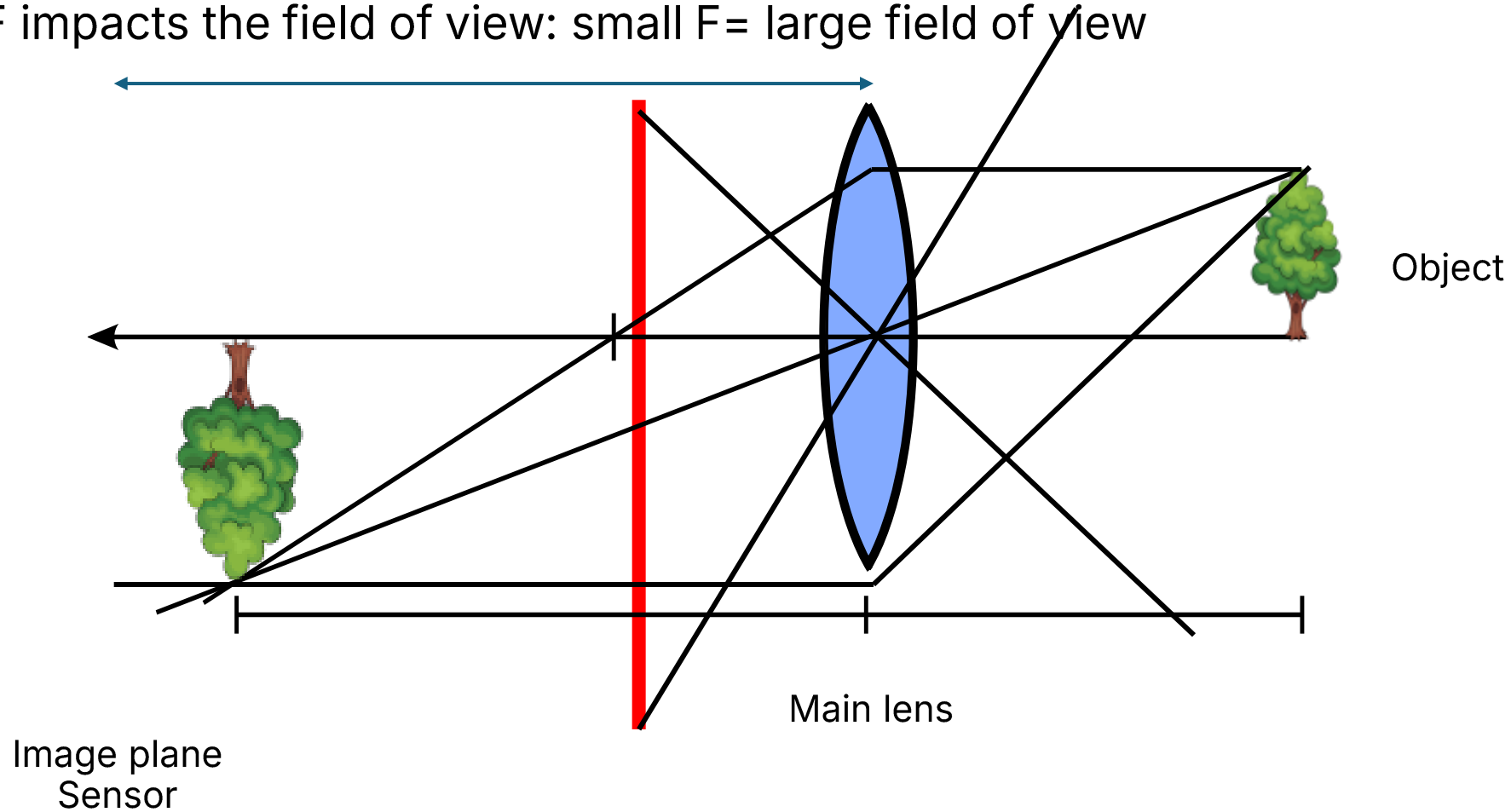
Pinhole camera model: What is F?

- Schematically, this is the situation, the lens is replaced by a pinhole
- F impacts the field of view : large F= small field of view



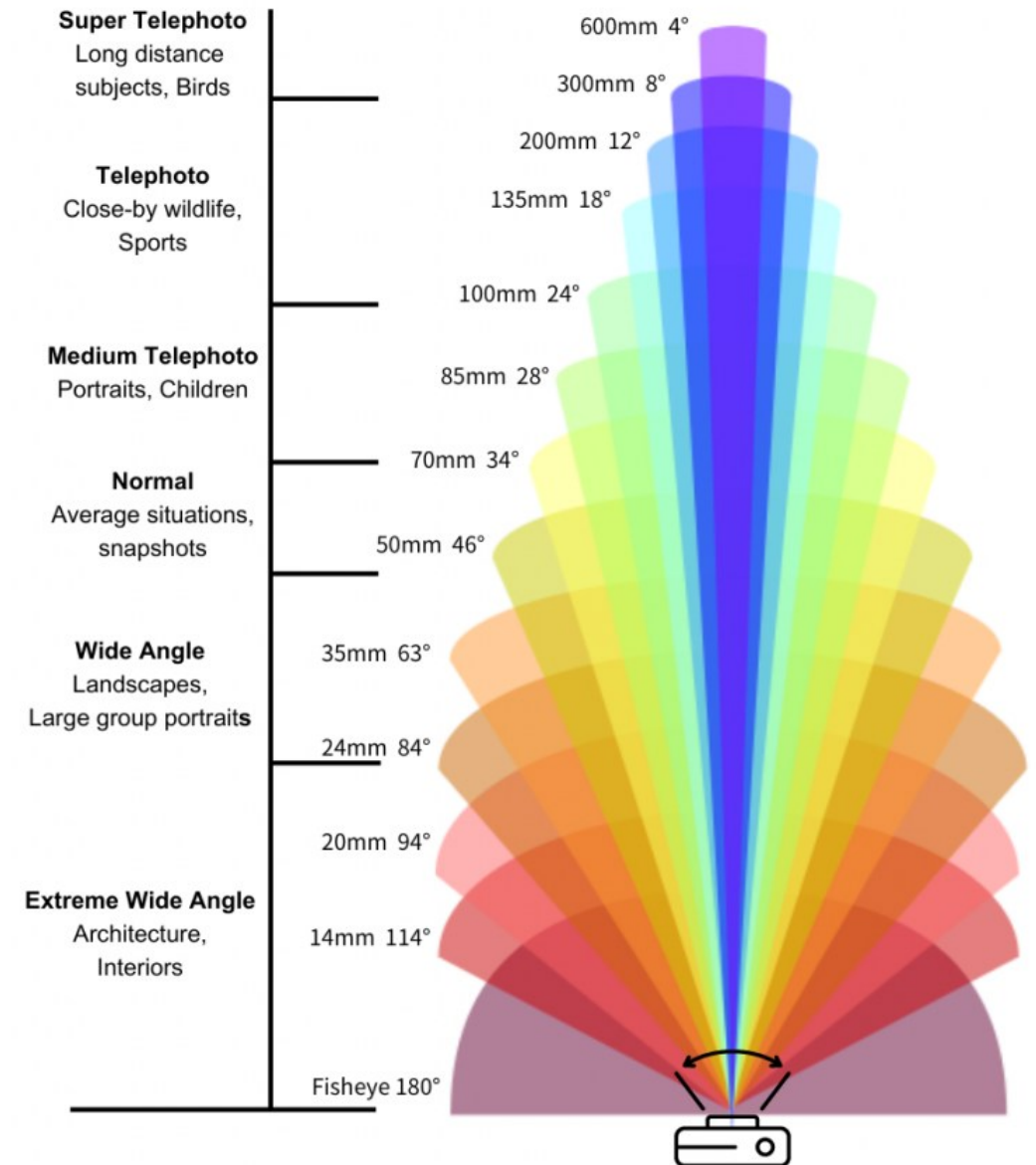
Pinhole camera model: What is F?

- Schematically, this is the situation, the lens is replaced by a pinhole
- F impacts the field of view: small F = large field of view



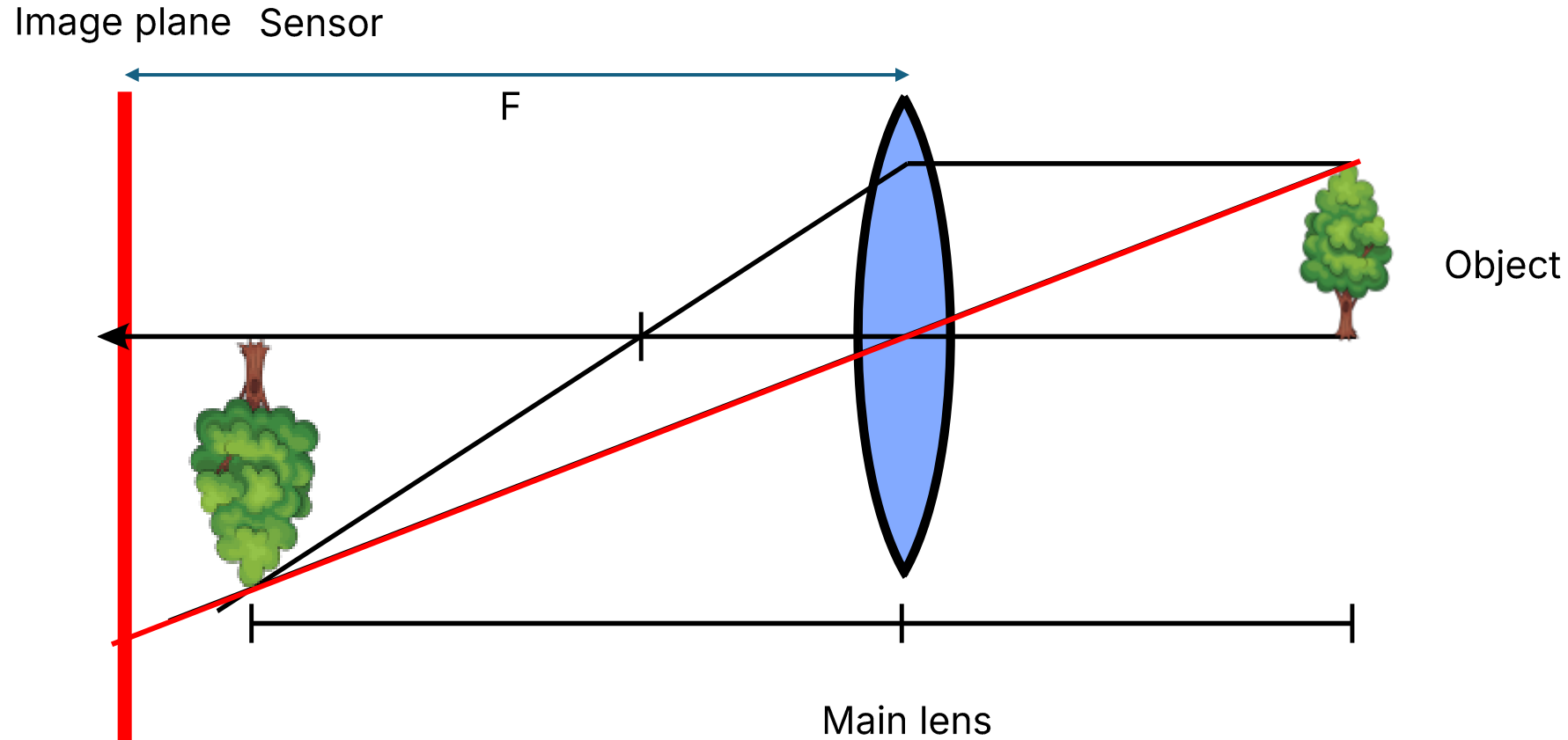
Pinhole camera model: What is F?

- There is a physical link between the focal length and the field of view.
- We can imitate a real camera graphics by
 - finding its field of view
 - Using our projection matrix



Pinhole camera model

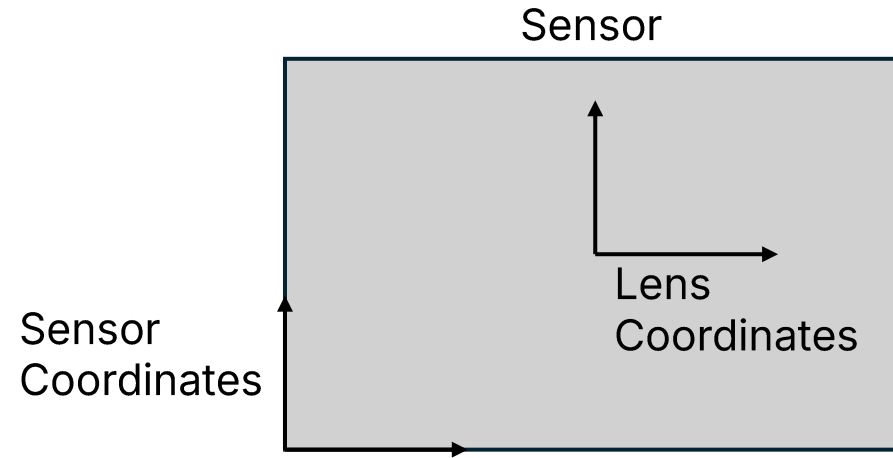
- Now, let's try to find the projection matrix



- We have 3 problems:
 - What is F ?
 - Finding the coordinates of the projected point to the sensor
 - Going from lens coordinates to sensor coordinates

Pinhole camera model - Finding the coordinates of the projected point to the sensor

- The lens is at the center of the sensor, they do not have the same origin

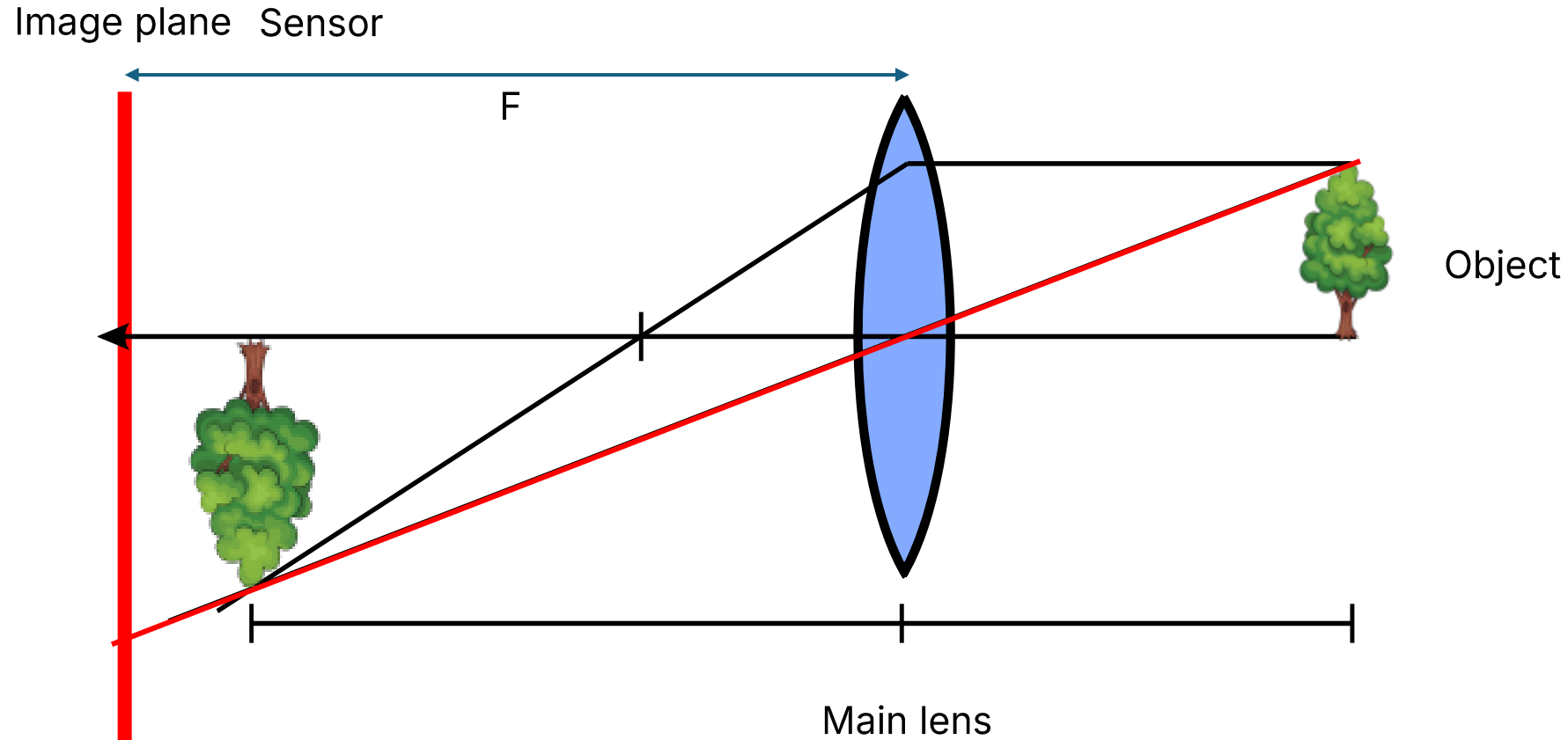


- We need to translate it

$$T_c = \begin{pmatrix} 1 & 0 & p_x \\ 0 & 1 & p_y \\ 0 & 0 & 1 \end{pmatrix}$$

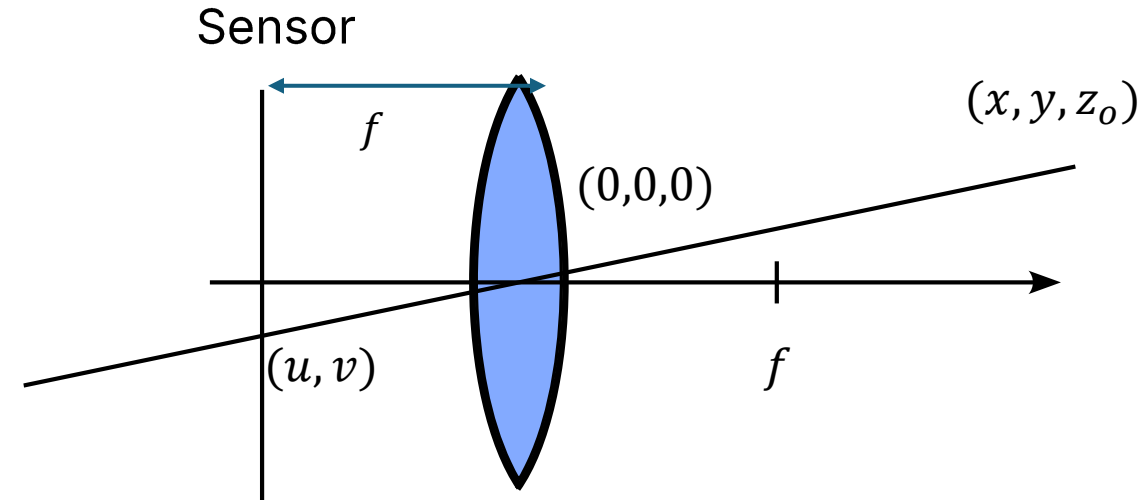
Pinhole camera model

- Now, let's try to find the projection matrix



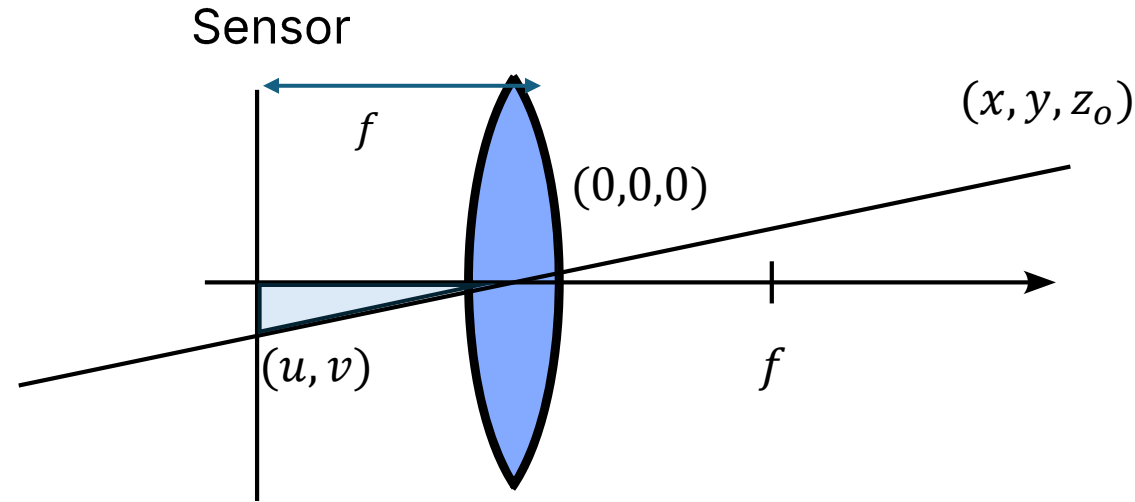
- We have 3 problems:
 - What is F ?
 - Finding the coordinates of the projected point to the sensor
 - Going from lens coordinates to sensor coordinates

Pinhole camera



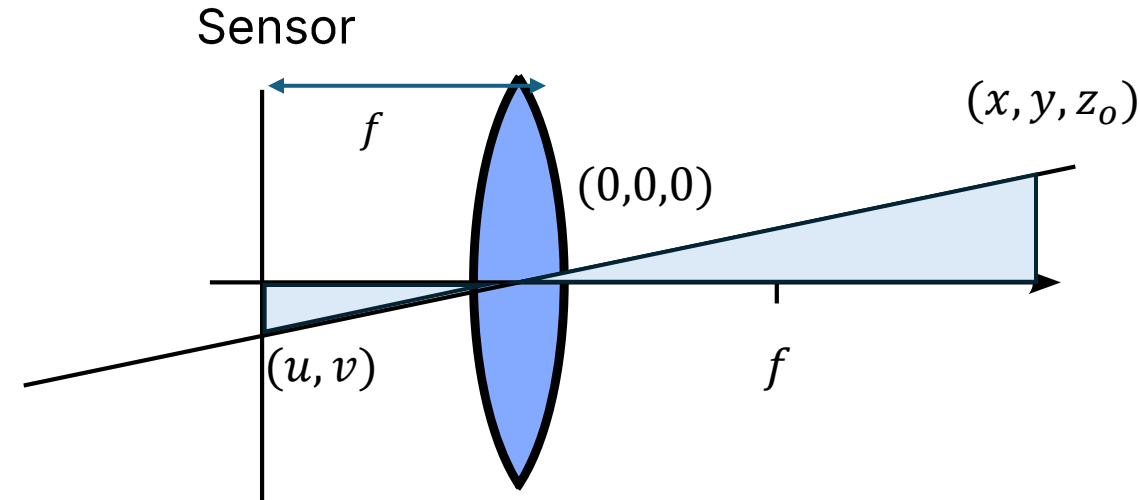
- We need to have the intersection of the ray with the sensor
- We need to project (x, y, z_o) on the sensor (at f) at coordinates (u, v)

Pinhole camera



- We need to have the intersection of the ray with the sensor
- We need to project (x, y, z_o) on the sensor (at f) at coordinates (u, v)
- Oh a triangle

Pinhole camera



- We need to have the intersection of the ray with the sensor
- We need to project (x, y, z_o) on the sensor (at f) at coordinates (u, v)
- Oh a triangle

- $\frac{u}{x} = \frac{v}{y} = \frac{f}{z_o} \Leftrightarrow uz_o = fx, \quad vz_o = fy$

- In homogeneous coordinates: $(fx, fy, z_o) = \begin{pmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z_o \\ 1 \end{pmatrix}$

Pinhole camera – Combining translation and sensor projection

We found projection

$$P_c = \begin{pmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

And translation

$$T_c = \begin{pmatrix} 1 & 0 & p_x \\ 0 & 1 & p_y \\ 0 & 0 & 1 \end{pmatrix}$$

Combining

$$K_c = T_c P_c = \begin{pmatrix} 1 & 0 & p_x \\ 0 & 1 & p_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} = \begin{pmatrix} f & 0 & p_x & 0 \\ 0 & f & p_y & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Pinhole camera

- The projection through a pinhole is given by this projection matrix

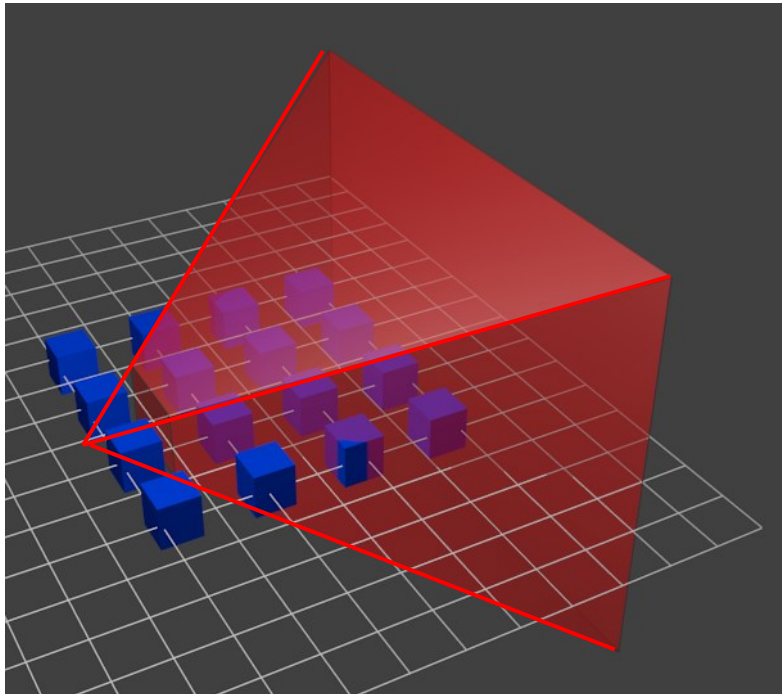
$$K_c = \begin{pmatrix} f & 0 & 0 & p_x \\ 0 & f & 0 & p_y \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Applied to any point of the scene:

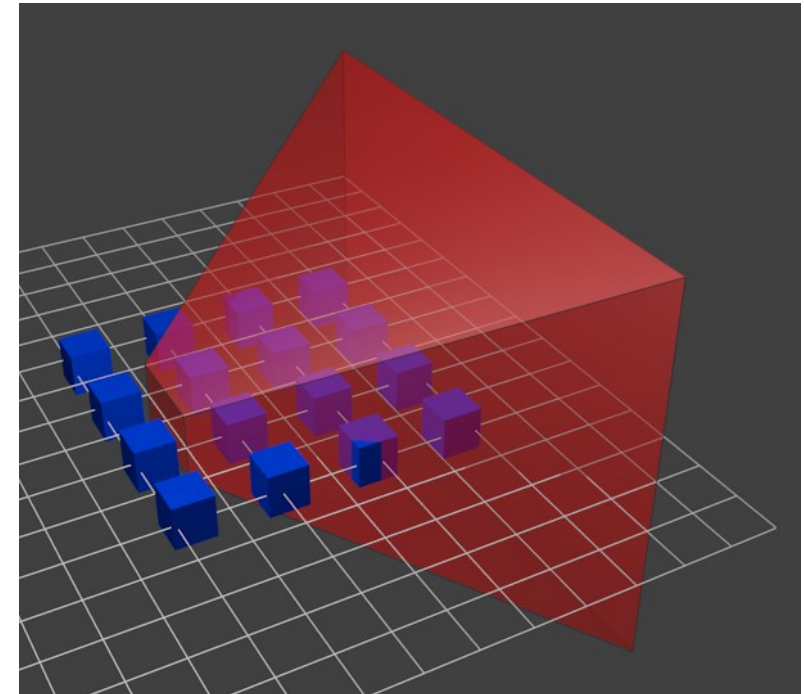
$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = \begin{pmatrix} f & 0 & p_x & 0 \\ 0 & f & p_y & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Link with opengl projection matrix

- A pinhole camera projects all the points to the image plane
 - But computationally, we want to cut far and near points
 - This gives the form of a truncated pyramid, the frustum
- We move again all our vertices, so that they are deformed following perspective !

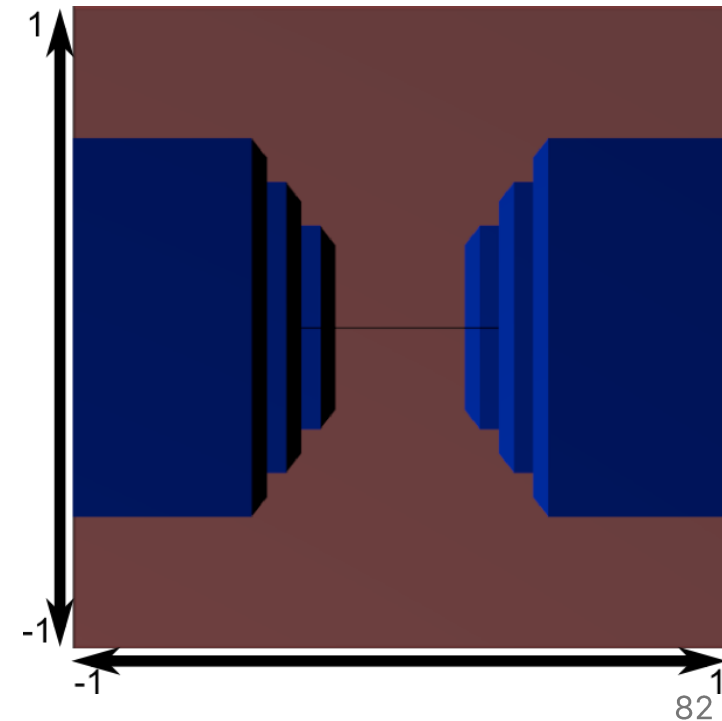
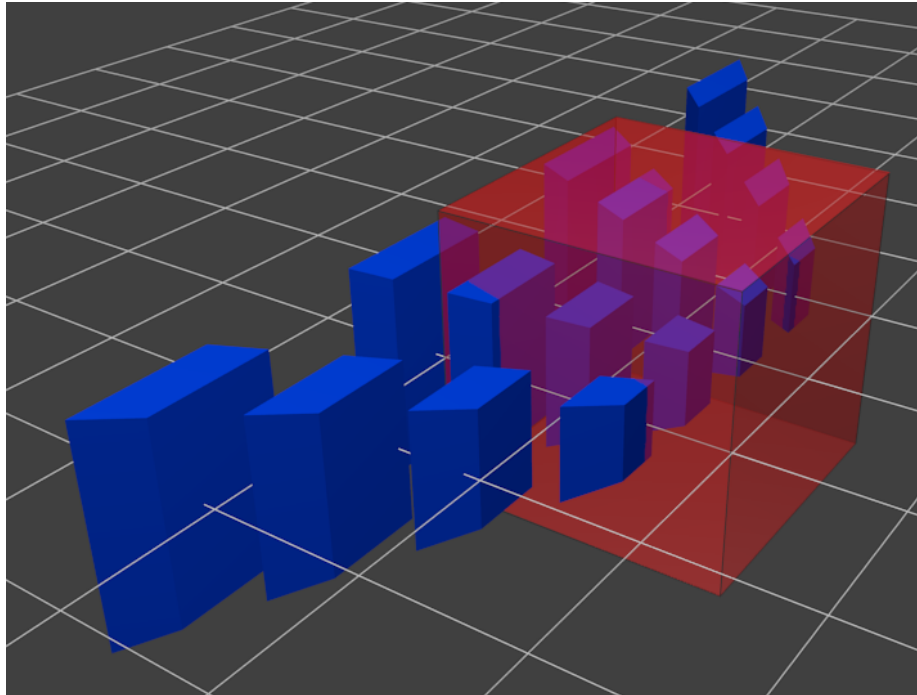
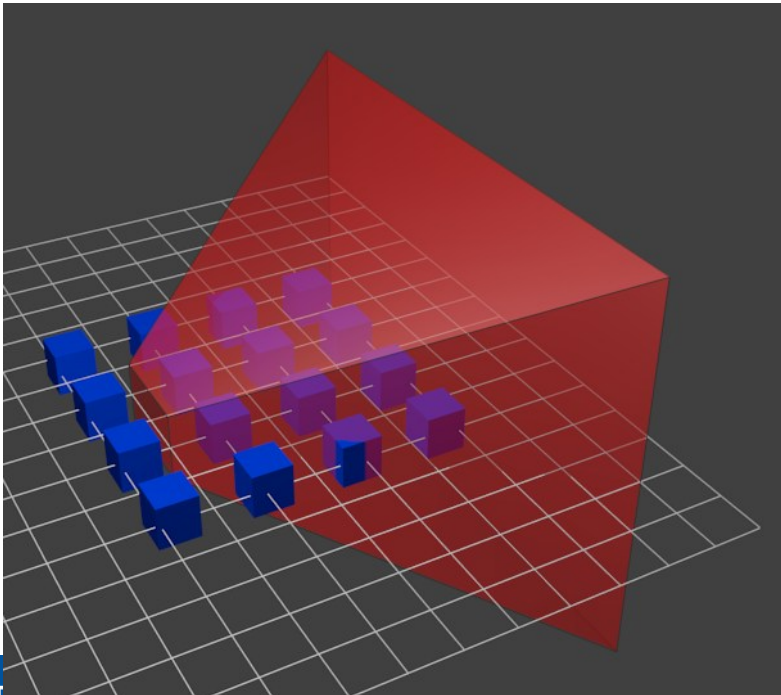


Truncate



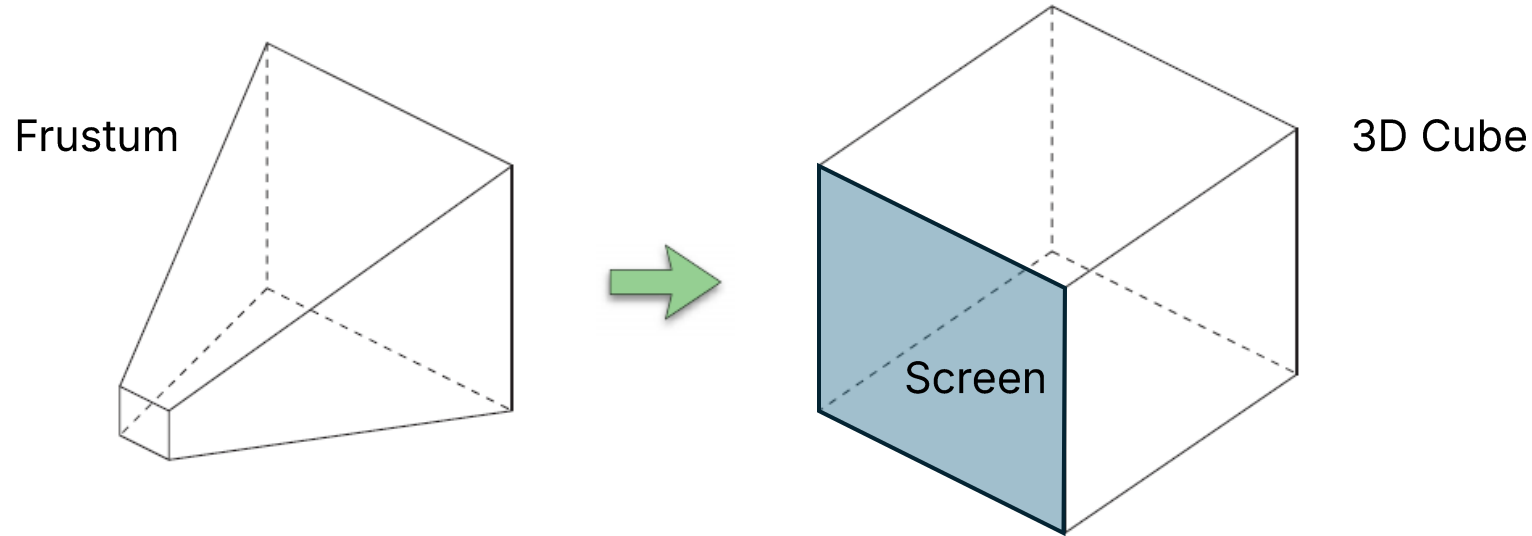
What is a projection matrix?

- Instead of projecting all the points to the image plane at once we will
 - 1. Deform the frustum to become a cube !
 - 2. Take an orthogonal projection of the cube as the image plane
- We move again all our vertices, so that they are deformed following perspective !



Projection Matrix

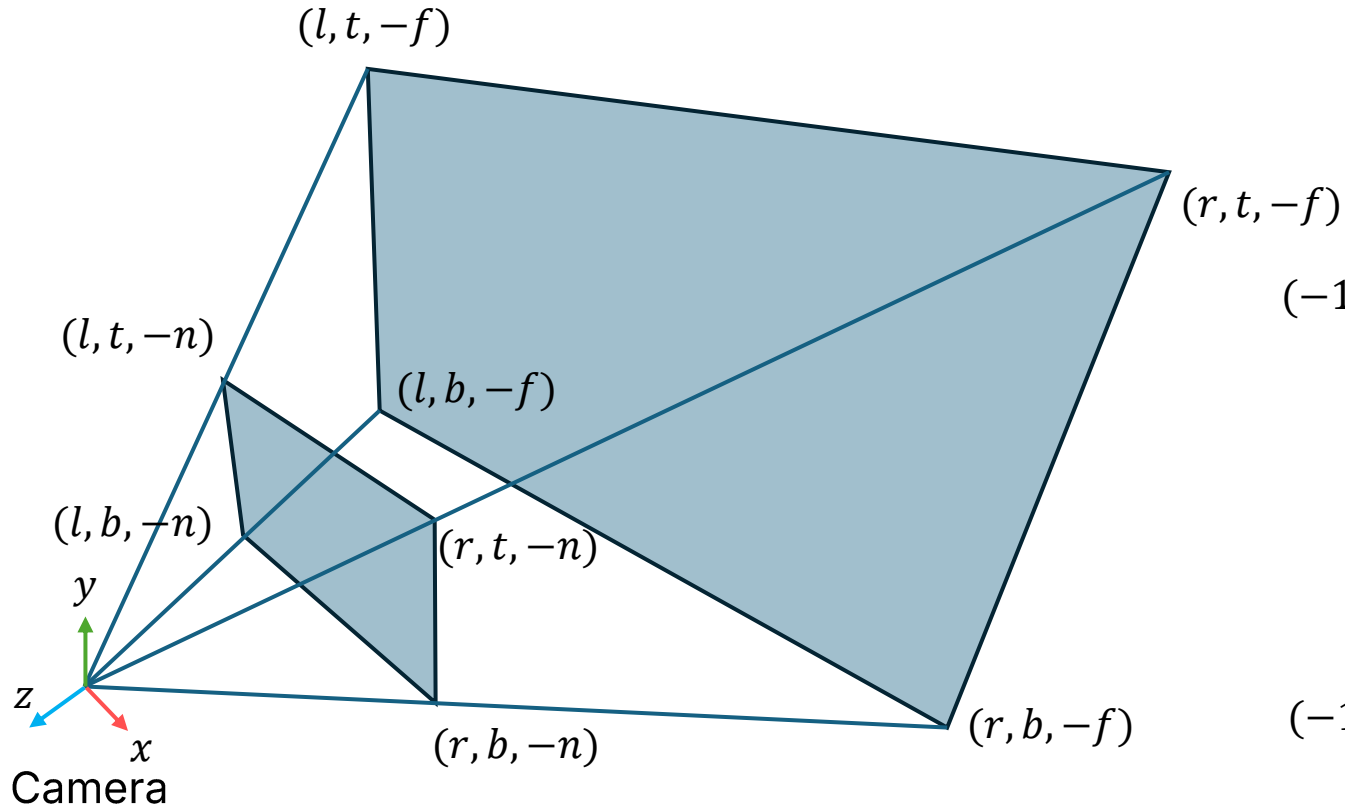
- We want to make a unique transformation that renders the perspective correctly



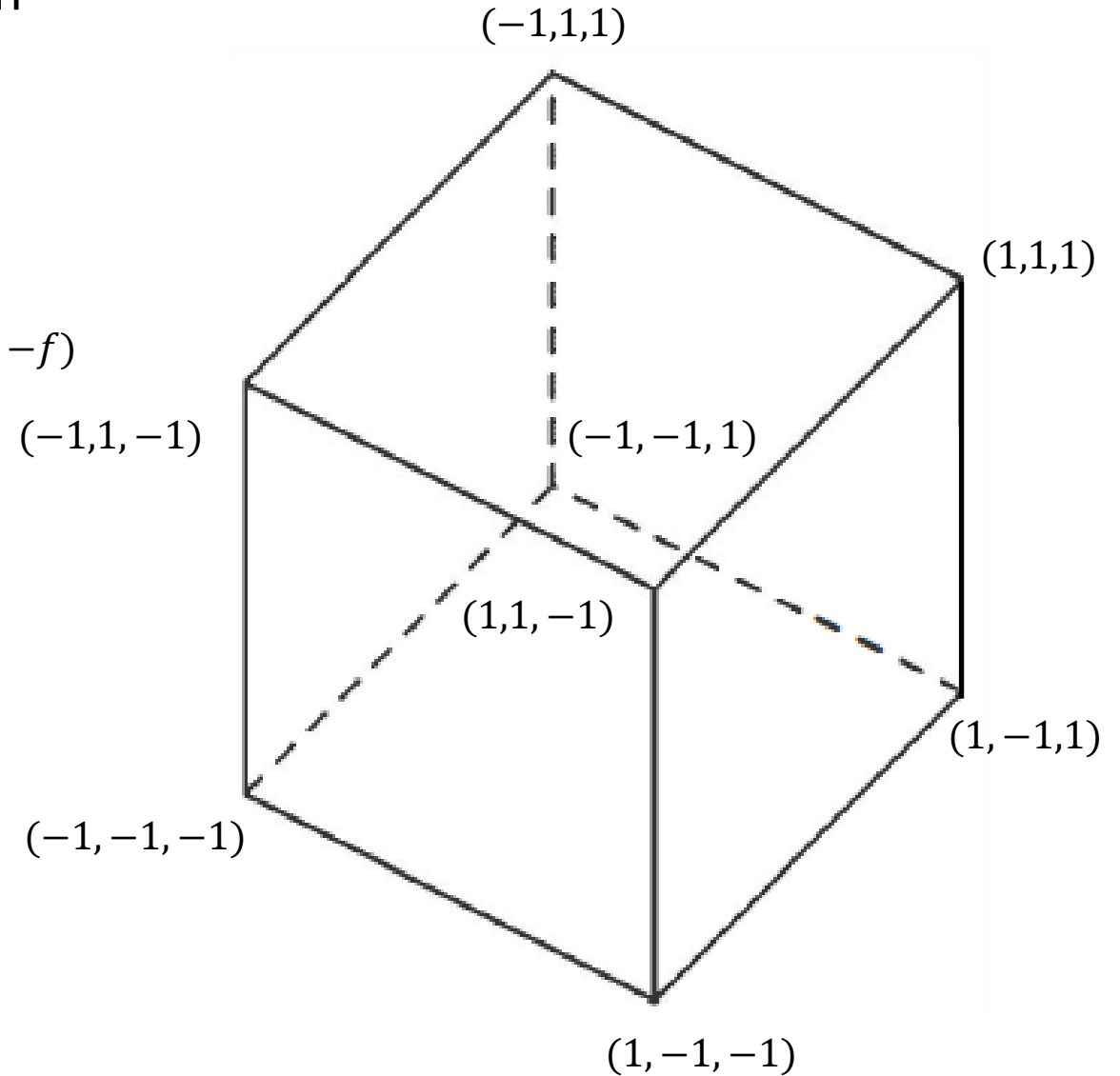
- The idea is to have a frustum (truncated pyramid) that represents the perspective we want.
 - Our scene will be the front face of this cube
- We want to find the mapping that transforms the frustum into a cube.
 - The front face of this cube can then be displayed on the screen.
- This cube is called the **Homogeneous Clip Space**.

Frustum and Cube labels

- Let's add coordinates to find our transformation

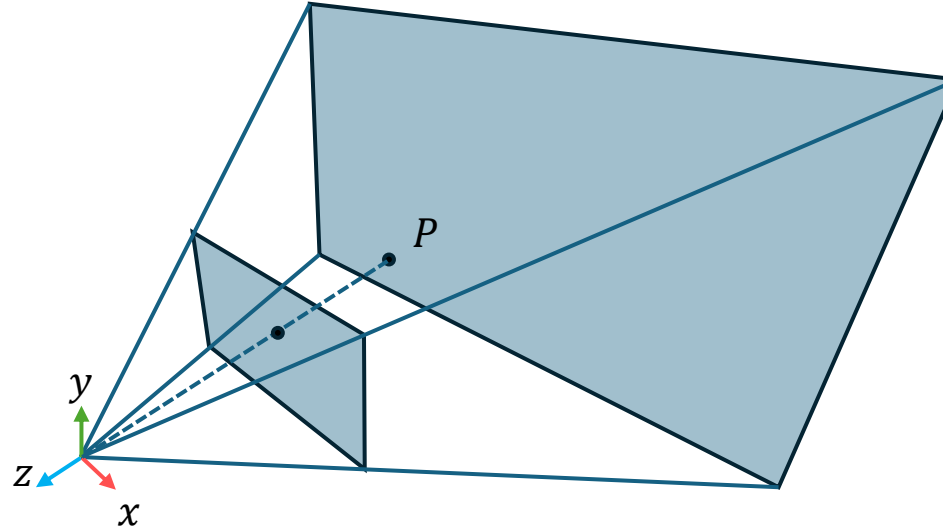


l = left, r = right
 t = top, b = bottom
 n = near, f = far



Projection matrix

- Let $P = (P_x, P_y, P_z, 1)$ be an homogeneous point in camera space that lies inside the view frustum.



- The projected x and y coordinates of P on the near plane:
 - $x = -\frac{n}{P_z} P_x$ $y = -\frac{n}{P_z} P_y$ (by similar triangles) (check yourself with previous lesson)
 - The point satisfy $l \leq x \leq r$ and $b \leq y \leq t$ (why?)
- We want to map ranges to the $[-1, 1]$ range to fit the view frustum into homogeneous clip space (the cube).
 - Linear transform: $x' = (x - l) \frac{2}{r-l} - 1$ $y' = (y - b) \frac{2}{t-b} - 1$

Convince yourself

Projection matrix

- We want to map ranges to the $[-1, 1]$ range to fit the view frustum into homogeneous clip space. Linear transform:

$$\bullet \ x' = (x - l) \frac{2}{r-l} - 1 \quad y' = (y - b) \frac{2}{t-b} - 1$$

- By substituting with the original x and y :

$$\bullet \ x' = \frac{2n}{r-l} \left(-\frac{P_x}{P_z} \right) - \frac{r+l}{r-l} \quad y' = \frac{2n}{t-b} \left(-\frac{P_y}{P_z} \right) - \frac{t+b}{t-b}$$

$$x = -\frac{n}{P_z} P_x \quad y = -\frac{n}{P_z} P_y$$

Convince yourself

- For the moment, it was not that difficult.
 - We mapped our point P to the image plane
 - We mapped our image plane to the range $[-1, 1]$
- z mapping seems more difficult, due to the projective equations
- The mapping varies in $1/z$

Projection matrix

$$l \equiv \frac{1}{z_3} = \frac{1}{z_2}(1 - t) + \frac{1}{z_6}t$$

- To map the z-coordinate in the $[-1, 1]$ range, we still have

$$-f \leq P_z \leq -n$$

- We want to find a function that maps $-n \mapsto -1$ and $-f \mapsto 1$
- Since z-coordinates (depth) must have their reciprocals interpolated during rasterization, we construct the (unknown) mapping linear in $\frac{1}{z}$

$$z' = A \begin{pmatrix} 1 \\ \frac{1}{z} \end{pmatrix} + B$$

Projection matrix

- Since z-coordinates (depth) must have their reciprocals interpolated during rasterization, we construct the (unknown) mapping linear in $\frac{1}{z}$

$$z' = A \left(\frac{1}{z} \right) + B$$

- We solve for the unknown A and B by plugging our constraints:

- $-1 = \frac{A}{-n} + B$ and $1 = \frac{A}{-f} + B$

- Solving results in:

- $A = \frac{2nf}{f-n}$ and $B = \frac{f+n}{f-n}$

- So, the z-coordinate of P is mapped to $[-1, 1]$ by

$$-n \mapsto -1 \text{ and } -f \mapsto 1$$

$$z' = -\frac{2nf}{f-n} \left(-\frac{1}{P_z} \right) + \frac{f+n}{f-n}$$

Convince yourself

We introduce the $-$ sign to make this equation more similar to x' and y'

Projection matrix

- We now have
- $x' = \frac{2n}{r-l} \left(-\frac{P_x}{P_z} \right) - \frac{r+l}{r-l} \quad y' = \frac{2n}{t-b} \left(-\frac{P_y}{P_z} \right) - \frac{t+b}{t-b} \quad z' = -\frac{2nf}{f-n} \left(-\frac{1}{P_z} \right) + \frac{f+n}{f-n}$
- But, those equations are **not** linear in P_x, P_y, P_z , but in $P_x, P_y, \frac{1}{P_z}$
- So we cannot transform those equations in a matrix form !
- What should we do ?

Projection matrix

- We now have

$$x' = \frac{2n}{r-l} \left(-\frac{P_x}{P_z} \right) - \frac{r+l}{r-l} \quad y' = \frac{2n}{t-b} \left(-\frac{P_y}{P_z} \right) - \frac{t+b}{t-b} \quad z' = -\frac{2nf}{f-n} \left(-\frac{1}{P_z} \right) + \frac{f+n}{f-n}$$

- But, those equations are **not** linear in P_x, P_y, P_z , but in $P_x, P_y, \frac{1}{P_z}$
- Yes! Homogeneous coordinates to the rescue !
- A point $P' = (x', y', z') \in \mathbb{R}^3$ is equivalent to the 4D homogeneous point $P' = (-x'P_z, -y'P_z, -z'P_z, -P_z)$, after division by the w-coordinate.
- Rewriting our equations in homogeneous space:

$$-x'P_z = \frac{2n}{r-l}P_x + \frac{r+l}{r-l}P_z \quad -y'P_z = \frac{2n}{t-b}P_y + \frac{t+b}{t-b}P_z \quad -z'P_z = -\frac{2nf}{f-n} - \frac{f+n}{f-n}P_z \quad w = -P_z$$

- We are now linear in P_x, P_y, P_z ! Matrix form !

Projection matrix

$$-x'P_z = \frac{2n}{r-l}P_x + \frac{r+l}{r-l}P_z, \quad -y'P_z = \frac{2n}{t-b}P_y + \frac{t+b}{t-b}P_z, \quad -z'P_z = -\frac{2nf}{f-n} - \frac{f+n}{f-n}P_z, \quad w = -P_z$$

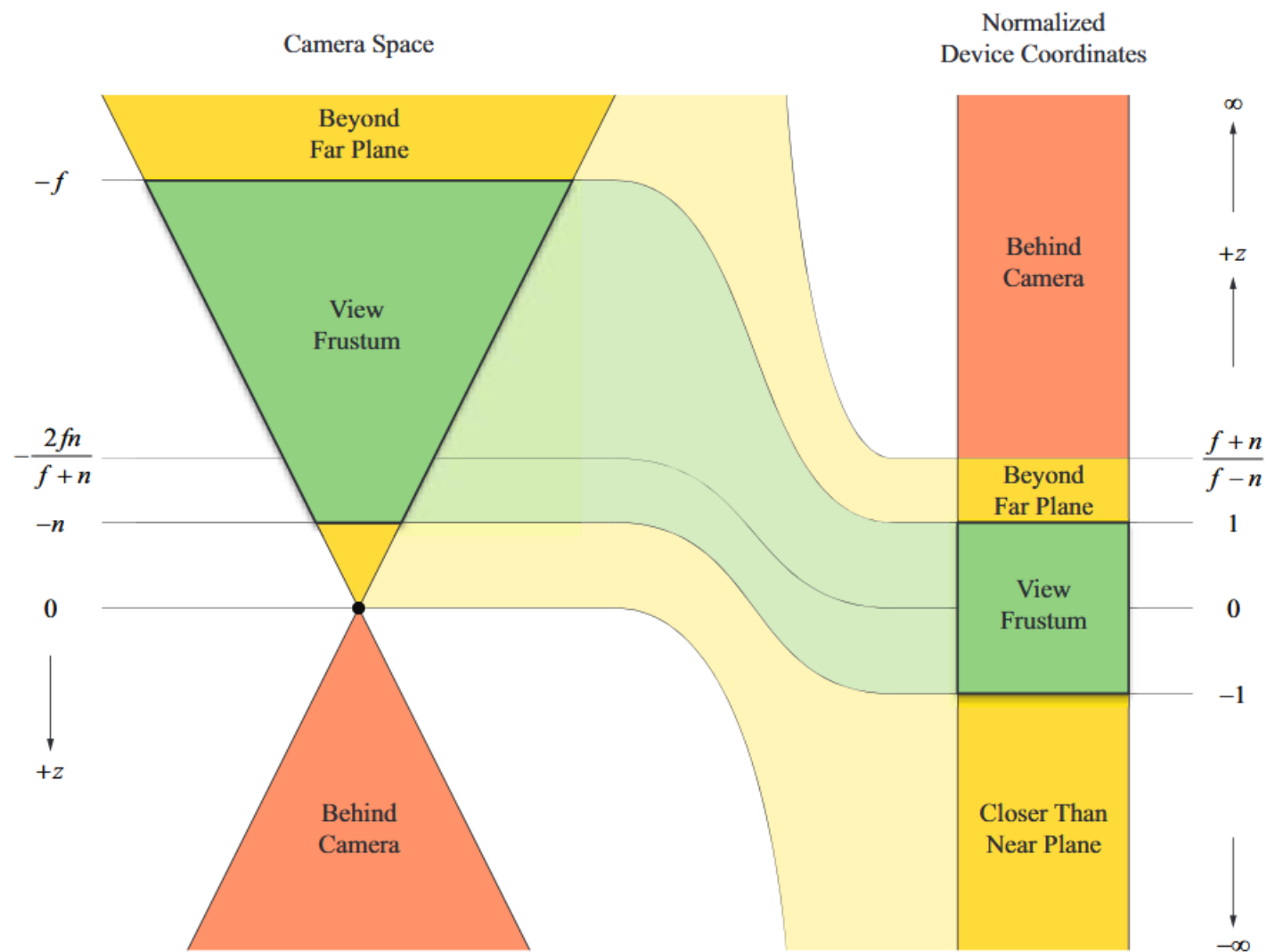
- We are now linear in P_x, P_y, P_z ! Matrix form !

$$P' = \begin{pmatrix} -x'P_z \\ -y'P_z \\ -z'P_z \\ -P_z \end{pmatrix} = \begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2nf}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} P_x \\ P_y \\ P_z \\ 1 \end{pmatrix} = M_f P$$

The division by $-P_z$ is done in the shaders

- This matrix M_f , sends points in **clip coordinates**, which allows to correctly project a point with a perspective, by transforming the frustum into the cube.

Effect of the Normalized Device Coordinates



Why behind is here?

Why closer is here?

Figure 5.16. This diagram illustrates how z coordinates are mapped from camera space to normalized device coordinates by the standard perspective projection matrix.

Projection matrix – Interpretation

$$P' = \begin{pmatrix} x' \\ y' \\ z' \\ w' \end{pmatrix} = M_f P = \begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2nf}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} P_x \\ P_y \\ P_z \\ 1 \end{pmatrix}$$

- $r - l = w$ the width of the image plane
- $t - b = h$ the height of the image plane
- $f - n = d$ the depth of the frustum (all visible objects in this depth)

$$P' = \begin{pmatrix} x' \\ y' \\ z' \\ w' \end{pmatrix} = M_f P = \begin{pmatrix} \frac{2n}{w} & 0 & \frac{r+l}{w} & 0 \\ 0 & \frac{2n}{h} & \frac{t+b}{h} & 0 \\ 0 & 0 & -\frac{f+n}{d} & -\frac{2nf}{d} \\ 0 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} P_x \\ P_y \\ P_z \\ 1 \end{pmatrix}$$

Projection matrix – Interpretation

$$P' = \begin{pmatrix} x' \\ y' \\ z' \\ w' \end{pmatrix} = M_f P = \begin{pmatrix} \frac{2n}{w} & 0 & \frac{r+l}{w} & 0 \\ 0 & \frac{2n}{h} & \frac{t+b}{h} & 0 \\ 0 & 0 & -\frac{f+n}{d} & -\frac{2nf}{d} \\ 0 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} P_x \\ P_y \\ P_z \\ 1 \end{pmatrix}$$

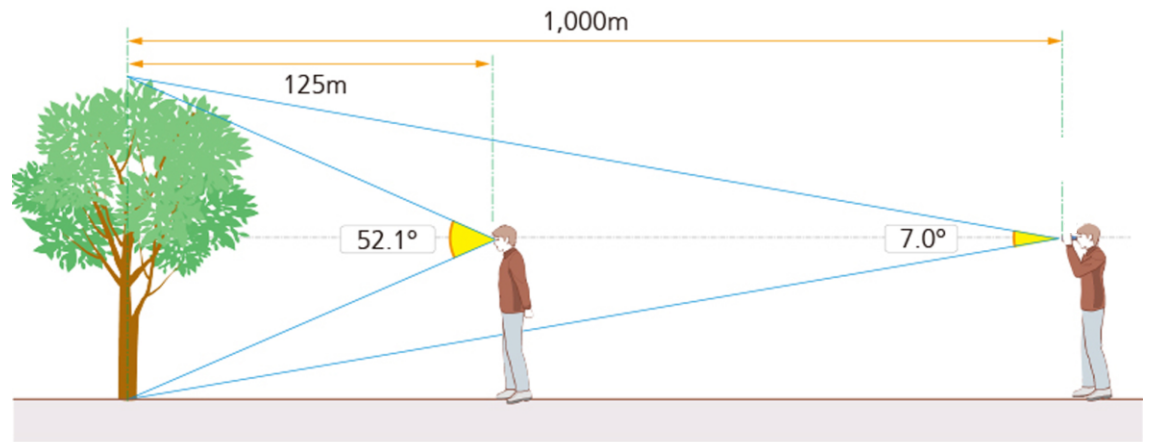
- $\frac{2n}{w}, \frac{2n}{h}$ are scaling factors for the x and y coordinates to $[-1, 1]$
- 0 indicates no coupling between x, y, z = no skew
- $\frac{r+l}{w}, \frac{t+b}{h}$ adds offsets to the x –, y –coordinates, (non symmetrical frustum)
 - It moves the center of the frustum horizontally/vertically to align with the optical axis
- $-\frac{f+n}{d}$ scales the z -coordinate to be in $[-1, 1]$
- $-\frac{2nf}{d}$ adds an offset to the z -coordinate, works with the previous term

Easier to use Frustum

- The projection matrix is solved.
- But, it is often not easy to decide on the parameters n, f, t, b, l, r
- Is there an easier way?

Easier to use Frustum

- The projection matrix is solved.
- But, it is often not easy to decide on the parameters n, f, t, b, l, r
- Is there an easier way?



- Usual cameras are symmetric.
- And we are more interested in specifying the horizontal field-of-view (= 2 angles)
- Let's change our projection matrix !

Projection matrix – Symmetrical Frustum

$$P' = \begin{pmatrix} x' \\ y' \\ z' \\ w' \end{pmatrix} = M_f P = \begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2nf}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} P_x \\ P_y \\ P_z \\ 1 \end{pmatrix}$$

- If the frustum is symmetrical, $l = -r$, $b = -t$, so

$$\frac{r+l}{r-l} = 0$$

$$\frac{t+b}{t-b} = 0$$

$$r-l = 2r$$

$$t-b = 2t$$

- No more offsets

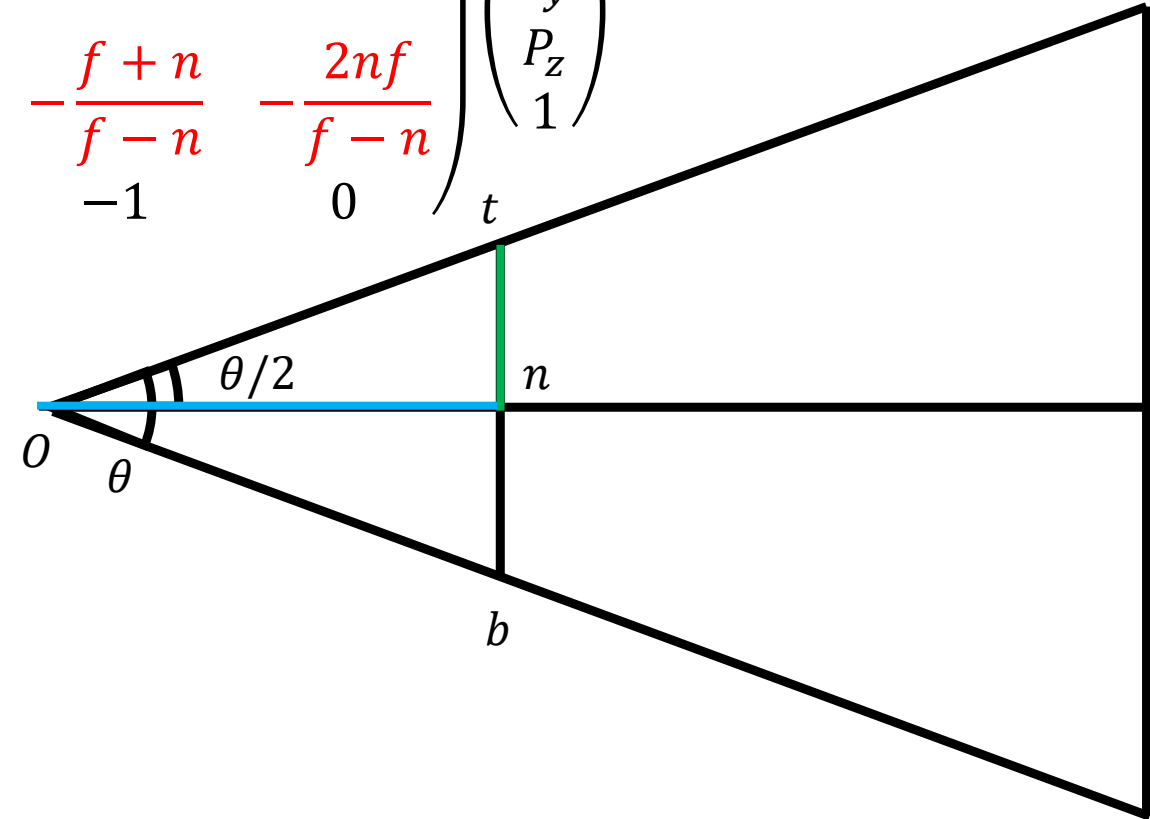
$$P' = \begin{pmatrix} x' \\ y' \\ z' \\ w' \end{pmatrix} = M_{fsym} P = \begin{pmatrix} \frac{n}{r} & 0 & 0 & 0 \\ 0 & \frac{n}{t} & 0 & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2nf}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} P_x \\ P_y \\ P_z \\ 1 \end{pmatrix}$$

Projection matrix – Symmetrical Frustum

$$P' = \begin{pmatrix} x' \\ y' \\ z' \\ w' \end{pmatrix} = M_{fsym} P = \begin{pmatrix} \frac{n}{r} & 0 & 0 & 0 \\ 0 & \frac{n}{t} & 0 & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2nf}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} P_x \\ P_y \\ P_z \\ 1 \end{pmatrix}$$

$$\bullet \tan \frac{\theta}{2} = \frac{t}{n} \Leftrightarrow n = \frac{t}{\tan \frac{\theta}{2}}$$

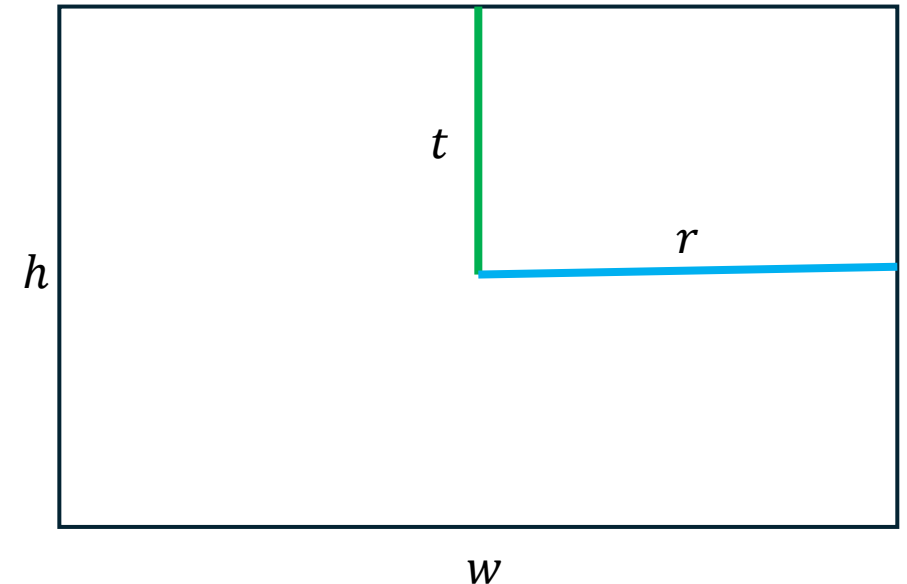
$$\bullet M_{fsym} = \begin{pmatrix} \frac{t}{r \tan \frac{\theta}{2}} & 0 & 0 & 0 \\ 0 & \frac{t}{t \tan \frac{\theta}{2}} & 0 & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2nf}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$



Projection matrix – Symmetrical Frustum

$$M_{fsym} = \begin{pmatrix} \frac{t}{r \tan \frac{\theta}{2}} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan \frac{\theta}{2}} & 0 & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2nf}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

- We still have t/r in the first element
- To remove it, we introduce the **aspect ratio**
- $ar = \frac{h}{w}$, so that $\frac{r}{w} = \frac{t}{h} \Leftrightarrow r = t \frac{w}{h} = t \cdot ar$
- It is a conversion factor between the two directions. $r = t \cdot ar$



Projection matrix – Symmetrical Frustum

$$M_{fsym} = \begin{pmatrix} \frac{t}{r \tan \frac{\theta}{2}} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan \frac{\theta}{2}} & 0 & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2nf}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix} = \begin{pmatrix} \frac{1}{ar \tan \frac{\theta}{2}} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan \frac{\theta}{2}} & 0 & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2nf}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

- With $r = t.ar$ we arrived at the final form of our matrix!
- To create it in OpenGL, we only need to specify:
 - Horizontal field-of-view θ
 - Aspect ratio ar
 - Near and far distances n, f

Pinhole Camera – Sensor

- The pinhole camera projective transformation is

$$P_c T_c = \begin{pmatrix} f & 0 & p_x & 0 \\ 0 & f & p_y & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

- If we compare it to our projection matrix

$$M_f = \begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2nf}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

- We see that the focal length play the role of the scale
- And the p_x, p_y the role of the offsets !

Projection matrix – Final steps

- Now that we have derived the projection matrix
 - And gained some intuition..
-
- Let's focus on the last steps to render to the screen !

Normalized Device Coordinates

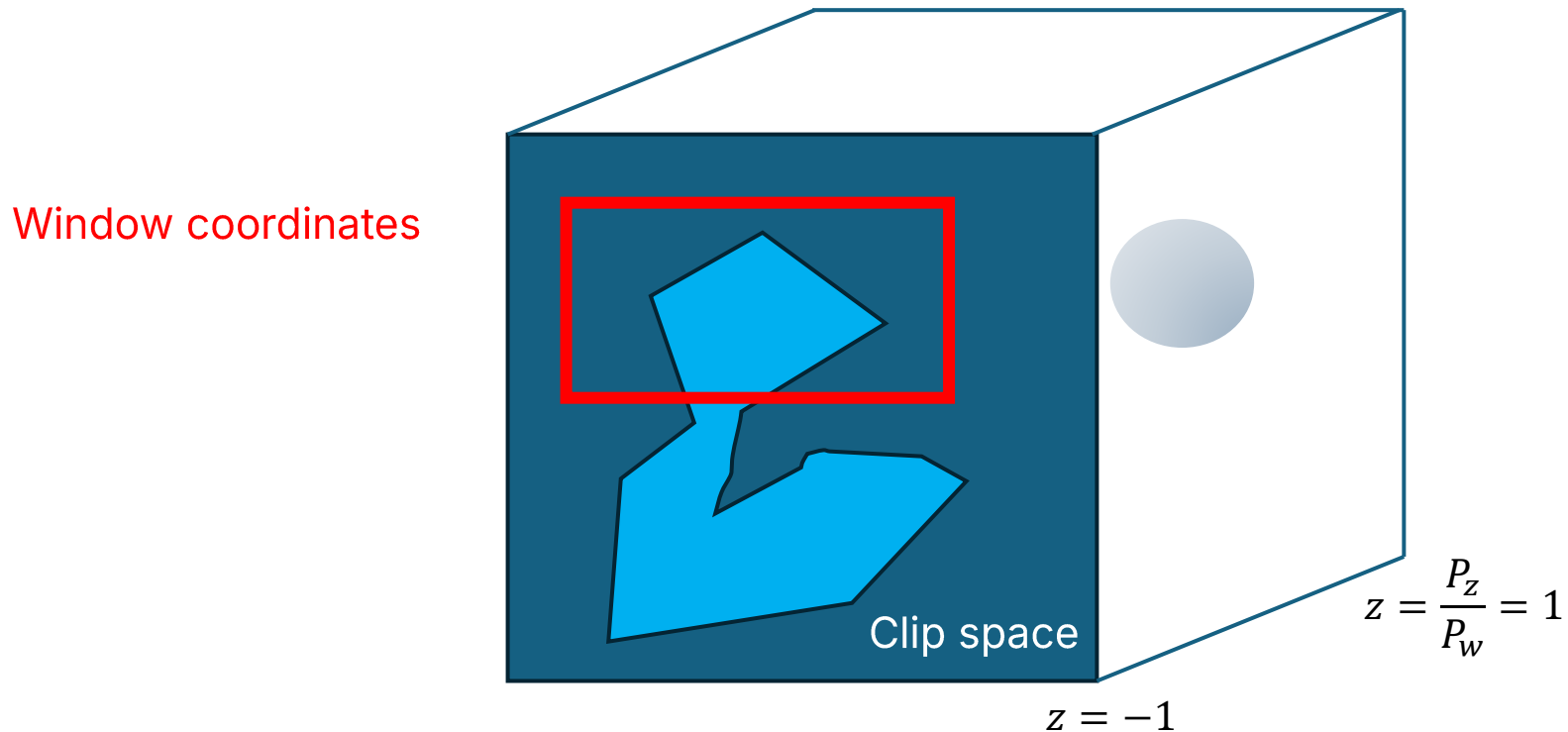
- The projection matrix M_f send points in clip coordinates
 - All the point outside on the frustum are *clipped*
- By dividing by w the clip coordinates, we have the projection of our point on the screen, the Normalized Device Coordinates (NDC)

$$\begin{pmatrix} P_x \\ P_y \\ P_z \\ P_w \end{pmatrix} \mapsto \left(\frac{P_x}{P_w}, \frac{P_y}{P_w}, \frac{P_z}{P_w} \right)^T$$

- Notice that in NDC, we still have a (pseudo-)depth value P_z/P_w

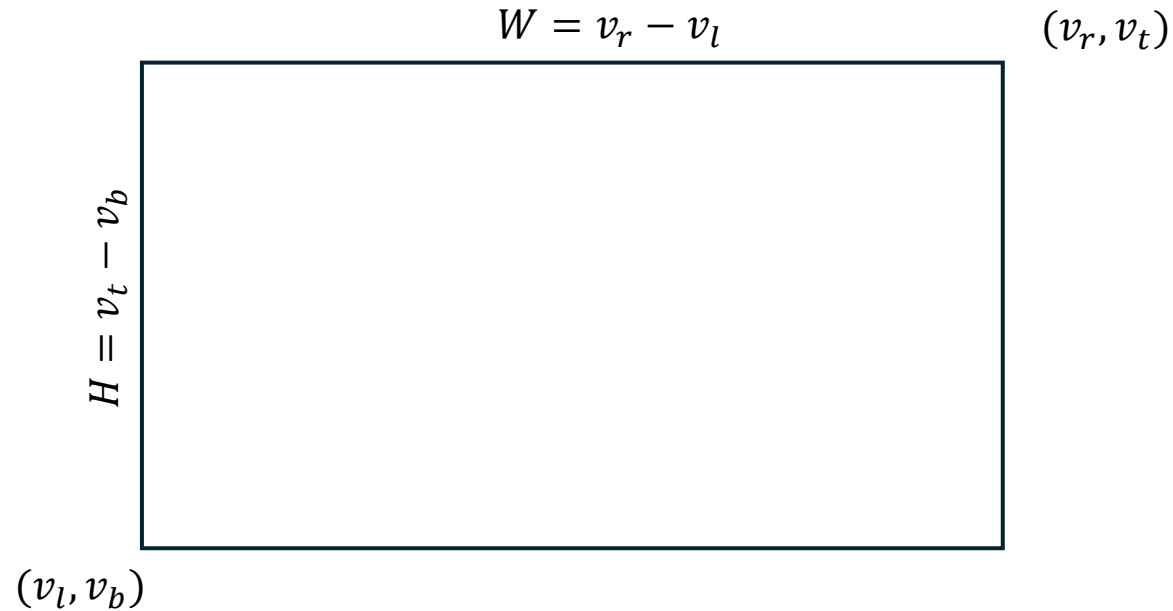
The Viewport

- The screen is not a square.
- We need to display the front face of our cube to the screen.
- It would be even nicer to display a part of it on the screen



The Viewport

- The Viewport Transformation is composed of 2 stages:
 1. A scaling to the shape of the viewport
 2. A transformation to the position of the viewport
- The Viewport is a rectangle between (v_l, v_b) and (v_r, v_t)



The Viewport – scaling to the viewport

- The Viewport Transformation is composed of 2 stages:
 1. **A scaling to the shape of the viewport**
 2. A transformation to the position of the viewport
- Our cube has a width, height and depth of **2** (cube from $(-1, -1, -1)^T$ to $(1, 1, 1)^T$)
- We scale it to $\left[-\frac{v_r - v_l}{2}, \frac{v_r - v_l}{2}\right] \times \left[-\frac{v_t - v_b}{2}, \frac{v_t - v_b}{2}\right] \times \left[-\frac{1}{2}, \frac{1}{2}\right]$, by dividing by the current size and multiply by the desired size, a simple matrix:

$$S = \begin{pmatrix} \frac{v_r - v_l}{\text{Old size}} & 0 & 0 & 0 \\ 0 & \frac{v_t - v_b}{\text{Old size}} & 0 & 0 \\ 0 & 0 & \frac{1}{\text{Old size}} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

 Old size
 New size

The Viewport – scaling to the viewport

- The Viewport Transformation is composed of 2 stages:
 1. A scaling to the shape of the viewport
 2. **A transformation to the position of the viewport**
- The scaled cube corners are not in $[0, 1]$ (but in $\left[-\frac{W}{2}, \frac{W}{2}\right] \times \left[-\frac{H}{2}, \frac{H}{2}\right] \times \left[-\frac{1}{2}, \frac{1}{2}\right]$)
- Center the origin (add $\frac{v_r - v_l}{2} = \frac{W}{2}$ to x and $\frac{v_t - v_b}{2} = \frac{H}{2}$ to y)
- Move the origin to the position of the viewport (add x_v to x and y_v to y)
- The z coordinate is in the range $\left[-\frac{1}{2}, \frac{1}{2}\right]$ it has to be shifted by $+\frac{1}{2}$

$$T = \begin{pmatrix} 1 & 0 & 0 & x_v + \frac{v_r - v_l}{2} \\ 0 & 1 & 0 & y_v + \frac{v_t - v_b}{2} \\ 0 & 0 & 1 & \frac{1}{2} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



Identity rotation/scaling

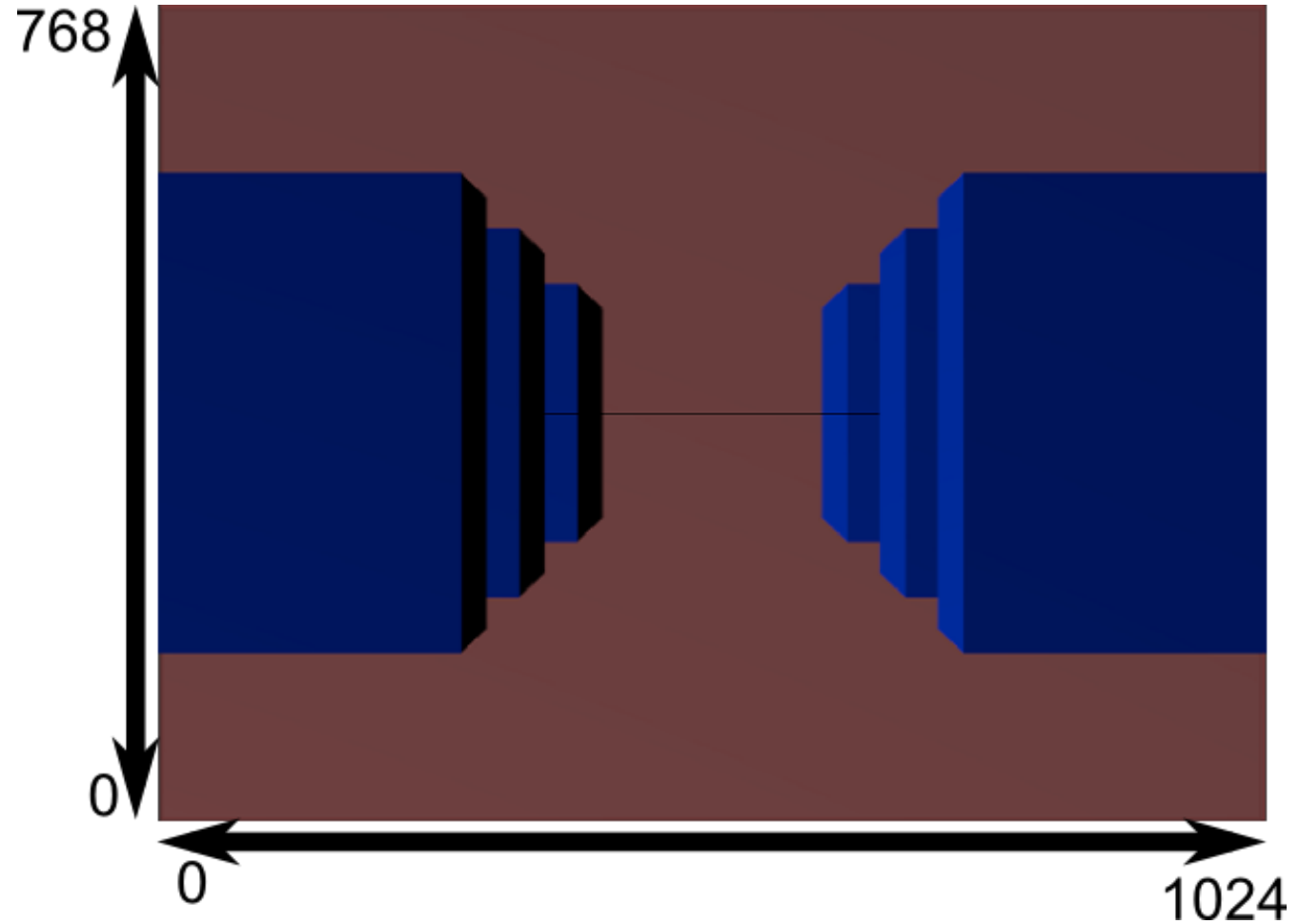
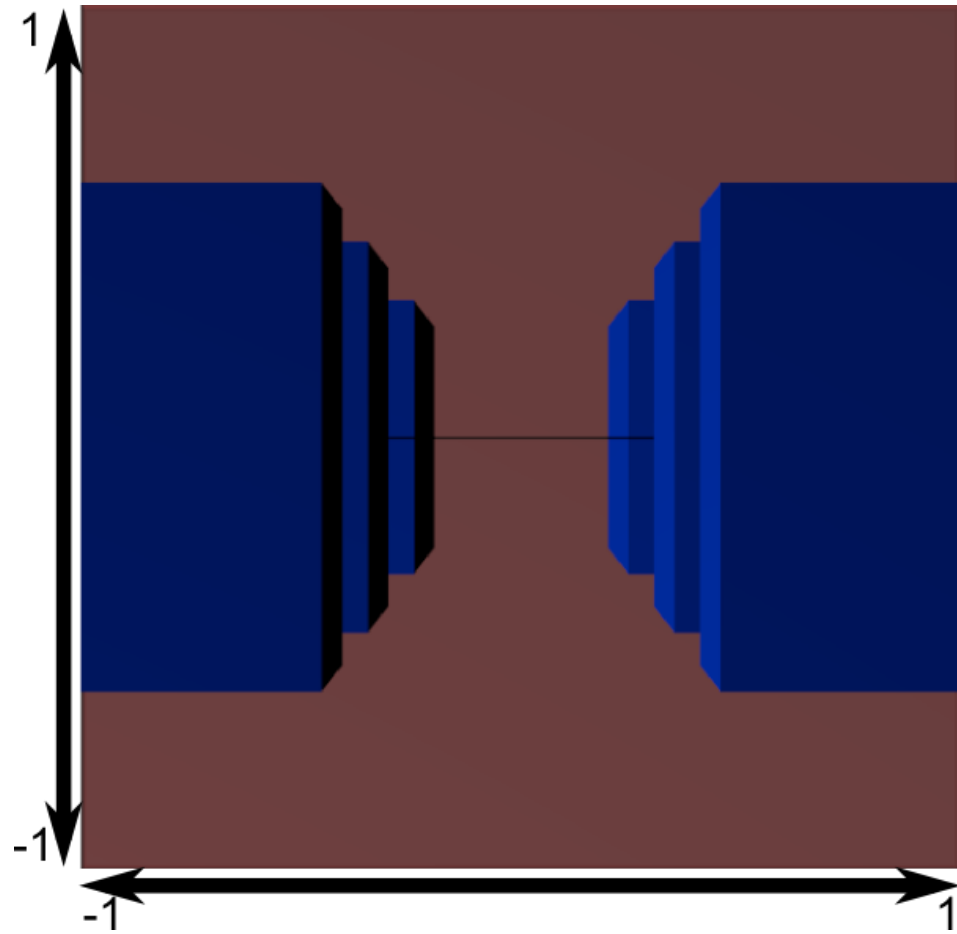
The Viewport – Final viewport

- The Viewport Transformation is composed of 2 stages:
 1. **A scaling to the shape of the viewport**
 2. **A transformation to the position of the viewport**
- The final viewport is a composition of the two matrices:

$$V = ST = \begin{pmatrix} \frac{v_r - v_l}{2} & 0 & 0 & 0 \\ 0 & \frac{v_t - v_b}{2} & 0 & 0 \\ 0 & 0 & \frac{1}{2} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & x_v + \frac{v_r - v_l}{2} \\ 0 & 1 & 0 & y_v + \frac{v_t - v_b}{2} \\ 0 & 0 & 1 & \frac{1}{2} \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} \frac{v_r - v_l}{2} & 0 & 0 & x_v + \frac{v_r - v_l}{2} \\ 0 & \frac{v_t - v_b}{2} & 0 & y_v + \frac{v_t - v_b}{2} \\ 0 & 0 & \frac{1}{2} & \frac{1}{2} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

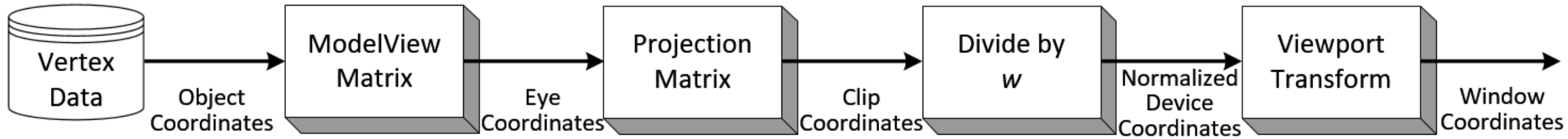
The Viewport – Final viewport

- We are now in **windows coordinates**



Summary of the transforms

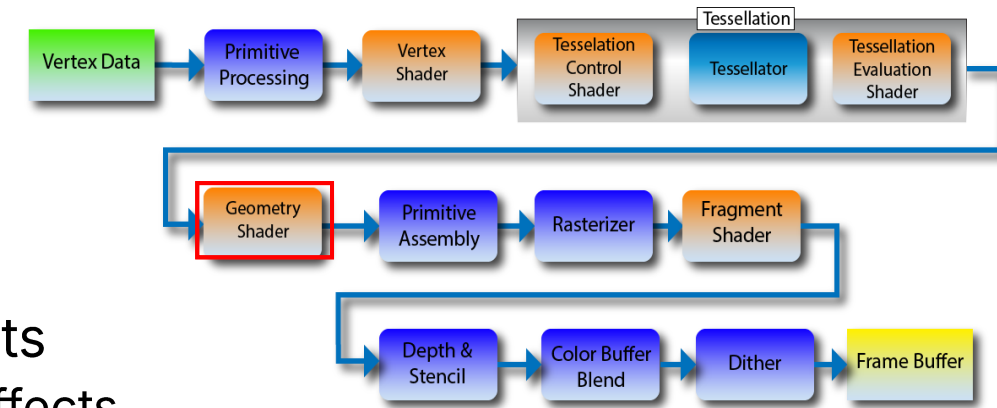
- In the vertex shader we apply successively:



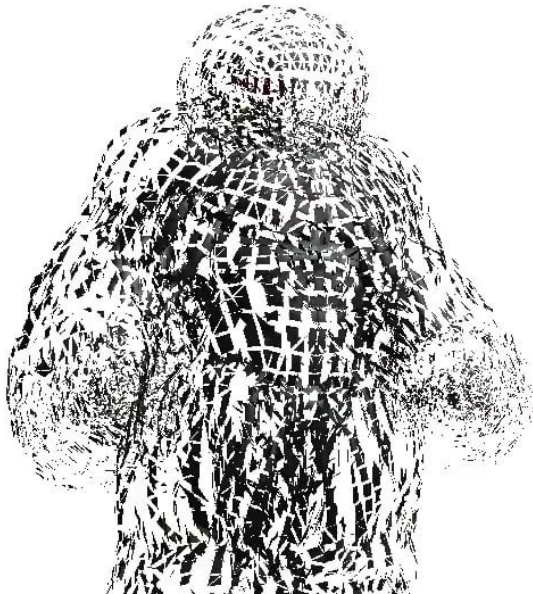
- The vertices are in their final position..
- Or are they?

Summary of the transforms

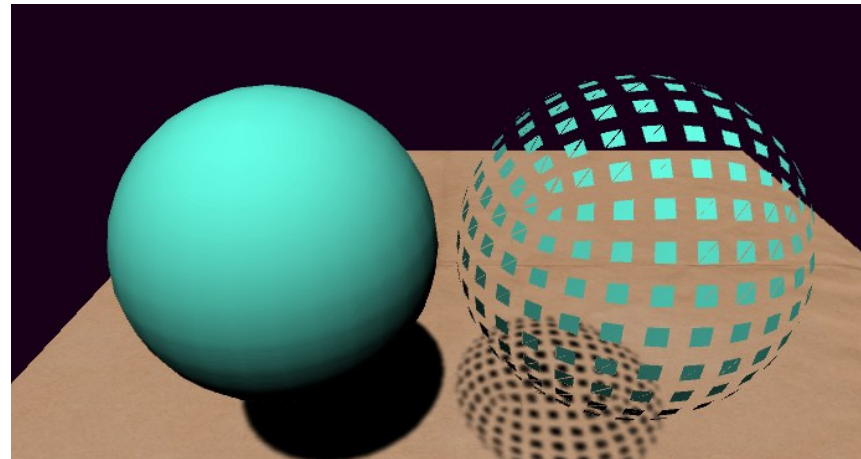
- The vertices are in their final position..
- Or are they?
- We can still use the geometry shader to get some effects
 - This shader can still move the coordinates to reach some effects
 - But it also can create new primitives from old ones (e.g., a point become a two points)



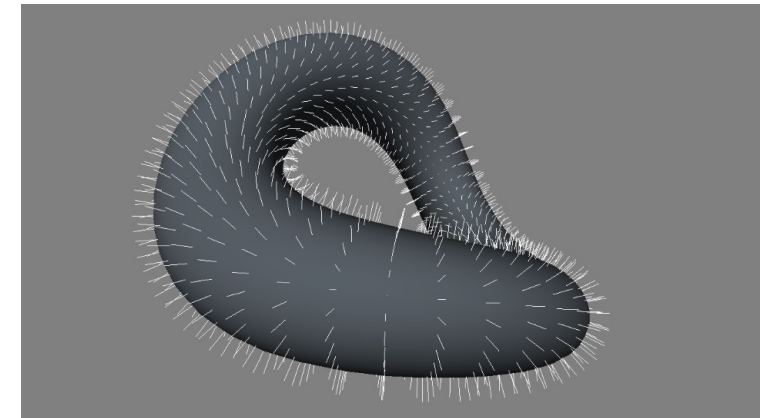
Explosion effect
We shift all vertices
using the normal



Shrinking primitives



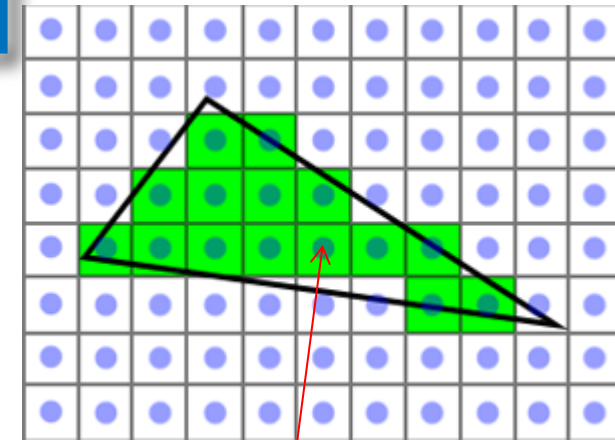
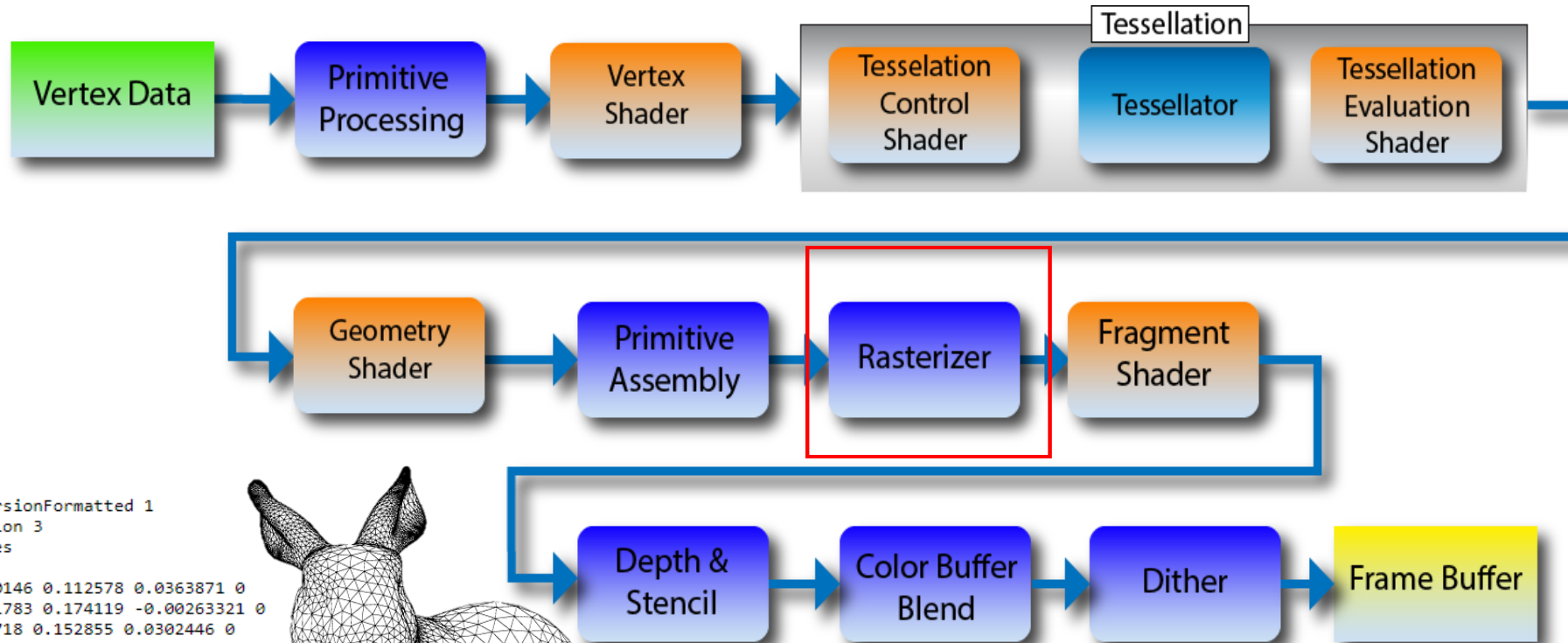
Displaying normals



OpenGL Pipeline – Rasterization

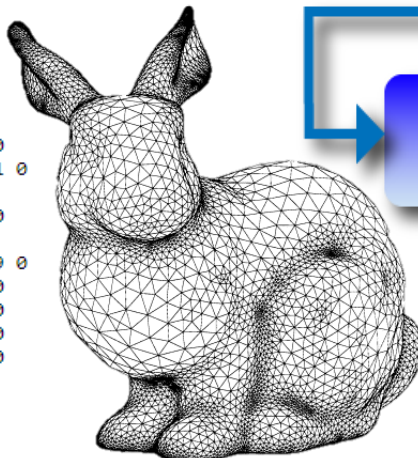
triangles (primitives)
Are broken down into
Fragments

Computer Screen
=
array of pixels



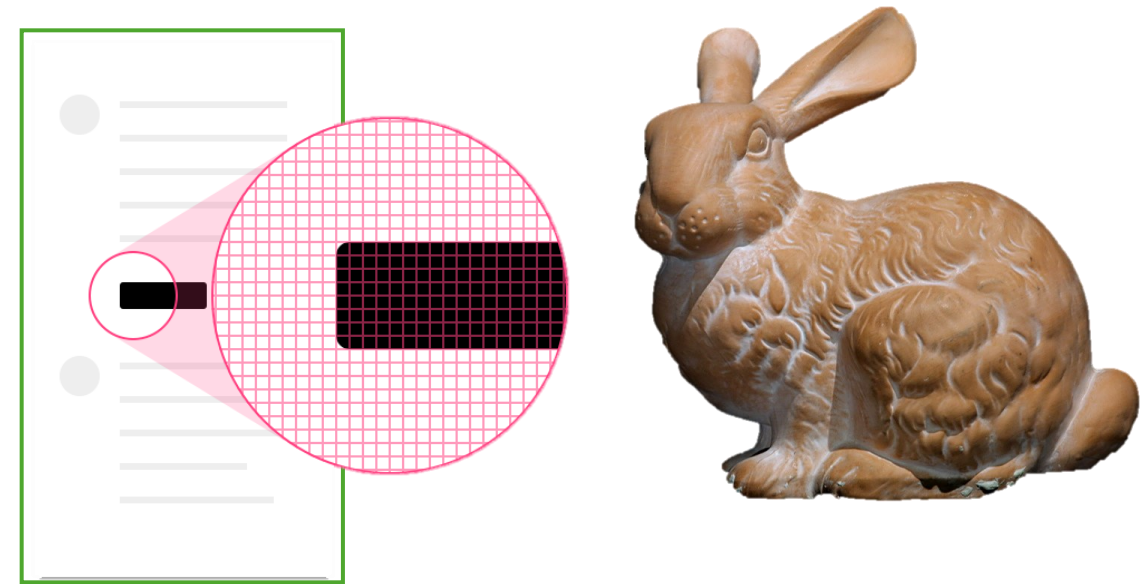
Fragment

```
MeshVersionFormatted 1
Dimension 3
Vertices
5433
-0.0260146 0.112578 0.0363871 0
-0.0321783 0.174119 -0.00263321 0
-0.080718 0.152855 0.0302446 0
-0.0231294 0.112186 0.0386436 0
0.0164928 0.122293 0.0314234 0
-0.0248365 0.156574 -0.00377469 0
0.0452628 0.0863563 0.0200367 0
-0.071339 0.155715 0.00712639 0
-0.0100758 0.125949 0.0297379 0
-0.0720133 0.154518 0.0303984 0
```



What is rasterization

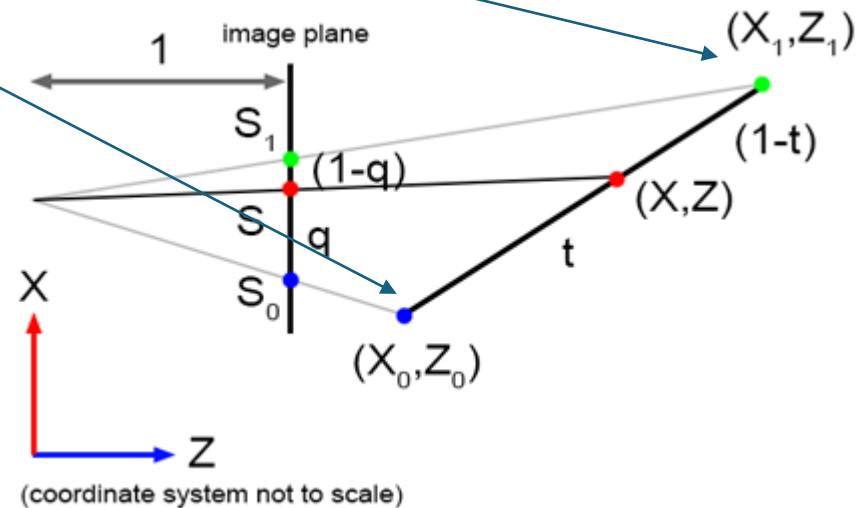
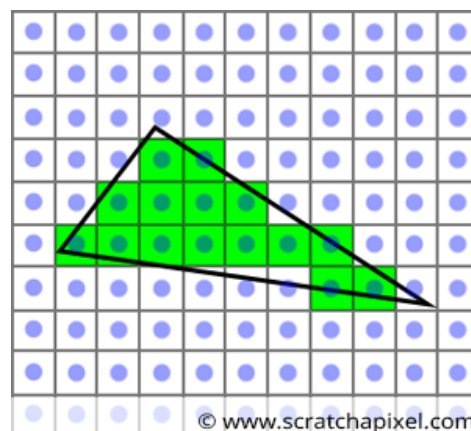
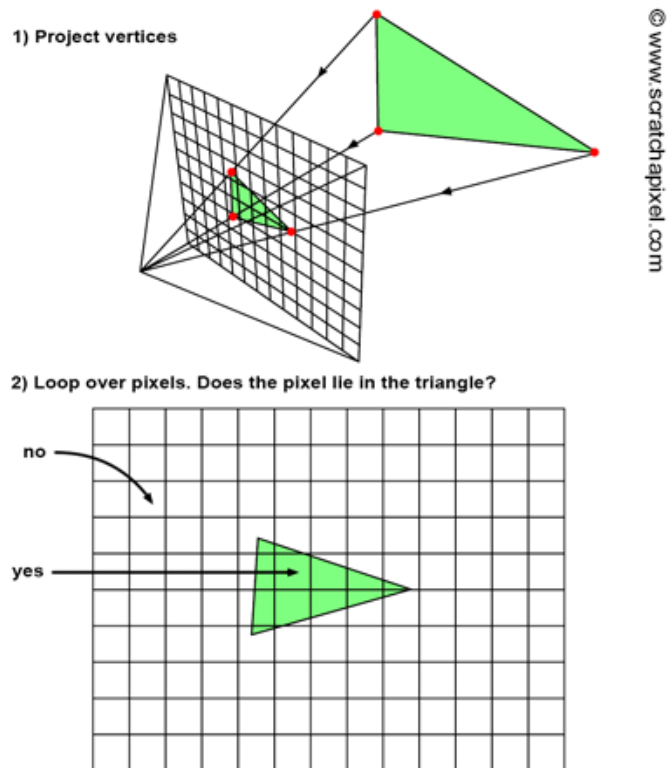
- The screen is a grid of pixels (discrete)
- The 3D objects are continuous in space



- We need to map the continuous object into the discrete grid.
- Those small elements are fragments of the object.

Rasterization – How many fragments ?

- The projection of 3D elements change using the depth of the points!
 - Less fragments for far away points
 - More fragments for closer points



© www.scratchapixel.com

What I Expect for the exam

- After this lesson I expect that:
 - Are able to explain what the Model, View, Projection matrices do
 - You can rederive the projection matrix
 - Know how rasterization works

Next time

- The rendering equation, lights and Raytracing

